Lecture 6: Reinforcement Learning: Learning to Act from Rewards

Every model in this course so far has been trained on examples of the right answer. A language model is shown the next token of a real text, and a chat model in fine-tuning is shown an answer that a person wrote for the prompt. This is supervised learning, and in the second case it is the imitation of an expert: the data say, for each situation, what the expert did.

People rarely learn a skill this way alone. A chess player can study a book of solved puzzles, a learner driver can watch recordings of good drivers, and a student can read the answer key of last year's exam. Each of these teaches something, but only about the situations the solutions cover, and none says what happens after the learner's own mistakes. Most of the skill comes from practice: playing games and seeing how they end, driving while an instructor reacts, taking a practice test and getting it back marked. The learner acts, and the world responds to that action.

The two kinds of feedback differ in what they say. *Instructive* feedback names the right action, as a label does. *Evaluative* feedback says only how good the action taken was: a score, a win or a loss, a mark on the test. **Reinforcement learning** is learning to act from evaluative feedback: an agent receives a number, the *reward*, for its own actions. The reward does not say which action would have been best. It may come long after the actions that earned it, and the agent learns only about the actions it tries (§1).

The methods of this lecture rest on three assumptions, two about the environment and one about the goal. The first is that the environment has a *state* that summarizes the past: what happens next depends only on the current state and the action, not on how the agent got there. This is the Markov property. The second is that the environment is *stationary*: the probabilities with which an action leads from one state to another, and the rewards it earns, do not change over time, so experience gathered now predicts what will happen later. The third is the reward hypothesis of §1: the goal can be written as the *sum* of the rewards, so credit for a result can be divided among the steps that led to it. Together these define a Markov decision process (§3).

The methods follow from one another. An agent that knew how its world works could compute how good each state is and act on that, which is dynamic programming (§6). It rarely knows, so it estimates the same values from its own experience, by averaging what followed (Monte Carlo) or by learning from its own next prediction (temporal differences), and it must try actions it is unsure of to learn their values at all (§§7–9). A table of values cannot cover a game's positions, and a network in its place makes learning unstable, which deep Q-networks overcame (§10). Acting greedily on values cannot express a random choice, or choose well among thousands of actions, so Part 3 learns the policy directly. Its gradient can be estimated from the agent's own samples, but noisily, and a baseline, a learned critic, and PPO's limit on each step make it usable (§§11–14). The systems that worked added more: a model to plan with, demonstrations to start from, and, for language models, a reward learned from people (§§16–19).

The lecture has four parts:

1. reinforcement learning and its terminology (§§1–5): the agent and its environment; a short history; Markov decision processes; values and the Bellman equations; a map of the methods;
2. value-based learning (§§6–10): dynamic programming; Monte Carlo and temporal-difference learning; exploration; SARSA and Q-learning; function approximation and deep Q-networks;

3. policy-based learning (§§11–15): the policy gradient and REINFORCE; baselines and advantages; actor–critic methods; trust regions and PPO; and the methods of Parts 2 and 3 in equations (§15);
4. what worked, and where it falls short (§§16–20): planning with models; learning from demonstrations; the landmarks and the limits; from games to language models.

Parts 2 and 3 are the core; §20 sums up and looks ahead to next week.

1. What reinforcement learning is

Reinforcement learning is easiest to see first in examples, informally. The formal definitions follow in §3 (Markov decision processes) and §4 (values).

The agent and its environment

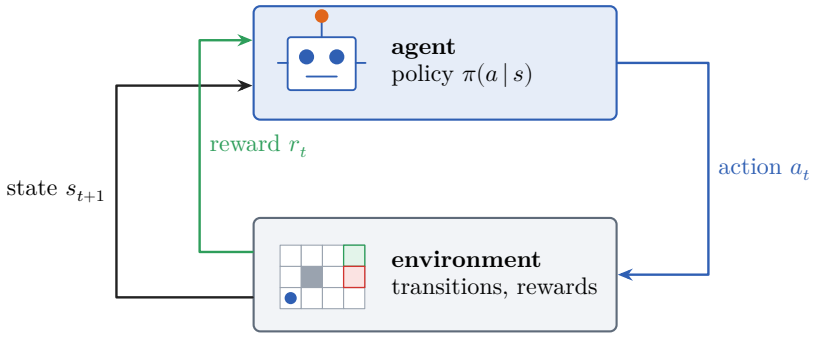


Figure 1. An agent acts; its environment answers with a reward and a new state. The loop of reinforcement learning, after Sutton and Barto (2018), Fig. 3.1. At step t the agent, whose behavior is its policy π (blue), takes action a_t; the environment returns a reward r_t (green) and moves to the next state s_{t+1}, from which the agent chooses again. Colors in this lecture: the policy and the actions it chooses blue, rewards and returns green, values and their estimates orange, the environment and anything held fixed gray, and what goes wrong red.

The learner is called the **agent**, and everything it interacts with is its **environment**. The two take turns, in discrete steps $t = 0, 1, 2, \ldots$ (Figure 1):

1. the agent observes the *state* of the environment, s_t;
2. it chooses an *action*, a_t;
3. the environment responds with a *reward* r_t, a single number, and moves to a new state s_{t+1}.

Then the loop repeats from the new state.

Where the line between agent and environment falls is a choice, and a useful rule is that the agent is the part we have complete control over (Sutton and Barto, 2018, Section 3.1). Gonkee's introductory video (Gonkee, 2024) makes the same point with a game of chess: the agent is whatever picks the moves, and the board, the rules, and the opponent are all environment. For a robot, the agent is the controller that sends commands to the motors; the hand and what it holds are environment, because a command to a motor does not fix exactly where a finger ends up. For a car, the agent is the software that turns the wheel and works the pedals, and the road and the traffic are environment. Figure 2 shows the three tasks in these terms.

A few more terms recur from here on, and §§3–4 define each precisely.

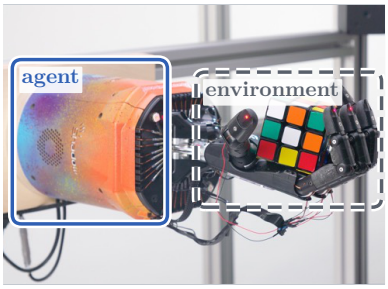
- The agent's rule for choosing actions is its *policy*, $\pi(a \mid s)$, the probability of each action in each state. A policy that always takes the same action in a given state is *deterministic*.

(a) a game of chess



state: the board position
action: a move
reward: +1 for a win, −1 for a loss, at the end

(b) a robot hand turns a cube



state: joint angles and the cube's pose
action: a command to each joint's motor
reward: +1 each time a face is turned to the goal

(c) a car that drives itself



state: camera and sensor readings
action: steering, throttle, brake
reward: progress along the route, a large penalty for a collision

Figure 2. Three tasks as an agent and an environment. In each photograph the agent is outlined in blue and the environment in gray. (a) A game of chess: the player whose moves are chosen is the agent; the board and the opponent are the environment. (b) A robot hand turning a puzzle cube: the controller, which sends commands into the hand's orange forearm, is the agent; the hand and the cube are the environment. (c) Inside a car that drives itself: the driving software, which turns the steering wheel, is the agent; the road, the other cars, and everything around them are the environment. Below each photograph: what the state, the action, and the reward are. (a) Photo: Nokedlyy, CC0, Wikimedia Commons. (b) Reproduced from OpenAI et al. (2019), Fig. 1. (c) Photo: 9yz, CC BY-SA 4.0, Wikimedia Commons, outlines added.

- Many tasks come in *episodes* that end: a game ends in a win or a loss, an attempt at a task in success or failure, an answer with its last token. Others, such as controlling the temperature of a building, go on without end.
- The agent's goal is a large total of rewards over time, called the *return*, rather than a large reward at the next step. A chess player gives up a piece to win the game, and a robot crouches before it jumps.
- A *value function* says how much return the agent can expect from a state, or from taking an action in a state, if it keeps following its policy.
- A *model* of the environment, or *world model*, predicts what the environment will do: the next state and the reward that follow an action.

Three kinds of learning

Lecture 1 (Section 3) introduced the kinds of machine learning (Figure 3). *Supervised learning* is given inputs paired with the right outputs and fits a model that maps one to the other; the fine-tuning of Lecture 5 (Section 13) was supervised. *Unsupervised learning* is given data without labels and finds structure in it; its *self-supervised* form makes the labels from the data, as a language model does when it predicts each next token (Lectures 2–5). *Reinforcement learning* is given neither the right outputs nor, usually, a fixed dataset. The agent acts, and the environment scores what it did. This changes the problem in three ways.

1. **Evaluation without instruction.** A reward is evaluative feedback, in the sense of the opening: it says how good the chosen output was, not what it should have been, so to learn that another action would have earned more, the agent has to try that action.

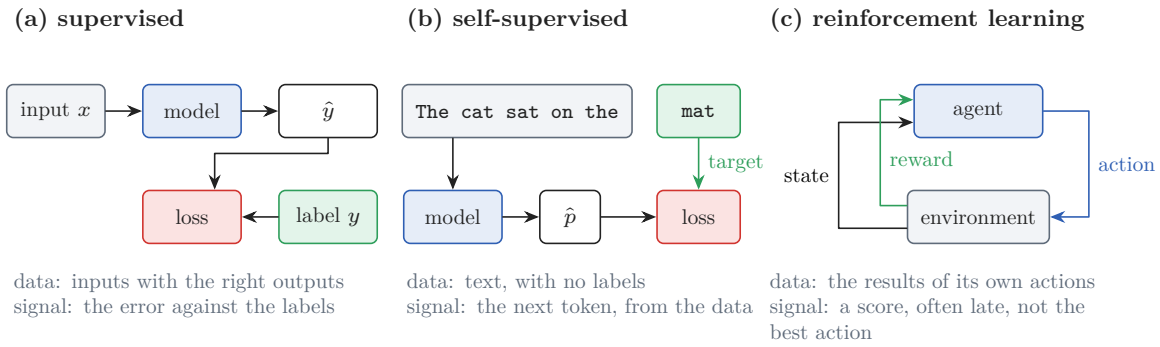


Figure 3. Three kinds of learning differ in what the data say. (a) Supervised learning: inputs paired with the right outputs, and a loss that compares the prediction with them. (b) Self-supervised learning: no labels, only data, whose structure supplies the signal; a language model predicts each next token of a text, and a loss compares its prediction $\hat{p}$ with the token that came next. (c) Reinforcement learning: the agent's own actions, and a reward that scores them, possibly much later; no one says what the best action was.

2. **Delay.** A move in chess is rewarded only when the game ends, many moves later. Deciding which of the moves deserve the credit for the result is the *credit assignment* problem (§§7, 12–13).
3. **Data that depend on the actions.** A supervised learner is trained on a fixed dataset. An agent sees only the results of what it chose to do, so it must *explore*, trying actions it is unsure of, while it *exploits* what it already knows (§8). Every change to its policy changes the data it collects next (§14).

Rewards

A reward is a single number that the environment produces after each action. It is part of the environment, not of the agent: the agent cannot change how rewards are computed, only which actions it takes. The whole field rests on one assumption about what a reward can express.

Reward hypothesis (in the form of Sutton and Barto, 2018, Section 3.2, paraphrased). *Every goal that we would want an agent to pursue can be expressed as the maximization of the expected value of the cumulative sum of a scalar reward.*

The hypothesis is not a theorem but a working assumption, and it puts the burden on the designer, who must write the goal as a reward. Two choices recur.

- **Sparse or dense.** A *sparse* reward is 0 almost everywhere and nonzero only at a goal: +1 for winning a game, 1 for a correct answer. It says exactly what is wanted, but the agent may act for a long time before it earns anything to learn from (§18). A *dense* reward arrives at every step, such as a robot's forward speed or the distance still to go. It is easier to learn from, and easier to get wrong.
- **Shaping.** A sparse reward can be made denser by adding a bonus for progress. An arbitrary bonus can change which policy is best, and an agent will then pursue the bonus instead of the goal. A bonus that is the change, from one state to the next, of some function Φ of the state never does: it changes every return from a state by the same amount, whichever actions follow, and so leaves the best policies unchanged (Theorem 8, Appendix B.0).

The reward specifies what the agent should achieve, not how. An agent maximizes the reward it is given, and when that reward differs from the designer's intention, the agent finds the difference (§18).

States and observations

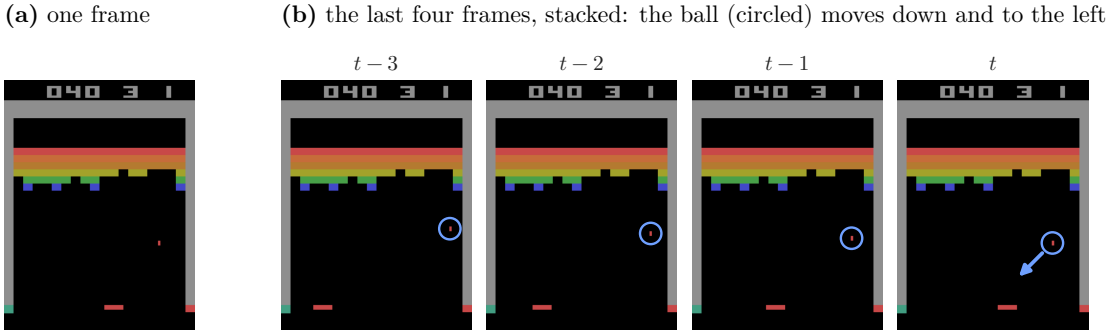


Figure 4. One observation can leave out what matters; a short history restores it. Breakout, rendered by the Arcade Learning Environment (Bellemare et al., 2013) at its 210 × 160 pixels. (a) One frame shows where the ball is, but not where it is going. (b) The last four frames, one per step of the agent (each step repeats its action for four frames of the game), show its direction (the ball circled; blue arrow). Deep Q-networks (§10) give their network the last four frames, reduced to 84 × 84 pixels in gray scale, for this reason.

The state should contain everything about the environment that matters for what happens next. Often the agent cannot see all of it. A poker player does not see the other players' cards, a robot sees its surroundings through a camera, and an Atari player sees one frame of the screen, which shows where the ball is but not which way it is moving (Figure 4a). What the agent receives is then an *observation* rather than the state, and the environment is *partially observable*. An agent that acts on its latest observation alone can be misled, so it builds a state of its own from the history of its observations. It can stack the last few frames (Figure 4b), or read the whole history with a recurrent network or a Transformer.

2. A short history

Reinforcement learning grew from three threads that met in the 1980s (Sutton and Barto, 2018, Section 1.7), and deep networks made it work at scale in the 2010s (Figure 5).

1. **Trial and error.** Thorndike (1898) found that animals repeat the actions that are followed by a good outcome, the *law of effect*. Samuel (1959)'s checkers program learned the same way, by playing against itself.
2. **Optimal control.** Bellman (1957) described sequential decisions as Markov decision processes and gave the recursion for their values that bears his name (§4).
3. **Temporal differences.** Learning a prediction from the next prediction, already in Samuel's program, became temporal-difference learning (Sutton, 1988) and, combined with Bellman's recursion, Q-learning (Watkins, 1989). Williams (1992) gave the policy-gradient method REINFORCE.

The first striking success was TD-Gammon at backgammon in the early 1990s (Tesauro, 1995). Twenty years later, deep networks learned Atari games from the pixels on the screen (Mnih et al., 2015), and AlphaGo beat the European Go champion Fan Hui (Silver et al., 2016) and, in March 2016, Lee Sedol. Agents then mastered chess, shogi, and Go from the rules alone, beat the world champions at the video game Dota 2 and reached the top level of StarCraft II, and left games: they solved a puzzle cube with a robot hand, shaped the plasma in a fusion device (Degrave et al., 2022), and raced drones (Kaufmann et al., 2023). Since 2022, rewards have

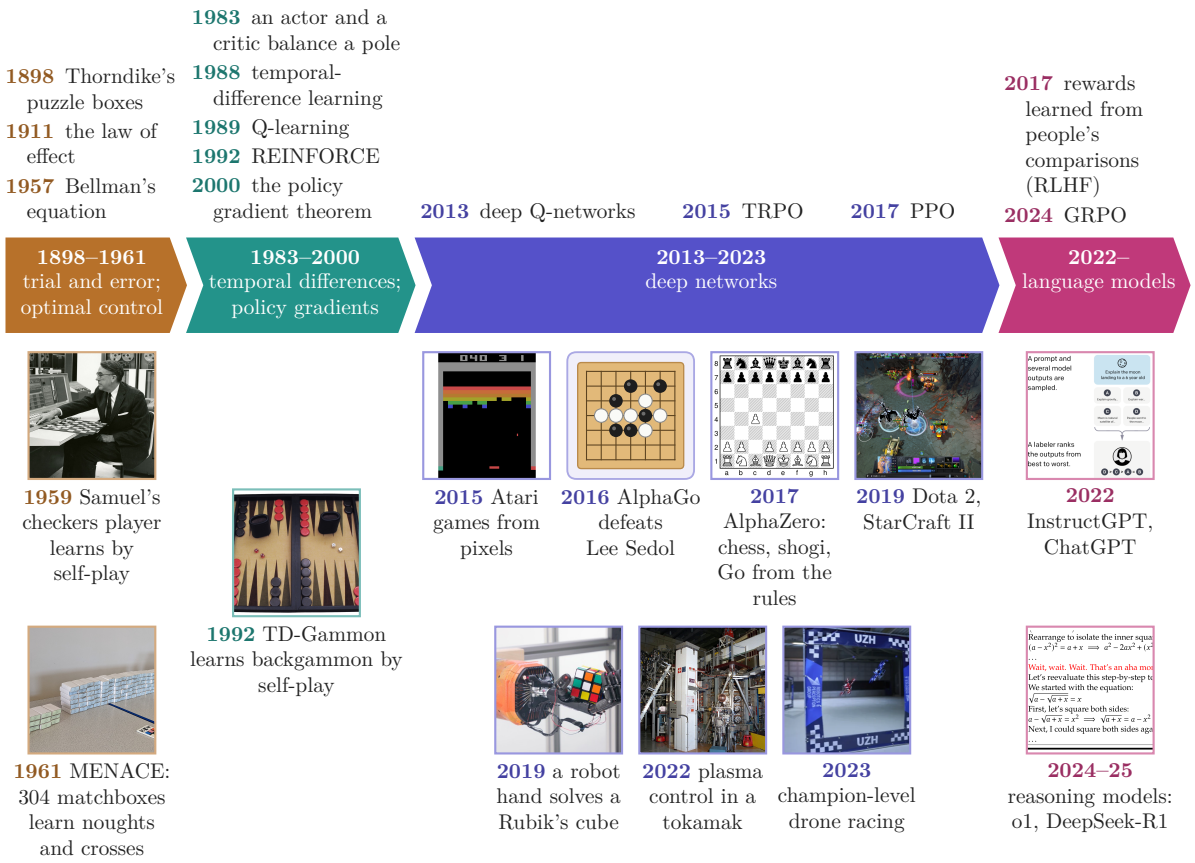


Figure 5. From puzzle boxes to reasoning models. Above the colored band, ideas and methods; below it, systems and what they achieved, in four periods. §18 explains how each system learned, and Appendix F tells the history in more detail. Pictures, in order: Samuel at an IBM 7090, photo courtesy of IBM (IBM History); a replica of MENACE, photo: Matthew Scroggs, CC BY-SA 4.0, Wikimedia Commons; backgammon, photo: Ptkfgs, public domain, Wikimedia Commons; Breakout, rendered by the Arcade Learning Environment; AlphaGo, drawn (no photograph of the match could be obtained); AlphaZero, one board of Silver, Hubert, et al. (2017), Table 2; Dota 2, reproduced from Berner et al. (2019), Fig. 11a; the robot hand, reproduced from OpenAI et al. (2019), Fig. 1; the TCV tokamak, photo: CRPP-EPFL, CC BY-SA 2.5, Wikimedia Commons; the Swift drone, reproduced from Kaufmann et al. (2023), Fig. 1b, CC BY 4.0; part of Ouyang et al. (2022), Fig. 2; part of DeepSeek-AI (2025), Table 3.

been the last stage in training chat models (Ouyang et al., 2022), and the way reasoning models learn to work through a problem before they answer (DeepSeek-AI, 2025). Barto and Sutton received the 2024 Turing Award for the foundations of the field.

3. Markov decision processes

To say what a good policy is, we need a precise description of the environment and of the agent's goal. The standard description is the Markov decision process, which turns the three assumptions of the opening into a definition.

Definition. A *Markov decision process* (MDP) is a tuple $(\mathcal{S}, \mathcal{A}, p, r, \gamma)$ with

- a set of *states* $\mathcal{S}$, with a start state s_0 (or a distribution over start states) and, in tasks that end, a set of *terminal* states, where an episode ends;
- a set of *actions* $\mathcal{A}$ (or $\mathcal{A}(s)$, the actions available in state s);

- *transition probabilities* $p(s' \mid s, a)$, the probability that action a taken in state s leads to state s';
- an *expected reward* $r(s, a, s')$: the reward r_t received after taking a_t in s_t may be random, and $r(s, a, s') = \mathbb{E}[r_t \mid s_t = s, a_t = a, s_{t+1} = s']$ is its mean;
- a *discount* γ between 0 and 1.

The transition probabilities and expected rewards are the environment's *dynamics*. They depend only on the current state and action (and the expected reward also on the next state). This is the **Markov property**,

$$\Pr(s_{t+1} = s' \mid s_0, a_0, \ldots, s_t, a_t) = p(s' \mid s_t, a_t), \tag{1}$$

and the same holds for the reward. The dynamics also do not change with t: the MDP is stationary. Anything that predicts the dynamics, exactly or approximately, is a *model* of the environment.

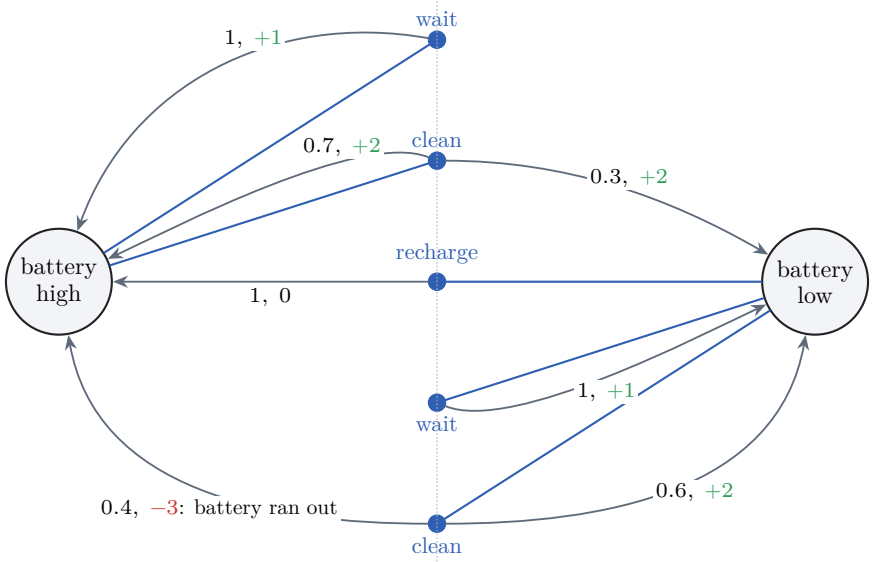
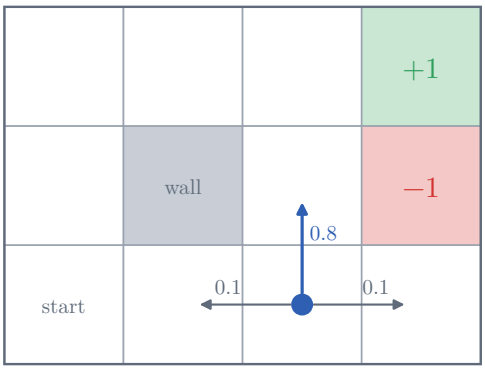


Figure 6. An MDP drawn as a graph. A robot vacuum cleaner whose battery is high or low (gray circles, the states). The two states are on the left and right, and every action is a blue dot on the vertical line between them. Blue lines join each state to the actions available there; gray arrows lead from each action to the states it can reach, labeled with the probability and the reward. Cleaning earns +2, but with a low battery it runs the battery out 4 times in 10, and the robot must be carried to its charger (−3). After Sutton and Barto (2018), Example 3.3.

A small MDP can be drawn as a graph (Figure 6). A robot vacuum cleaner has two states, its battery high or low. With a high battery it can clean, which earns +2 and may run the battery low, or wait, which earns +1. With a low battery it can also recharge. The question an MDP poses is what the robot should do in each state to collect the most reward over time: clean on a low battery and risk running out, or recharge first.

Figure 7 shows a second MDP, which the next sections return to, the 4 × 3 world. An agent walks a grid from the bottom-left corner toward an exit worth +1, past an exit worth −1. Its moves slip sideways one time in five, and every step costs a little. Of the two shortest paths to +1, one passes right beside −1, where a slip can end the episode badly, so the best route from each cell is not obvious. §4 computes it.



states: the 11 open cells; the two on the right end the episode

actions: up, right, down, left

transitions: the move goes the intended way with probability 0.8, and at right angles with probability 0.1 each (shown for up from the blue dot); into the wall or an edge, it stays put

rewards: −0.04 for every step; +1 or −1 more on reaching an exit

discount: $\gamma = 1$

Figure 7. A gridworld MDP: the 4 × 3 world of Russell and Norvig (2020), Chapter 17. An agent moves between the open cells of a grid. Its actions slip: "up" from the blue dot reaches the cell above with probability 0.8, and the cells to the left and right with probability 0.1 each. Every step costs 0.04, the step into an exit included, and reaching +1 or −1 adds that reward and ends the episode.

Policies and trajectories

A **policy** $\pi(a \mid s)$ gives the probability of each action in each state. When the policy has parameters θ, such as the weights of a network, we write π_θ. An episode that runs from the start to a terminal state at step T produces a *trajectory*, $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T)$. By the Markov property (1) its probability is a product of the policy's choices and the environment's transitions,

$$p_\theta(\tau) = p(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t)\, p(s_{t+1} \mid s_t, a_t). \tag{2}$$

Only the policy's factors depend on θ. §11 uses this to compute a gradient without knowing the transitions.

Returns and discounting

The **return** from step t is the discounted sum of the rewards still to come,

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{k \geq 0} \gamma^k r_{t+k}. \tag{3}$$

A reward k steps ahead is weighted by γ^k. The discount does two jobs. In a task that never ends, it keeps the return finite: if every reward is at most $r_{\max}$ in size, the return is at most $r_{\max}/(1-\gamma)$. And it makes the agent prefer a reward sooner to the same reward later. The total weight, $1/(1-\gamma)$, is a rough horizon: with $\gamma = 0.9$ the agent looks about 10 steps ahead, and with $\gamma = 0.99$ about 100. In a task whose episodes always end, $\gamma = 1$ is allowed, and the return is then the plain sum of the remaining rewards. Splitting off the first term of (3) gives a recursion that the rest of the lecture relies on,

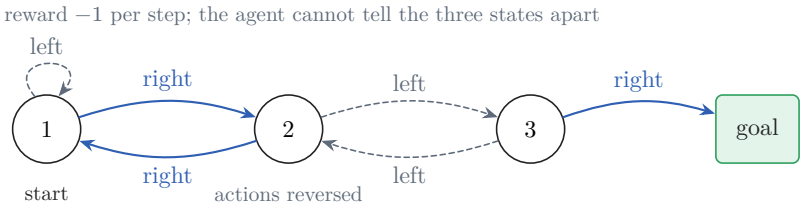
$$G_t = r_t + \gamma G_{t+1}. \tag{4}$$

What is still to come is the next reward plus what is still to come after it. The agent's *objective* is the expected return from the start, the average of the return over the trajectories its policy produces:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[G_0]. \tag{5}$$

We write states, actions, and rewards in lowercase, with r_t the reward that follows a_t, as most papers on deep reinforcement learning do. Sutton and Barto (2018) write S_t, A_t, and R_{t+1} for the same quantities.

A toy example: the short corridor



step t	0	1	2	3	4	5	6	7	
state s_t	1	1	2	1	1	1	2	3	goal
action a_t	left	right	right	left	left	right	left	right	
reward r_t	−1	−1	−1	−1	−1	−1	−1	−1	
return G_t	−8	−7	−6	−5	−4	−3	−2	−1	

Figure 8. The short corridor: three states that look alike, one of them with its actions reversed. The task of Sutton and Barto (2018), Example 13.1. Every episode starts in state 1 and ends at the goal; every step earns −1, so the return is minus the number of steps. "Right" (blue) moves right and "left" (gray, dashed) moves left, except in state 2, where they are reversed, and "left" in state 1, which stays put. The agent cannot tell the states apart, so its policy is one number, the probability p of "right". Below, one episode sampled at $p = \frac{1}{2}$: eight steps, and the return from each.

The corridor has three states and a goal (Figure 8). Every step costs 1, so the return is minus the number of steps the agent takes to reach the goal. In state 2 the actions are reversed: "right" moves left and "left" moves right. The agent cannot tell the three states apart, so it must use the same probabilities in all of them, and its policy is a single number, the probability p of "right". Always going right shuttles between states 1 and 2 forever, and always going left never leaves state 1. Only a random policy reaches the goal. §4 computes how good each p is, and §§11–14 use the corridor to compare policy-gradient methods, because its expected return can be computed exactly.

Bandits: one step, no state

The simplest case of all has a single state and episodes of one step: the agent chooses an action, receives a reward, and the episode ends. This is a *multi-armed bandit*, named after a row of slot machines, each a "one-armed bandit". With no sequence there is no credit assignment problem, and what remains is the question of which arm to pull while the agent is still learning what the arms pay. §8 uses the bandit to study exploration, and §11 to derive the policy gradient.

4. Value functions and the Bellman equations

A return (3) is known only at the end of an episode, and it varies from one episode to the next. To choose an action now, the agent needs the return it can expect from each choice. Value functions supply that expectation, and the Bellman equations make them computable.

Value functions

Under a given policy π, the **value** of a state s is the expected return from it, and the **action value** of action a in state s is the expected return if the agent takes a in s and follows π afterward:

$$V^\pi(s) = \mathbb{E}_\pi\big[G_t \mid s_t = s\big], \qquad Q^\pi(s,a) = \mathbb{E}_\pi\big[G_t \mid s_t = s,\, a_t = a\big]. \tag{6}$$

The expectations average over everything random that follows: the policy's later choices, and the environment's transitions and rewards. Their difference is the **advantage**,

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s), \tag{7}$$

which says how much better action a is than the policy's average in state s. All three depend on the policy. Because a value sums up the future of a state in one number, the agent can compare two actions by looking one step ahead, at what each leads to.

The Bellman equations

The two values are tied to each other by two one-line relations. The value of a state averages the values of the actions the policy takes there:

$$V^\pi(s) = \sum_a \pi(a \mid s)\, Q^\pi(s,a). \tag{8}$$

And the value of an action is the expected reward of one step plus the discounted value of where it leads. This follows from the recursion (4), $G_t = r_t + \gamma G_{t+1}$: given $s_t = s$ and $a_t = a$, the next state is s' with probability $p(s' \mid s,a)$, the reward then averages to $r(s,a,s')$, and the return from s' on averages to $V^\pi(s')$ by the Markov property (1). So

$$Q^\pi(s,a) = \sum_{s'} p(s' \mid s,a)\big[r(s,a,s') + \gamma V^\pi(s')\big]. \tag{9}$$

Substituting each relation into the other gives equations for one kind of value alone, the **Bellman equations** of the policy:

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s,a)\big[r(s,a,s') + \gamma V^\pi(s')\big], \tag{10}$$

$$Q^\pi(s,a) = \sum_{s'} p(s' \mid s,a)\Big[r(s,a,s') + \gamma \sum_{a'} \pi(a' \mid s')\, Q^\pi(s',a')\Big]. \tag{11}$$

Terminal states have value 0.

Figure 9 draws the equations as **backup diagrams**. The value at the top of each diagram is computed from the values below it. This step is called a backup, because it carries values back from later states to earlier ones. Every method of Part 2 is a way of computing or estimating these backups.

Matrix form. For a fixed policy, (10) is a system of linear equations, one for each non-terminal state. Collect the values in a vector V^π, the expected one-step rewards in a vector r_π with entries $r_\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s,a)\, r(s,a,s')$, and the probabilities of moving between non-terminal states in a matrix P_π with entries $P_\pi(s,s') = \sum_a \pi(a \mid s)\, p(s' \mid s,a)$. Then

$$V^\pi = r_\pi + \gamma P_\pi V^\pi, \qquad \text{so} \qquad V^\pi = (I - \gamma P_\pi)^{-1} r_\pi. \tag{12}$$

The inverse exists when $\gamma < 1$, and also when $\gamma = 1$ if the policy reaches a terminal state with probability 1, because the powers of P_π then shrink to zero. Solving the system directly costs a number of operations that grows with the cube of the number of states, which is why §6 solves it by iteration instead.

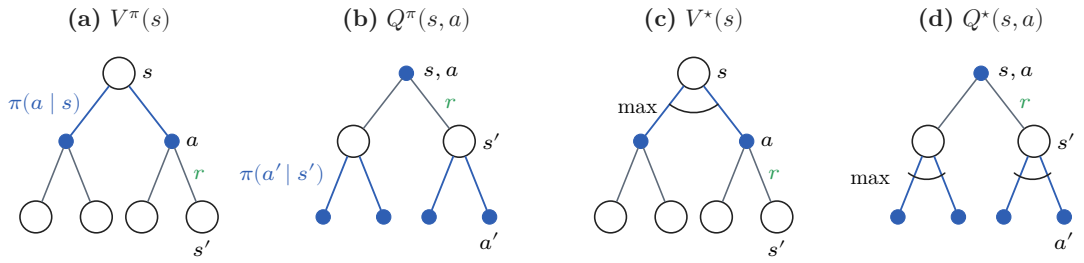


Figure 9. Backup diagrams: each value is computed from the values one step below it. Open circles are states and blue dots are actions. Read each diagram from the top: (a) the value of a state averages over the policy's actions (blue edges) and, for each, over the environment's next states and rewards (gray edges); (b) the value of an action averages over the next states and then over the policy's next actions. (c, d) The same for the best policy: the arc means the largest value is taken instead of the policy's average. After Sutton and Barto (2018), Section 3.5 and Figure 3.4.

Example: values in the corridor

In the corridor (Figure 8), every step earns -1, $\gamma = 1$, and the goal is terminal. Under the policy that goes right with probability $p = \frac{1}{2}$ in every state, state 1 moves to state 2 or stays, state 2 moves back to 1 or on to 3 (its actions are reversed), and state 3 reaches the goal or returns to 2. In the matrix form (12),

$$P_\pi = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & \frac{1}{2} & 0 \end{pmatrix}, \qquad r_\pi = \begin{pmatrix} -1 \\ -1 \\ -1 \end{pmatrix}, \qquad V^\pi = (I - P_\pi)^{-1} r_\pi = \begin{pmatrix} -12 \\ -10 \\ -6 \end{pmatrix}.$$

An episode from the start takes 12 steps on average. The action values follow from (9), $Q^\pi(s,a) = -1 + V^\pi(s')$, where s' is where a leads. "Right" is worth -11, -13, and -1 in the three states, and "left" -13, -7, and -11. So the advantages (7) of "right" are $+1$, -3, and $+5$. Going right is better than average in states 1 and 3, and worse in state 2, where the actions are reversed.

For a general p, the same algebra (Appendix A.1) gives the expected return from the start,

$$J(p) = V^\pi(1) = -\frac{2(2-p)}{p(1-p)}, \tag{13}$$

which is largest at $p^\star = 2 - \sqrt{2} \approx 0.586$, where $J = -11.66$ (Figure 10a). The best policy that cannot see the state is random. An agent that could see which state it is in would do far better: "right" in state 1, "left" in state 2 (which moves right), and "right" in state 3 reach the goal in 3 steps, a return of -3. The corridor's difficulty is that the agent cannot tell its states apart, so it must use one random choice in all three.

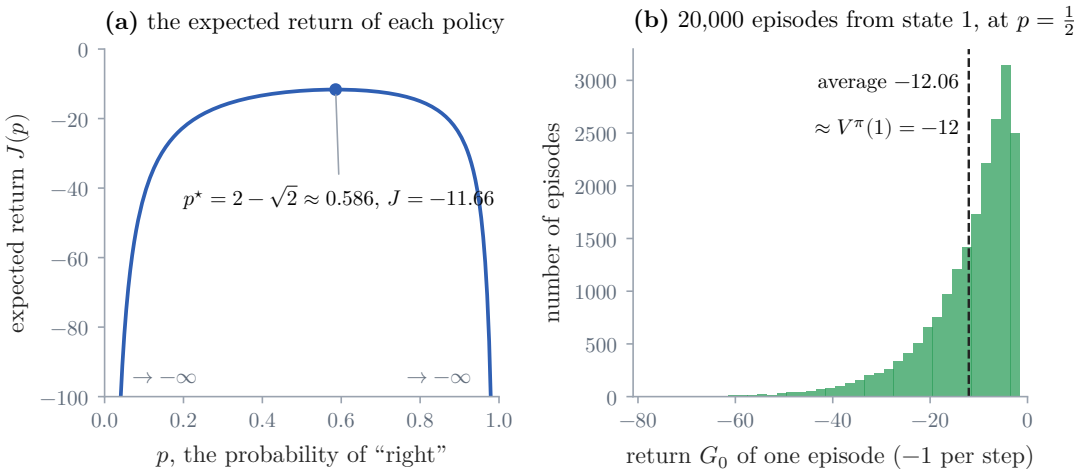


Figure 10. In the corridor the best policy is random, and the value of a state is an average of returns. (a) The expected return $J(p)$ of the policy that chooses "right" with probability p: largest at $p^\star = 2 - \sqrt{2}$, and falling without bound toward $p = 0$ and $p = 1$, where episodes never end. (b) A check of (a) by simulation: 20,000 episodes of the policy with $p = \frac{1}{2}$, each started in state 1, and how many of them had each return G_0 (one bar for every two values). Every step costs 1, so an episode of n steps has return $-n$: the shortest possible episode has 3 steps, 83% take fewer than 20, and a few take far longer (6 episodes longer than 80 steps are counted in the leftmost bar). The average return, −12.06 (dashed line), estimates the value of state 1, whose exact value is −12; averaging returns in this way is the Monte Carlo method of §7.

Optimal values

The best values any policy can reach are the **optimal values**, defined state by state:

$$V^\star(s) = \max_\pi V^\pi(s), \qquad Q^\star(s,a) = \max_\pi Q^\pi(s,a). \tag{14}$$

Nothing in this definition says that one policy reaches all these maxima at once: the best policy for one start state might, in principle, be a poor one from another. In an MDP it does not happen.

Theorem 1 (optimal policies). *In a finite MDP with $\gamma < 1$, and for episodic tasks like ours (Appendix B.6), the optimal values are the unique solution of the* **Bellman optimality equations**

$$\begin{aligned} V^\star(s) &= \max_a \sum_{s'} p(s' \mid s,a)\big[r(s,a,s') + \gamma V^\star(s')\big], \\ Q^\star(s,a) &= \sum_{s'} p(s' \mid s,a)\Big[r(s,a,s') + \gamma \max_{a'} Q^\star(s',a')\Big], \end{aligned} \tag{15}$$

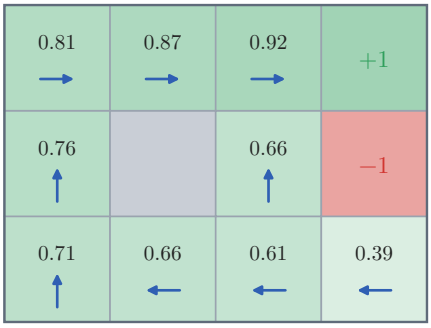
and any policy that is greedy with respect to $Q^\star$, taking in each state an action with the largest $Q^\star(s,a)$, reaches $V^\star(s)$ in every state at once. In particular, there is a deterministic optimal policy.

The optimality equations differ from (10) and (11) in one place: the maximum over actions replaces the policy's average (Figure 9c, d). Appendix B proves the theorem. The idea is that one sweep of the backup in (15) brings any two value functions closer by a factor γ, so the equations have exactly one solution, and the greedy policy's own values satisfy the same equations. The maximum makes the equations nonlinear, so they cannot be solved as a linear system, and §6 solves them by iteration.

Optimal action values say how to act without a model: in its state, the agent compares the numbers $Q^\star(s,a)$ for its actions and takes the largest. To act from $V^\star$ alone, it would need the

transition probabilities, to look one step ahead as the first equation of (15) does. The theorem needs the agent to see the state: the corridor, whose states the agent cannot tell apart, has no deterministic policy as good as its best random one.

(a) $V^\star(s)$ and the greedy action



(b) $Q^\star(s,a)$, one triangle per action; the largest in blue

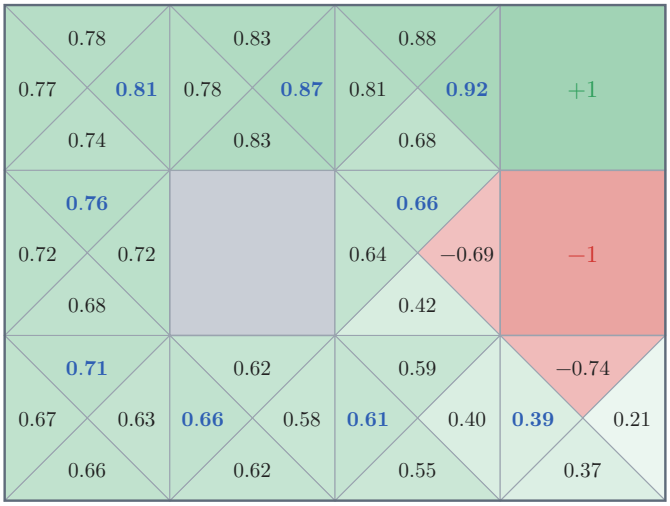


Figure 11. The optimal values of the 4 × 3 world, and the policy they imply. Computed by value iteration (§6). Green shades positive values and red negative ones, darker for larger. (a) $V^\star$ in each cell, and the greedy action (blue arrow). (b) $Q^\star$: each cell is split into four triangles, one per action, pointing the way the action moves; the largest of the four is in blue.

Figure 11 shows the optimal values of the 4 × 3 world. They are highest next to +1 (0.92) and lowest in the corner below −1 (0.39). The start is worth 0.71. The arrows show the best route. From the start the agent goes up the left side and along the top row, which is as short as the route along the bottom row and never passes beside −1. The two cells to the right of the start point left, back toward that route: from the cell left of the corner it takes 7 moves instead of 3, rather than go up past −1. The action values show why. In the cell left of −1, "right" would walk into it with probability 0.8, and its action value is −0.69. From the corner below −1, "up" has the same problem, and its action value is −0.74. Going left from the corner is worth 0.39, so the best route from there is long and safe.

5. A map of the methods

The rest of the lecture develops about a dozen algorithms. They differ along a few lines, and it helps to see them once before the details (Figure 12).

What is learned. A *value-based* method learns V or Q and acts greedily on its estimates (§§6–10). A *policy-based* method learns the policy directly and needs no values to act (§11). An *actor–critic* method learns both: a policy, called the actor, and a value function, called the critic, which helps the actor learn (§§13–14).

With a model or without. A *model-based* method can ask what would happen after an action without taking it. Dynamic programming computes values from known dynamics (§6), and AlphaZero searches ahead with the rules of the game (§16). A *model-free* method learns only from the experience it gets by acting, which is the setting of most of this lecture.

Prediction or control. *Prediction*, also called policy evaluation, estimates the values of a given policy. *Control* finds a good policy. Most control methods alternate the two: evaluate the current policy, then improve it (§6).

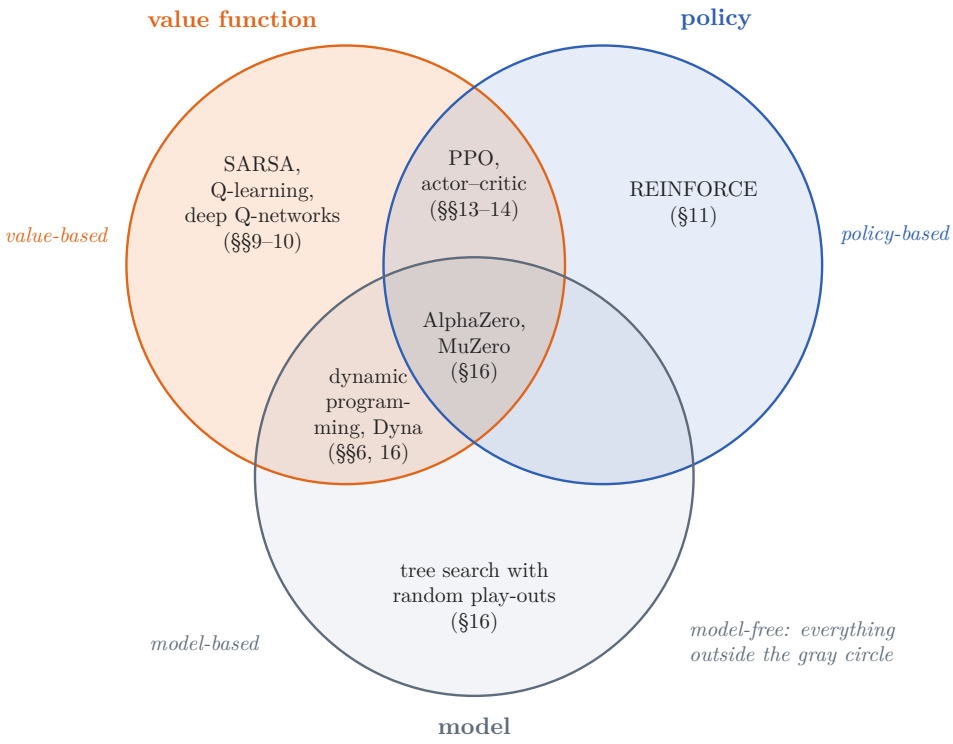


Figure 12. Methods differ in what they learn: a value function, a policy, a model of the environment, or several of these. The methods of this lecture placed by what they learn, with the sections that teach them, after the diagram in David Silver's first lecture of his course. Everything outside the gray circle is model-free.

On-policy or off-policy. Two policies can be involved in learning. The *behavior policy* is the one that chose the actions in the data. The *target policy* is the one being learned about: the policy whose values are estimated, or whose parameters are improved. An *on-policy* method needs the two to be the same, so its data must come from the policy it is learning about. An *off-policy* method can learn about the target policy from data that another policy collected. For example, an agent that explores the 4×3 world by taking a random action one time in ten can use its experience in two ways. SARSA (§9) learns the values of that exploring policy itself, so it is on-policy. Q-learning (§9) learns, from the same experience, the values of the greedy policy that never explores, so it is off-policy. Off-policy learning can reuse old data and learn from other agents' data, which saves experience, but it is harder to keep stable (§10).

Online or offline. This is a different question, about when the data are collected. An *online* method keeps acting in the environment while it learns. An *offline* method learns from a fixed dataset of past interactions and never acts while it learns (§17). An offline method must be off-policy, since its data come from whatever policy recorded them, but an online method can be either: Q-learning is online and off-policy, and SARSA online and on-policy.

Tables or approximations. A *tabular* method keeps one number for each state, or for each state and action, as in the gridworlds. When there are more states than could ever be listed, as in Atari's screens, Go's positions, or a language model's prompts, the values or the policy are *approximated* by a function with parameters, today usually a deep network. This is deep reinforcement learning (§10).

Part 2 follows the left circle of Figure 12, from dynamic programming to deep Q-networks. Part 3 follows the right circle and the overlap of the two. Part 4 adds the gray circle and learning from demonstrations, which uses no rewards at all (§17).

6. Dynamic programming

When the transitions and rewards are known, the Bellman equations of §4, (10) and (15), can be solved by computation alone, without any experience. The methods that do this are called **dynamic programming**, Bellman's name for solving a problem by solving and combining its subproblems. They need a model, and they are rarely usable as they stand. But they set out the two operations that every later method approximates: evaluating a policy, and improving it. The examples are the 4×3 world (Figure 7), whose model is known exactly.

Policy evaluation

The Bellman equation (10) can be solved directly, as in (12), but for large problems, and as a model for what follows, it is solved by iteration. **Iterative policy evaluation** turns the equation into an update. Write V_k for the estimate after k sweeps:

1. start from any values V_0, for example $V_0(s) = 0$ in every state that is not terminal;
2. sweep through the states, replacing each value by its one-step backup under π, computed from the previous sweep's values:

$$V_{k+1}(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a)\big[r(s, a, s') + \gamma V_k(s')\big]; \tag{16}$$

3. repeat until no value changes by more than a small tolerance.

In the matrix notation of (12), one sweep is $V_{k+1} = r_\pi + \gamma P_\pi V_k$.

Theorem 2 (policy evaluation converges). *For $\gamma < 1$ and any start V_0, the iterates of (16) converge to V^π, and after k sweeps the largest error is at most $\gamma^k \max_s |V_0(s) - V^\pi(s)|$.*

The idea is that the rewards cancel between the update (16) and the Bellman equation (10), and what is left is γ times an average of the old errors, so each sweep shrinks the largest error by at least that factor (Appendix B.2, and B.6 for $\gamma = 1$). In the 4×3 world, evaluating the policy "always up" takes 703 sweeps to get within 10^{-6} of its values. Under that policy the agent spends a long time slipping back and forth along the top row before it reaches an exit, and each sweep looks only one step further. A direct solve of the 9 linear equations, one for each cell that is not an exit, gives the same values at once: the start is worth -1.47 under "always up", against 0.71 under the best policy.

Policy improvement

Once the values of a policy are known, they show how to improve it. In each state, look one step ahead with the values of π and take the best action:

$$\pi'(s) = \arg\max_a \sum_{s'} p(s' \mid s, a)\big[r(s, a, s') + \gamma V^\pi(s')\big] = \arg\max_a Q^\pi(s, a). \tag{17}$$

The new policy π' is *greedy* with respect to V^π.

Theorem 3 (policy improvement). *Let π' be greedy with respect to V^π, as in (17). Then $V^{\pi'}(s) \geq V^\pi(s)$ in every state s. If $V^{\pi'} = V^\pi$, then both policies are optimal.*

The idea is that taking the greedy action once and then following π is at least as good as following π throughout, and repeating the argument at every later step replaces π by π' (Sutton and Barto, 2018, Section 4.2; Appendix B.4).

Policy iteration

Policy iteration alternates the two operations:

1. start from any policy π_0;
2. evaluate it, computing V^{π_k};
3. improve it, making π_{k+1} greedy with respect to V^{π_k}, as in (17), and keeping the action of π_k wherever it is among the best;
4. stop when the policy no longer changes.

Theorem 4 (policy iteration). *In a finite MDP, policy iteration stops after finitely many rounds, with an optimal policy.*

By Theorem 3 each round makes the policy at least as good in every state, and a finite MDP has finitely many deterministic policies, so the rounds end, and they end only at an optimal policy (Appendix B.5). In practice policy iteration needs very few rounds.

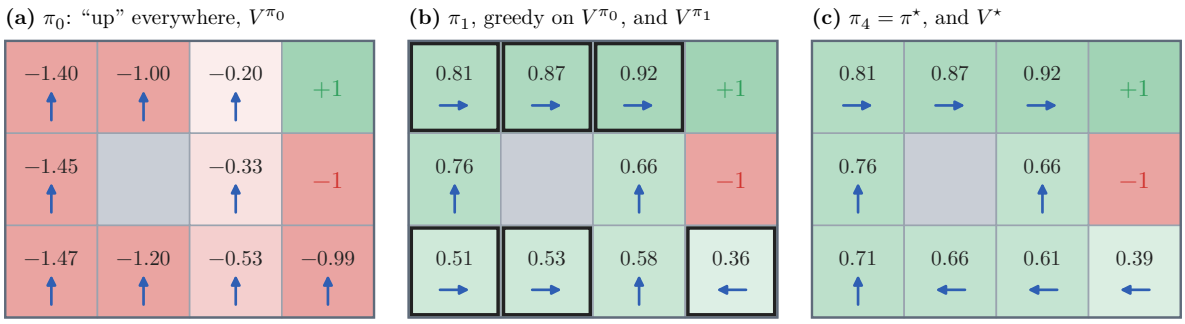


Figure 13. Policy iteration: evaluate the policy, then act greedily on its values. The 4 × 3 world, from the policy "always up". Each grid shows a policy (arrows) and its exact values (numbers, and the shading of Figure 11). (a) The starting policy and its values. (b) The policy greedy with respect to those values: six cells change their action (outlined), and every value rises. (c) After three more rounds, each of which changes one cell, the policy no longer changes: it is the optimal policy of Figure 11, and its values are $V^\star$.

In the 4 × 3 world, policy iteration from "always up" finds the optimal policy in four improvements (Figure 13). The first improvement does most of the work. Six cells change their action, and the value of the start rises from −1.47 to 0.51. The next three improvements each change one cell: the start turns up, and then the two cells to its right turn left, toward the long way round.

Value iteration

Policy iteration evaluates each policy completely before it improves it. **Value iteration** does not wait. It combines one sweep of evaluation with the improvement, by putting the maximum of (17) inside the backup. Its update is the Bellman optimality equation (15) used as an assignment:

1. start from any values V_0, for example $V_0(s) = 0$ in every state that is not terminal;
2. sweep through the states, replacing each value by its best one-step backup:

$$V_{k+1}(s) = \max_a \sum_{s'} p(s' \mid s, a)\big[r(s, a, s') + \gamma V_k(s')\big]; \tag{18}$$

3. repeat until no value changes by more than a small tolerance; then act greedily on the result.

Theorem 5 (value iteration converges). *For $\gamma < 1$ and any start V_0, the iterates of (18) converge to $V^\star$, and after k sweeps the largest error is at most $\gamma^k \max_s |V_0(s) - V^\star(s)|$.*

The argument is that of Theorem 2: a maximum moves no more than its arguments, so the backup in (18) also shrinks the largest error by the factor γ (Appendix B.2).

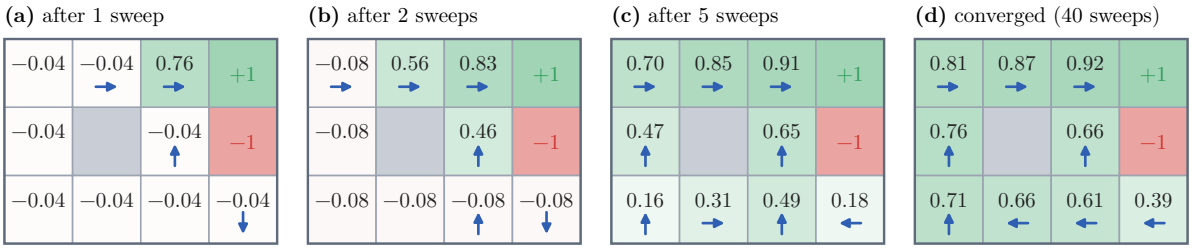


Figure 14. Value iteration spreads value outward from the rewards, one step per sweep. The 4 × 3 world, from $V = 0$ in every cell but the two exits, which hold their rewards (the same as giving the step into an exit its reward and the exit the value 0): the values after 1, 2, and 5 sweeps, and after convergence (40 sweeps, to within 10^{-10}), with the greedy action (blue arrows) where one action is best. Shading as in Figure 11.

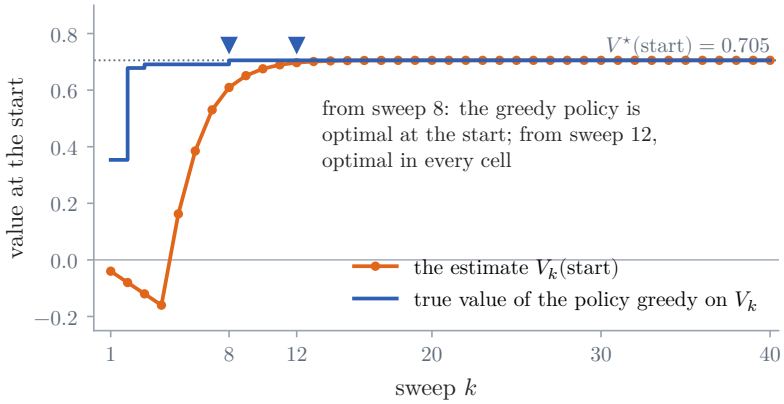


Figure 15. The greedy policy is optimal long before the values converge. Value iteration on the 4 × 3 world, as in Figure 14: the estimate V_k of the start's value after each sweep (orange), and the true value at the start of the policy that is greedy with respect to V_k (blue), computed exactly. The dotted line is $V^\star$ at the start, 0.705.

Figure 14 runs value iteration on the 4 × 3 world. The first sweep raises only the cell next to +1, while every other cell pays the cost of one step. The second sweep raises the cells two

steps from +1, and so on: each sweep carries the reward one step further back. The start's estimate turns positive after five sweeps, the length of the shortest path to +1, and converges to $V^\star$ after 40.

Unlike policy iteration, value iteration has no policy in hand while it runs. An intermediate V_k is in general not the value of any policy: V_k is the value of acting optimally for k steps and then receiving V_0, and for the first four sweeps the estimate at the start is negative, although no sensible policy is worth that little. A policy can be read off at any sweep by acting greedily on V_k, and its true value can be computed (Figure 15). That policy is already worth 0.68 at the start after two sweeps, and it is optimal at the start from sweep 8 and in every cell from sweep 12, while the values themselves take 40 sweeps to settle. Acting well needs only the order of the action values, not their exact size. The reinforcement learning explorer runs value iteration on this world and on two others, showing the arithmetic of each update.

Generalized policy iteration

Value iteration and policy iteration are two ends of one pattern. Both alternate evaluation, which makes the values agree with the policy, and improvement, which makes the policy greedy with respect to the values. Policy iteration evaluates fully between improvements, and value iteration does one sweep. Sutton and Barto (2018), Section 4.6, call the pattern **generalized policy iteration**. Almost every method in this lecture follows it, with evaluation and improvement both done approximately, from samples.

The limits of dynamic programming

Dynamic programming needs two things that are usually missing. The first is the model: every backup sums over the next states with their probabilities, which the agent rarely knows. The second is a state space small enough to sweep. A sweep costs a sum over every state, every action, and every next state. Backgammon has about 10^{20} positions and Go about 10^{170}. Learning from experience removes the first need: it replaces the sum over next states by the next state that actually happened (§7). Approximating the values with a network, which generalizes from the states it has seen to those it has not, removes the second (§10).

7. Monte Carlo and temporal-difference learning

From here on the agent is not given the model. It does not know where its actions lead or what they earn; it sees only the states it visits, the actions it takes, and the rewards it receives. The first problem is prediction: estimating the values of a fixed policy from the experience that policy produces. §§8–9 then use such estimates to improve the policy. The running example is the corridor of Figure 8 under the coin-flip policy, $p = \frac{1}{2}$, whose values were computed from the model in §4: $V^\pi = (-12, -10, -6)$ for states 1, 2, and 3. The question is how close an agent can get to these numbers by watching episodes alone.

Monte Carlo prediction

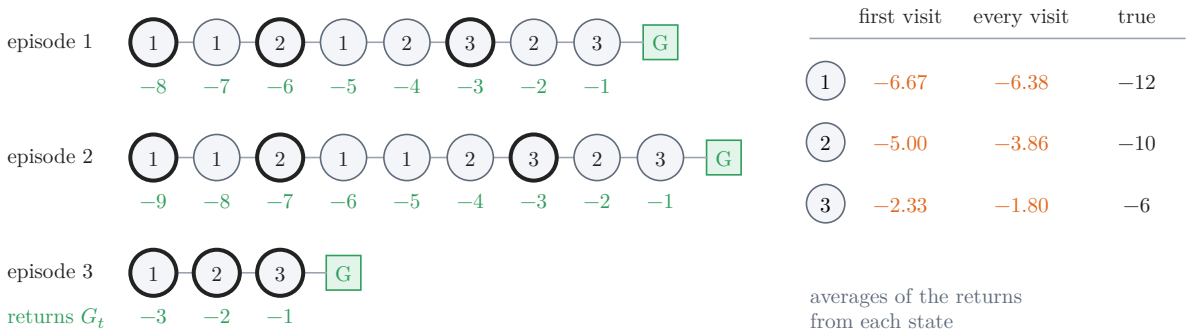


Figure 16. Monte Carlo prediction averages the returns that followed each state. Three episodes of the corridor at $p = \frac{1}{2}$, each a row of the states visited (circles, numbered by state), with the return from each step below in green. A thick ring marks the first visit to each state in an episode. Right: the averages of the returns from each state, over first visits only and over every visit, against the true values.

A value is an expected return (6), so the most direct estimate of it is an average of observed returns. Figure 16 shows three episodes. The first starts in state 1, stays there once, moves to state 2, and so on, reaching the goal after eight steps. Since every step costs 1, the return from each step is minus the number of steps left: -8 from the start, -7 from the next step, and so on down to -1.

To estimate $V^\pi(s)$, collect the returns that followed visits to s. If $N(s)$ episodes visit s, and $G^{(i)}(s)$ is the return after the first visit to s in the ith of them, the **first-visit Monte Carlo** estimate is

$$\hat{V}(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} G^{(i)}(s). \tag{19}$$

The *every-visit* estimate averages the returns after all visits instead, so that an episode that passes through s three times contributes three returns. If $M(s)$ is the number of visits to s in all the episodes, and $G^{(j)}(s)$ the return after the jth of them,

$$\hat{V}(s) = \frac{1}{M(s)} \sum_{j=1}^{M(s)} G^{(j)}(s). \tag{20}$$

In Figure 16, state 1 is visited first at the start of each episode, with returns -8, -9, and -3, so its first-visit estimate (19) is -6.67. Counting every visit adds five more returns, and the estimate becomes -6.38. Both are far from the true -12: three episodes are few, and these three happened to be short. The method is named after the casino, for its use of random samples.

Both estimates converge to V^π as the number of episodes grows. For first visits the reason is the law of large numbers: returns from different episodes are independent, and each has mean $V^\pi(s)$ by the definition of a value (6), so their average converges, with an error that shrinks like one over the square root of the number of episodes. The returns of one episode are not independent of each other, but the every-visit average converges too (Sutton and Barto, 2018, Section 5.1). After 20,000 episodes, the first-visit estimate for state 1 is -12.06 (Figure 10b). In this sense the return is an **unbiased** target: right on average. It is also very noisy. The returns from state 1 range from -3 to -95.

The average need not be recomputed from scratch for each new return. If V_{n-1} is the average of the first $n-1$ returns from a state and G_n is the nth, then

$$V_n = \frac{1}{n}\sum_{i=1}^{n} G_i = \frac{1}{n}(G_n + (n-1)V_{n-1}) = V_{n-1} + \frac{1}{n}(G_n - V_{n-1}). \tag{21}$$

The new estimate is the old one plus a fraction of the error. Writing the fraction as a step size η gives the general Monte Carlo update, applied for each step t of an episode:

$$V(s_t) \leftarrow V(s_t) + \eta(G_t - V(s_t)). \tag{22}$$

It is one step of gradient descent with learning rate η on the squared error $\frac{1}{2}(G_t - V(s_t))^2$, the same form that §10 uses with a network in place of the table. With $\eta = 1/n$ for the nth return from a state, (22) computes the running mean (21) exactly. With a constant η, unrolling the update shows what it computes instead:

$$V_n = (1-\eta)^n V_0 + \sum_{i=1}^{n} \eta(1-\eta)^{n-i} G_i, \tag{23}$$

an average in which the weight of a return decays geometrically with its age. That suits learning while the policy changes, as it will in §9, since old returns then describe an old policy.

The return G_t (3) includes every reward until the episode ends, so none of these updates can be made before then. Every-visit Monte Carlo prediction runs as follows:

1. start from any values $V(s)$;
2. run one episode with π, recording $s_0, a_0, r_0, \dots, s_T$;
3. going back from the last step, with $G = 0$, for $t = T-1, \dots, 0$: set $G \leftarrow r_t + \gamma G$, so that $G = G_t$ by (4), and update $V(s_t)$ by (22);
4. repeat from step 2.

The agent acts throughout, so this is still learning from interaction, not the offline setting of §5. But it learns only between episodes, and in a task that never ends it would never learn at all.

Temporal-difference learning

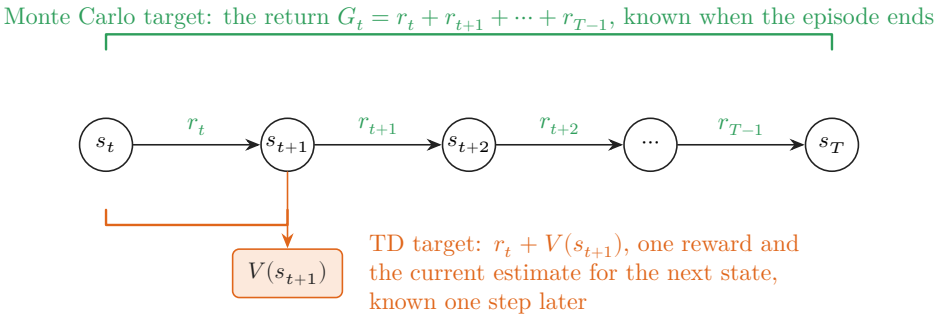


Figure 17. Monte Carlo waits for the return; TD uses one reward and its own next prediction. For an episode from step t: the Monte Carlo target is the return G_t (3), the sum of all the rewards still to come (green); the temporal-difference target is the first of them plus the current estimate for the next state, $r_t + V(s_{t+1})$ (orange). Drawn with $\gamma = 1$, as in the corridor.

The second way to learn values from experience uses the Bellman equation (10). Written as an expectation over one step, it says

$$V^\pi(s) = \mathbb{E}_\pi\big[r_t + \gamma V^\pi(s_{t+1}) \,\big|\, s_t = s\big].$$

Dynamic programming computes this expectation from the model, as in (16). Without the model, the agent can use the one sample of it that it has just seen, $r_t + \gamma V(s_{t+1})$, as the target in place of the return (Figure 17). This is **temporal-difference learning**, or TD (Sutton, 1988). Write V_k for the table of estimates after k updates, counted across episodes, one update per step of experience. If the kth step goes from state s to state s' with reward r, then

$$\delta_k = r + \gamma V_k(s') - V_k(s), \qquad V_{k+1}(s) = V_k(s) + \eta\,\delta_k, \tag{24}$$

and $V_{k+1}(x) = V_k(x)$ for every other state x.

The quantity δ_k is the **TD error**: the difference between two predictions of the same return made one step apart, before and after one reward is seen. The value of a terminal state is fixed at 0. Updating a prediction toward another prediction is called **bootstrapping**. Because the target uses one step, this method is also called TD(0). As an algorithm:

1. start from any values $V(s)$, with $V = 0$ at terminal states, and from the start state s;
2. choose a from $\pi(\cdot \mid s)$, take it, and observe the reward r and the next state s';
3. update $V(s)$ by (24);
4. if s' is terminal, start a new episode; otherwise set $s \leftarrow s'$. Repeat from step 2.

Each update needs one transition (s, a, r, s'), which can then be discarded. The agent learns during an episode, and it can learn in tasks that never end.

(a) TD(0): after each step, the state just left moves toward $-1 + V(\text{next state})$

step	0	1 1→1	2 1→2	3 2→1	4 1→2	5 2→3	6 3→2	7 2→3	8 3→G
$V(1)$	0.00	−0.50	−0.75	−0.75	−1.31	−1.31	−1.31	−1.31	−1.31
$V(2)$	0.00	0.00	0.00	−0.88	−0.88	−0.94	−0.94	−1.45	−1.45
$V(3)$	0.00	0.00	0.00	0.00	0.00	0.00	−0.97	−0.97	−0.98

(b) Monte Carlo: nothing changes until the episode ends; then every state moves toward its returns

step	0	1 1→1	2 1→2	3 2→1	4 1→2	5 2→3	6 3→2	7 2→3	8 3→G
$V(1)$	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	−6.38
$V(2)$	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	−4.25
$V(3)$	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	−1.75

Figure 18. TD changes one entry after every step; Monte Carlo changes them all when the episode ends. The first episode of Figure 16, from estimates of 0, with step size $\eta = \frac{1}{2}$. Each column is the table of values after that many steps; the transition of each step is written above it. (a) TD (24): the entry for the state just left is updated (orange), toward -1 plus the estimate for the next state. (b) Every-visit Monte Carlo (22): nothing changes until the episode ends, and then each state moves toward the returns that followed its visits (green).

Figure 18 runs both methods through the first episode of Figure 16. TD's first update, in state 1 at step 0, sees a reward of -1 and a next state still valued at 0, so $\delta_0 = -1$ and $V(1)$ moves halfway, to -0.5. Information about the cost of the corridor travels back only one state per update, and by the end of the episode no estimate is below -1.5. Monte Carlo waits, and then moves every state toward returns of up to -8. After one episode from zero, Monte Carlo is much closer to the truth.

On a single update, the two targets differ in what they get wrong. Take estimates $V = (-10, -8, -5)$ and $\eta = 0.1$, and the same episode. At step 2 the agent is in state 2, chooses "right", receives -1, and returns to state 1.

1. TD (24): $\delta = -1 + V(1) - V(2) = -1 - 10 + 8 = -3$, so $V(2)$ moves from -8 to $-8 + 0.1 \times (-3) = -8.3$.
2. Monte Carlo (22): the return from step 2 is $G_2 = -6$, so $V(2)$ moves from -8 to $-8 + 0.1 \times (-6 + 8) = -7.8$.

The true value of state 2 is -10. TD moved toward it, and Monte Carlo away from it, because this episode happened to be short. The TD target depends on one random action and transition rather than all the remaining ones, so it is far less noisy than the return. But it is **biased** for as long as the estimates are wrong, since it inherits the error of $V(s')$.

Speed and convergence

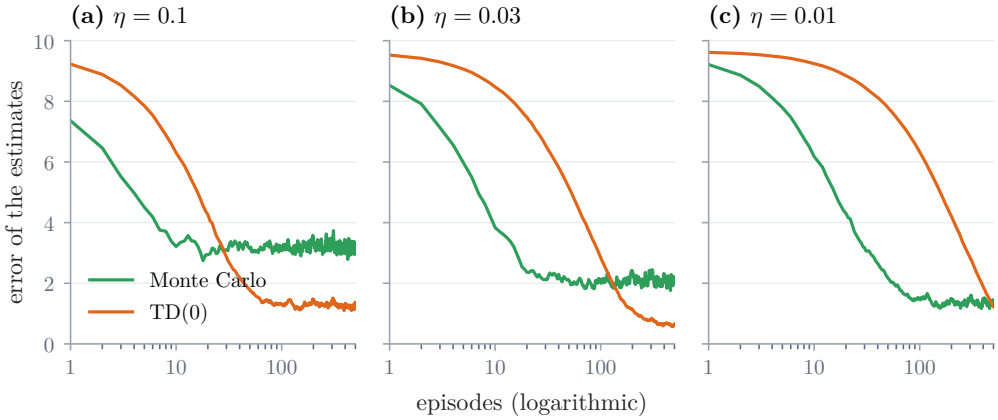


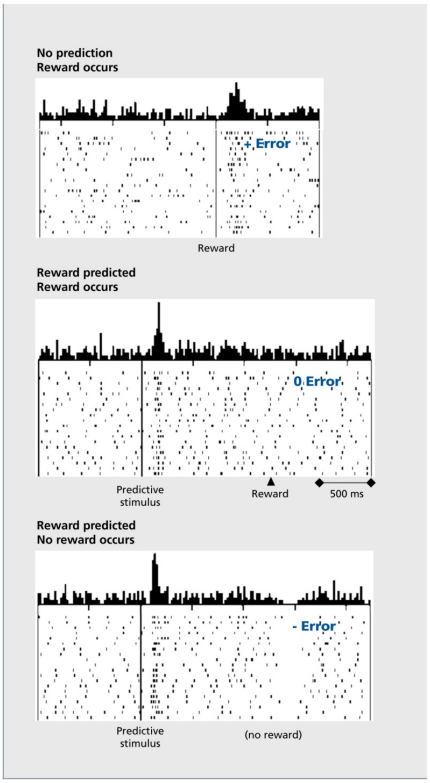
Figure 19. With a constant step size, Monte Carlo learns faster at first and TD settles closer. Estimates of V^π in the corridor at $p = \frac{1}{2}$, from $V = 0$, by every-visit Monte Carlo (green) and TD(0) (orange): the root mean square error over the three states, averaged over 100 runs, against the number of episodes (logarithmic), for 500 episodes with the constant step sizes (a) $\eta = 0.1$, (b) 0.03, and (c) 0.01.

Figure 19 compares the two methods over many episodes. Monte Carlo learns faster at first, for the reason Figure 18 shows: each return carries the whole episode's information to every state it visited. After 10 episodes at $\eta = 0.1$, the Monte Carlo estimates are off by 3.2 on average and the TD estimates by 6.3. But with a constant step size neither settles. Each estimate keeps moving with the noise of its latest targets, and the TD targets are less noisy. After 500 episodes the error is 3.3 for Monte Carlo and 1.4 for TD at $\eta = 0.1$, and 2.0 against 0.7 at $\eta = 0.03$. A smaller step size averages more targets and fluctuates less, but takes longer to get anywhere: at $\eta = 0.01$, both are still on their way after 500 episodes (1.4 and 1.2).

To converge, the step sizes must shrink, but not too fast; with steps that do, both methods converge to V^π (Theorem 9, Appendix B.7).

TD errors in the brain

(a) a dopamine neuron, recorded



(b) the TD error of our simulation

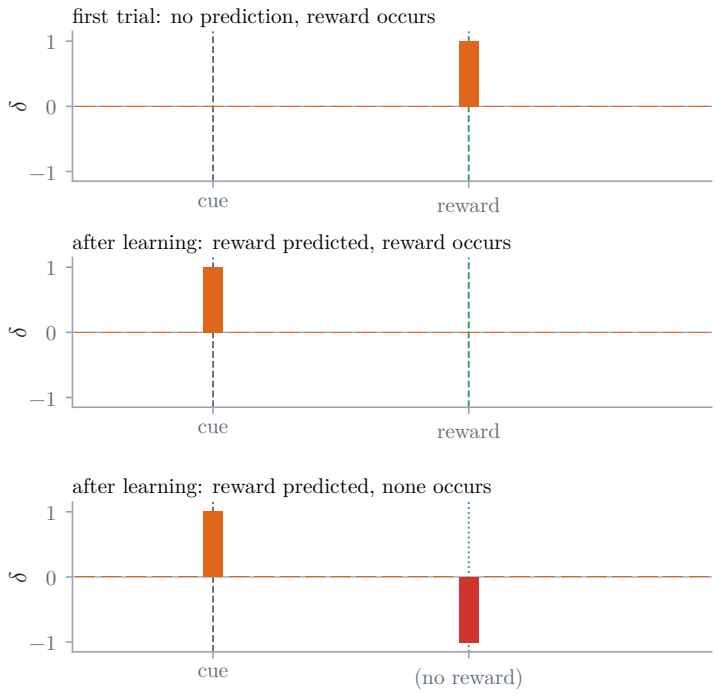


Figure 20. Dopamine neurons signal the TD error. (a) The firing of one dopamine neuron in a monkey's midbrain: in each panel, every row of ticks is one trial, aligned on the event below it, and the histogram above adds up the trials. A reward that nothing predicted makes the neuron fire (top, "+ Error"). Once a stimulus predicts the reward, the neuron fires at the stimulus and not at the reward (middle, "0 Error"). When the predicted reward is withheld, its firing dips at the time the reward was due (bottom, "− Error"). Reproduced from Schultz (2016), Fig. 2, CC BY-NC-ND 3.0; the recordings are those of Schultz, Dayan, and Montague (1997), Fig. 1. (b) The TD error (24) in a simulation of the same three situations, each row beside the matching recording: a cue at step 5 (gray line) is followed by a reward of 1 at step 15 (green line); TD(0) with one state per step since the cue, $\gamma = 1$, and $\eta = 0.2$; before the cue the value is 0, since the cue's timing cannot be predicted. Top: the first trial. Middle: after 200 trials. Bottom: after learning, a trial without the reward.

TD's error signal has a well-known counterpart in the brain. In the experiments of Wolfram Schultz, a monkey saw a light, and a moment later received a drop of juice. Dopamine neurons in the midbrain fired a burst when the juice arrived, at first. As the monkey learned that the light predicted the juice, the burst moved to the light, and the juice itself no longer changed their firing. When the juice was withheld after the light, their firing dipped below its baseline, at the moment the juice would have come (Figure 20a). Schultz, Dayan, and Montague (1997) pointed out that the TD error (24) does the same.

Figure 20b shows it in a simulation. Let the states be the steps since the cue, and let the reward of 1 come at step 15.

1. In the first trial all values are 0, so the only nonzero TD error is at the reward: $\delta = 1 + 0 - 0 = 1$ (Figure 20b, top).

2. The update raises the value of the state just before the reward. In the next trial, the step into that state has a positive TD error, and the error at the reward is smaller. Trial by trial, the error moves back toward the cue.
3. After learning, every state from the cue on has value 1, the reward that is coming. The error at the reward is $1 + 0 - 1 = 0$. The cue arrives at an unpredictable time, from states whose value is 0, so the error at the cue is $1 - 0 = 1$ (Figure 20b, middle).
4. If the reward is then withheld, the step where it was due has error $0 + 0 - 1 = -1$: the dip (Figure 20b, bottom).

The recordings of Figure 20a show the first and last states of this learning; the step-by-step backward movement of item 2 was long hard to see in recordings, and it has since been observed in mice (Amo et al., 2022). That dopamine signals a reward prediction error is now a standard account of how animals learn from reward, although dopamine does other things as well.

Between the two: n-step returns and TD(λ)

The Monte Carlo and TD targets are the two ends of a range. The n**-step return** uses n rewards and then the estimate of the state reached,

$$G_t^{(n)} = r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1} + \gamma^n V(s_{t+n}). \tag{25}$$

With $n = 1$ it is the TD target (24), and with n reaching the end of the episode it is the return (3). More rewards mean less bias from the estimate and more noise from the actions and transitions. **TD(λ)** uses an average of all of them,

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}, \tag{26}$$

where the n-step returns that run past the end of the episode are all equal to G_t. A single parameter λ between 0 and 1 moves the target from TD ($\lambda = 0$) to Monte Carlo ($\lambda = 1$). TD(λ) can be computed online, one step at a time, with a decaying record of recently visited states called an eligibility trace (Sutton and Barto, 2018, Chapter 12). TD-Gammon learned with TD(λ) (§10), and the advantage estimates of §13 (49) use the same averaging.

Figure 21 places the methods of §§6–7 side by side. They differ in how wide an update looks, at one sampled action and next state or at all of them, and in how deep, one step and then an estimate, or all the way to the end of the episode. Sampling removes the need for a model, and bootstrapping removes the wait for the end of the episode.

8. Exploration

Prediction estimates the values of a fixed policy. To improve the policy, the agent must also learn the values of actions its policy does not prefer, and it learns about an action only by taking it. The tension between using what it knows and finding out more is clearest in the simplest setting, the bandit of §3.

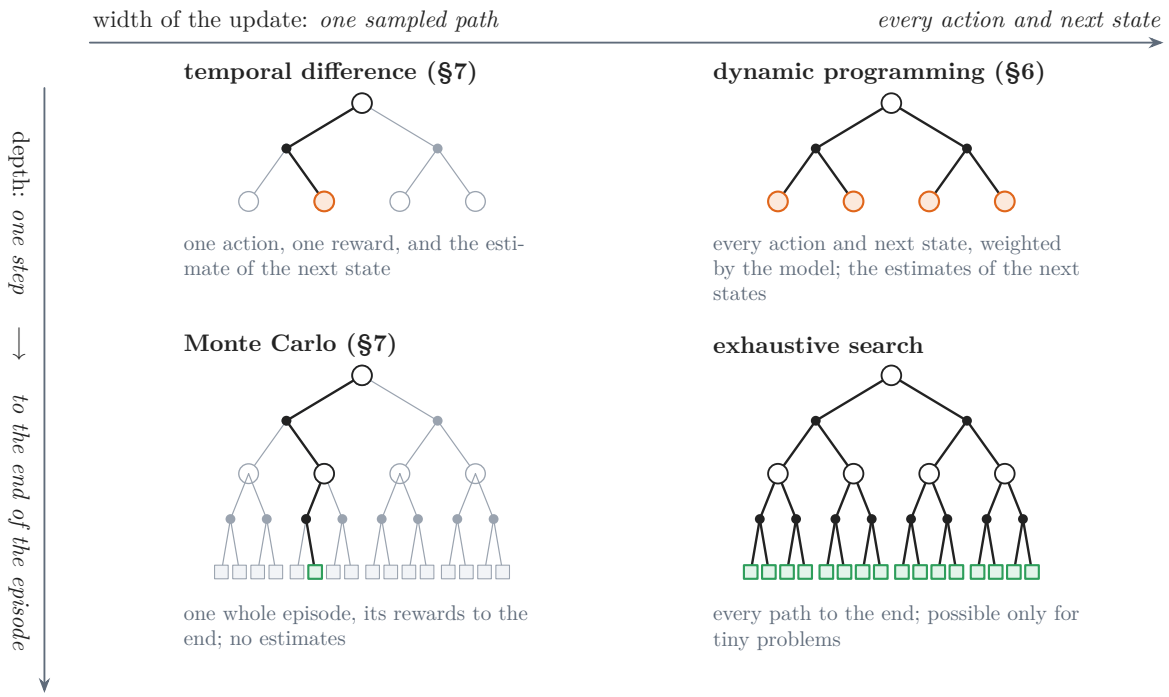


Figure 21. Four ways to back up a value, by how wide and how deep they look. Each tree starts at the state being updated (top); open circles are states, dots actions, squares the ends of episodes. What the update uses is drawn in black: estimates of later values in orange, rewards to the end of the episode in green. Temporal difference and Monte Carlo use one sampled path, one step deep or to the end; dynamic programming uses every action and next state, one step deep, and needs the model; exhaustive search uses every path to the end. After David Silver's lectures and Sutton and Barto (2018), Figure 8.11.

The multi-armed bandit

A bandit has K arms. Pulling arm a pays a random reward with mean μ_a, and the agent does not know the means. Its estimate of an arm's value, $\hat{Q}(a)$, is the average of the rewards that arm has paid so far. In our example two arms pay 1 with probabilities $\mu_1 = 0.7$ and $\mu_2 = 0.4$, and 0 otherwise. At each pull the agent must decide whether to pull the arm that looks best, which is **exploiting**, or the other arm, to learn more about it, which is **exploring**.

An agent that always exploits can settle on the worse arm for good (Figure 22). Suppose the first pull of arm 1 pays 0 and the first pull of arm 2 pays 1. A greedy agent then pulls arm 2 from then on. Its average stays above 0, the average of arm 1 stays at 0, and the agent never learns that arm 1 is better. In our runs, 18% of greedy agents never pulled arm 1 again after its first pull, and after 500 steps 21% were pulling the worse arm.

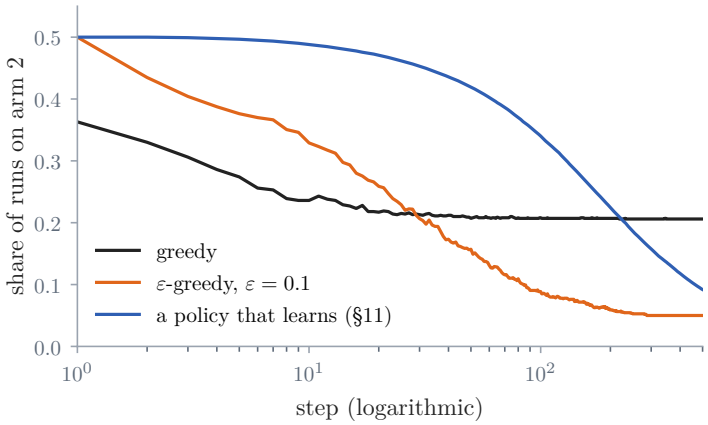


Figure 22. An agent that never explores can lock onto the worse arm for good. The two-armed bandit with means 0.7 and 0.4: the share of 1,000 runs that pull the worse arm at each step (for the two random rules, the average probability of pulling it), for three rules: greedy, which pulls the arm with the higher average so far after one pull of each; ε-greedy, which does the same but pulls a random arm one time in ten; and a random policy $p = \sigma(\theta)$, the probability of arm 1, that sharpens as it learns, by the policy gradient of §11. Logarithmic steps.

Three ways to explore

1. **Mostly greedy.** Exploit, but with a small probability ε take a random action instead: the *ε-greedy* rule. With $\varepsilon = 0.1$, half of the random pulls go to the worse arm, so it is pulled 5% of the time, forever. The rule is crude, but it is the most widely used, because it works with any value estimates.
2. **Optimism.** Add to each estimate a bonus that shrinks as the arm is tried more, and pull the arm whose optimistic estimate is largest. The bonus makes an arm that has been tried little look good until the agent has found out how good it is. The rule known as *UCB1*, for upper confidence bound, pulls the arm with the largest $\hat{Q}(a) + \sqrt{2 \ln t / n_a}$, where n_a is the number of pulls of arm a so far and t the total (Auer et al., 2002).
3. **Random choice in proportion to belief.** *Thompson sampling* (Thompson, 1933) keeps a probability distribution over each arm's mean, draws one sample from each, and pulls the arm whose sample is largest. An arm is then pulled about as often as the agent believes it to be the best. A policy that is random and sharpens as it learns, such as the policy gradient of §11 produces, also explores, though less efficiently (Appendix E).

How well a rule explores is measured by its *regret*, the reward it loses, in expectation, against always pulling the best arm: greedy and ε-greedy keep losing a fixed amount per pull, while optimism and Thompson sampling lose ever less, and Appendix E compares five rules on our bandit. The explorer's bandit lets you set the number of arms and their reward distributions, and runs these rules side by side.

Exploration beyond bandits

In a bandit the best action depends on nothing. In a **contextual bandit** it depends on a context the agent sees first, such as an ad shown to a particular user, or a prompt given to a language model. In an MDP it depends on the state, and an action's value includes everything that follows it, so an agent may have to explore many steps before it reaches anything new. Exploration in large MDPs remains hard, and most deep methods still use ε-greedy actions or a random policy.

9. SARSA and Q-learning

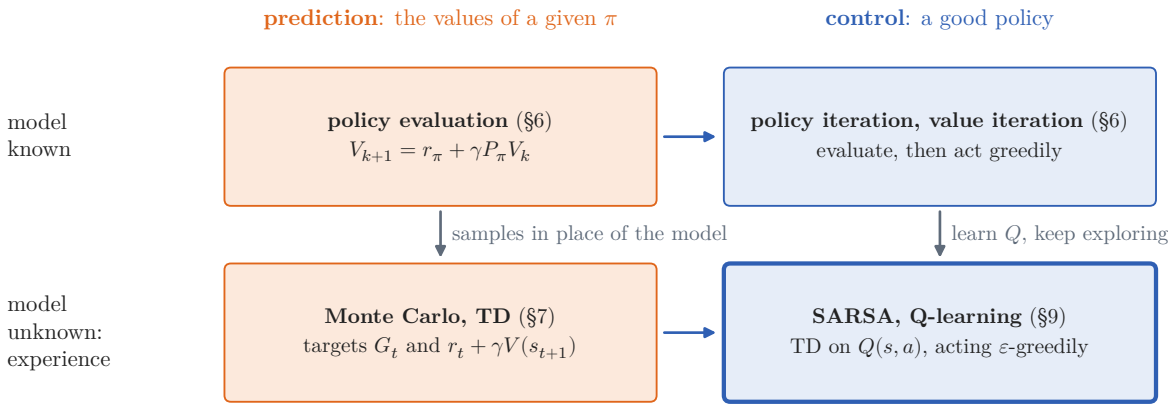


Figure 23. Part 2 in one table: two problems, with and without a model. Prediction estimates the values of a given policy; control finds a good policy. Dynamic programming solves both from the model (§6). Without the model, Monte Carlo and TD solve prediction from experience (§7), and SARSA and Q-learning solve control (§9). §10 replaces the tables by networks.

Part 2 has so far solved three of the four problems in Figure 23. With a model, dynamic programming evaluates a policy and finds an optimal one (§6). Without a model, Monte Carlo and TD evaluate a policy from experience (§7). The last step is to find a good policy from experience alone. The plan is that of policy iteration, to evaluate the current policy and then act greedily on its values, with the evaluation done by TD. Two things must change for it to work without a model: the agent must learn the values of actions rather than of states, and it must keep exploring.

Why action values

The improvement step of policy iteration (17) chooses in each state

$$\pi'(s) = \arg\max_a \sum_{s'} p(s' \mid s, a)\big[r(s,a,s') + \gamma V(s')\big],$$

and that needs the transition probabilities. Values of states say which states are good, not which actions lead there. Take the corridor, and suppose for now that the agent can see which state it is in. Under the coin-flip policy, $V^\pi = (-12, -10, -6)$: state 3 is much better than state 1. In state 2, the agent should therefore move to state 3. But in state 2 the actions are reversed, and nothing in V says that "left" is the action that moves right. The action values of §4 do say it: $Q^\pi(2, \text{left}) = -7$ and $Q^\pi(2, \text{right}) = -13$. Improving on action values needs only a comparison of numbers,

$$\pi'(s) = \arg\max_a Q(s,a), \tag{27}$$

so model-free control learns $Q(s, a)$.

The agent also cannot act purely greedily on its estimates, because it would never learn about the actions it does not take (§8). The usual compromise is the *ε-greedy* policy: with $|\mathcal{A}|$ actions,

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}| & \text{if } a = \arg\max_{a'} Q(s, a'), \\ \varepsilon/|\mathcal{A}| & \text{otherwise.} \end{cases} \tag{28}$$

It takes the best-looking action, except that with probability ε it picks an action uniformly at random. In the methods below, the agent does not wait for its values to converge before improving its policy. It updates Q after every step and always acts ε-greedily on the current Q, so the policy improves as the values do. This is generalized policy iteration (§6), with one TD update as the evaluation step.

SARSA

The first method applies TD (24) to action values. The Bellman equation for Q^π (11) says that $Q^\pi(s_t, a_t)$ is the expected value of $r_t + \gamma Q^\pi(s_{t+1}, a_{t+1})$, where a_{t+1} is the next action of π. One sample of it is available as soon as the agent has chosen a_{t+1}:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \eta[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)], \tag{29}$$

with $Q(s_{t+1}, \cdot) = 0$ when s_{t+1} is terminal. Each update uses the five quantities $s_t, a_t, r_t, s_{t+1}, a_{t+1}$, and the method is named after them: **SARSA**, for state, action, reward, state, action. Rummery and Niranjan (1994) introduced it as modified Q-learning, and the name came from Sutton. The algorithm:

1. initialize $Q(s, a)$, for example to 0, with $Q = 0$ in terminal states;
2. at the start of an episode, observe s_0 and choose a_0 by (28);
3. take a_t, observe r_t and s_{t+1}, and choose a_{t+1} by (28) from the current Q;
4. update $Q(s_t, a_t)$ by (29);
5. if s_{t+1} is terminal, start a new episode at step 2; otherwise continue from step 3 with $t + 1$.

The action a_{t+1} in the target is the action the agent then takes, so SARSA estimates the values of the policy it follows, exploration included. In the terms of §5 it is *on-policy*. If exploration stays at $\varepsilon = 0.1$, it learns the values of an agent that takes a random step one time in ten, and tends toward the best policy for such an agent. To learn $Q^\star$, it must explore less and less as it learns, and then, with suitably shrinking steps, it converges to $Q^\star$ (Theorem 10, Appendix B.7).

Q-learning

Q-learning (Watkins and Dayan, 1992) changes one term. Its target uses the best next action rather than the one the agent will take:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \eta\Big[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)\Big]. \tag{30}$$

The algorithm is that of SARSA, except that the update can be made as soon as s_{t+1} is seen, and a_{t+1} is chosen afterwards. The target is one sample of the Bellman optimality equation (15), so Q-learning estimates $Q^\star$ directly, the values of the greedy policy, while the agent follows an exploring one.

This is what *off-policy* means here, and why it works. The update (30) concerns $Q(s_t, a_t)$, the value of an action that has already been chosen, and what happens after it, the reward r_t

and the next state s_{t+1}, is up to the environment and not the policy. The only place a policy enters the target is the choice of the next action, and Q-learning replaces that choice by the maximum, the choice of the greedy policy. So the transition (s_t, a_t, r_t, s_{t+1}) is equally useful whichever policy chose a_t: an exploring policy, an older version of the agent, or a person.

Q-learning converges to $Q^\star$ whatever policy chose the actions, provided every state and action keeps being updated (Theorem 11, Appendix B.7). So its exploration need not fade, and it can learn from stored transitions, which deep Q-networks use (§10).

Cliff walking: what each method learns

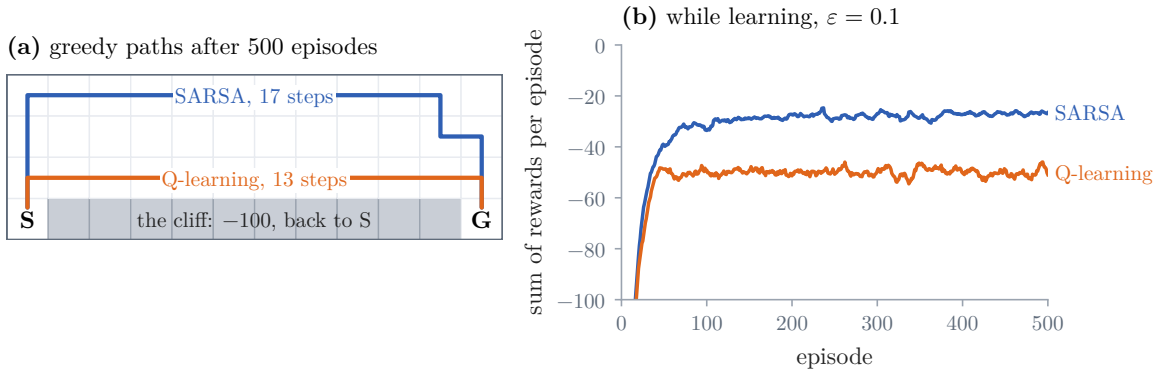


Figure 24. On-policy SARSA learns the safe path; off-policy Q-learning learns the optimal path, and falls off the cliff while exploring. Cliff walking (Sutton and Barto, 2018, Example 6.6): a 4 × 12 grid, from S to G, every step −1, and −100 and back to S for stepping into the cliff (gray). Both methods act ε-greedily with $\varepsilon = 0.1$ and step size 0.5. (a) The greedy path each has learned after 500 episodes: SARSA blue, Q-learning orange. (b) The sum of rewards per episode while learning, with $\varepsilon = 0.1$, averaged over 200 runs and over 10 successive episodes.

Cliff walking shows the difference (Figure 24). An agent walks from S to G along the edge of a cliff; every step costs 1, and a step into the cliff costs 100 and sends it back to the start. Both methods explore with $\varepsilon = 0.1$ throughout.

Q-learning learns the shortest path, 13 steps along the edge, which is optimal for an agent that does not explore: a return of −13. SARSA learns a path of 17 steps along the top row, as far from the cliff as the grid allows, a return of −17 without exploration. It values each state as an exploring agent experiences it, and an exploring agent next to the cliff sometimes falls in.

Figure 24b measures something else: the rewards the agents collect while they learn, with their random steps included. There SARSA does better, −26.9 per episode over the last 100 episodes against −50.4 for Q-learning, whose random steps from the edge send it over. Both results are correct for what they measure. Q-learning has learned the better policy for an agent that will stop exploring, and SARSA the better one for an agent that will not. Which matters depends on how the agent will be used, for example whether it keeps learning and exploring once deployed.

A table of values works for a gridworld with a few dozen states. Most tasks have far more states than any table could hold.

10. Function approximation and deep Q-networks

A table holds one value per state and action, and it learns nothing about a state until it visits it. Backgammon's positions (§6) and an Atari game's millions of distinct screens are far too many to visit. An agent in such a task sees almost every state for the first time, so it must **generalize** from the states it has seen to similar ones it has not. It does this by computing values with a function that has parameters, instead of looking them up.

Learning values with a function

Replace the table by a function $Q_w(s, a)$ with parameters w. It can be a weighted sum of features of the state and action, $Q_w(s, a) = w^\top \phi(s, a)$, or a neural network whose weights are w. An update made at one state now changes the values of every state the function treats as similar. This is how the agent generalizes, and it is also why its values can diverge: an update made for one state moves the values, and so the targets, of others.

Q-learning (30) becomes a step of regression. For a transition (s_t, a_t, r_t, s_{t+1}), compute the target with the current function, and take a gradient step that moves $Q_w(s_t, a_t)$ toward it:

$$y_t = r_t + \gamma \max_{a'} Q_w(s_{t+1}, a'), \qquad w \leftarrow w + \eta(y_t - Q_w(s_t, a_t))\nabla_w Q_w(s_t, a_t). \tag{31}$$

This is a gradient step on the squared error $\frac{1}{2}(y_t - Q_w(s_t, a_t))^2$ with the target y_t held fixed. It differs from supervised regression in one way: the target is computed from the same parameters that are being trained, and the gradient ignores that dependence. Such updates are called **semi-gradient** methods. Each step moves the targets of later steps. A table is the special case in which $\phi(s, a)$ has a single 1, in the entry for (s, a): then $\nabla_w Q_w(s_t, a_t)$ picks out that one entry, and (31) is exactly (30).

TD-Gammon

The first great success of this idea came long before deep learning. TD-Gammon (Tesauro, 1995) was a neural network with one hidden layer that estimated, from an encoding of a backgammon position, the probability of winning from it. It chose each move by evaluating the positions its legal moves led to. It was trained by TD(λ) (26) on games it played against itself, starting from random weights, with a reward only at the end of each game. Its first version, which saw only the board, learned to play as well as a strong intermediate player. Later versions, which were also given some features of the position designed by experts and searched two or three moves ahead, played close to the level of the best human players. Tesauro (1995) reports that their choices in some opening positions changed how expert players played them.

The deadly triad

TD-Gammon's success was hard to repeat in other games for the next two decades, and theory explains part of the reason. Three ingredients are each useful:

- function approximation, which generalizes;
- bootstrapping, which updates toward the learner's own estimates (§7);
- off-policy learning, in which the transitions used for updates come from a different distribution than the policy being evaluated would produce, as in Q-learning, whose data come from an exploring policy while it evaluates the greedy one (§9).

Together they can make the values diverge, growing without limit, even in small problems whose true values the function could represent exactly. Sutton and Barto (2018), Section 11.3, call the combination the **deadly triad**. Q-learning with a neural network (31) has all three.

Two states are enough to see it (Sutton and Barto, 2018, Section 11.2). Let one parameter w give the first state the value w and the second the value $2w$, and let the first state lead to the second with reward 0. If the second state then ends the episode with reward 0, both true values are 0, which $w = 0$ represents exactly. A semi-gradient update on the transition from the first state has TD error $\delta = 0 + \gamma \cdot 2w - w = (2\gamma - 1)w$, so

$$w \leftarrow w + \eta\,\delta \cdot 1 = \big(1 + \eta(2\gamma - 1)\big)\, w.$$

For $\gamma > \frac{1}{2}$, each such update makes $|w|$ larger. Updates on the transition from the second state would pull w back toward 0. An agent following its policy makes both kinds of update, and on-policy TD with a linear function is known to converge (Tsitsiklis and Van Roy, 1997). An off-policy agent can make the first kind of update far more often than the second, and then w diverges.

Deep Q-networks

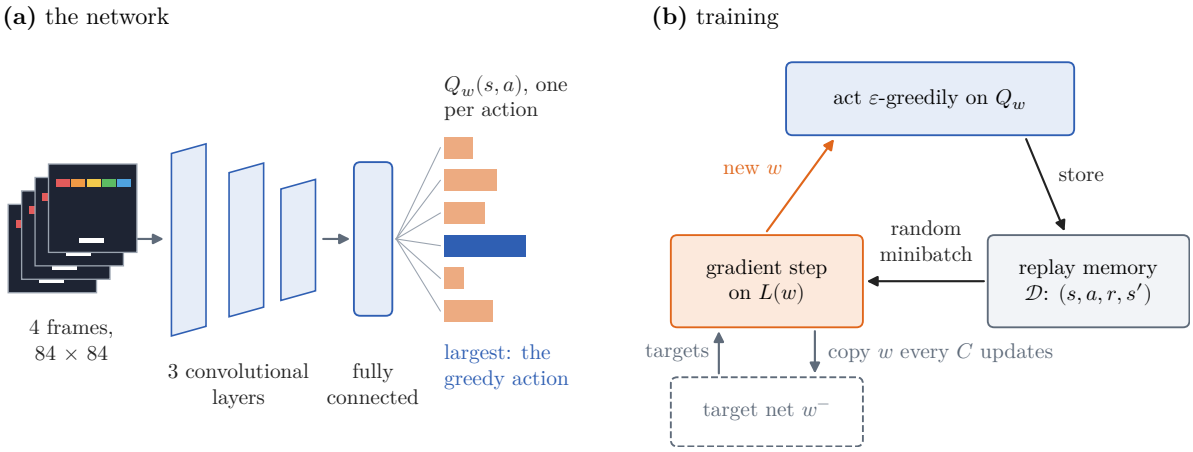


Figure 25. A deep Q-network: one network maps the screen to a value for each action, and learns from replayed transitions against a frozen copy of itself. (a) The network of Mnih et al. (2015): the last four frames, reduced to 84 × 84 pixels (in gray scale), pass through three convolutional layers and a fully connected layer to one output per action, here for 6 actions; the agent takes the largest, except when it explores. (b) The training loop. The agent acts ε-greedily on Q_w and stores each transition in a replay memory; each update samples a random minibatch from the memory and takes a gradient step on the loss (32), whose targets come from the target network w^-; every C updates, w^- is replaced by a copy of w.

Deep Q-networks (DQN; Mnih et al., 2015) made Q-learning with a deep network work (Figure 25). The network reads the last four frames of the screen (Figure 4), passes them through three convolutional layers and a fully connected layer, and outputs one value for each of the game's actions, so that one pass gives $Q_w(s,a)$ for every a. Two devices keep the deadly triad in check.

1. *A replay memory* $\mathcal{D}$ holds the most recent million transitions (s,a,r,s'), and each update uses a minibatch sampled at random from it. Consecutive transitions are strongly correlated, and a network trained on them in order would fit the last few seconds of play. Random samples from the memory are nearly independent, and each transition is reused in many

updates. This is possible because Q-learning is off-policy (Theorem 11): a transition is useful whichever earlier version of the agent collected it.

2. *A target network*, a copy of the network with parameters w^-, computes the targets, and is refreshed from w only every C updates (10,000 in the paper). Between refreshes the targets stand still, and each stretch of training is an ordinary regression.

Together they give the loss

$$L(w) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\Big[\big(r + \gamma \max_{a'} Q_{w^-}(s', a') - Q_w(s, a)\big)^2\Big], \tag{32}$$

with the target r alone when s' is terminal. The algorithm:

1. initialize w at random, set $w^- \leftarrow w$, and start with an empty memory $\mathcal{D}$;
2. in state s_t, choose a_t ε-greedily (28) on $Q_w(s_t, \cdot)$, take it, observe r_t and s_{t+1}, and add (s_t, a_t, r_t, s_{t+1}) to $\mathcal{D}$;
3. sample a minibatch of transitions from $\mathcal{D}$, and take a gradient step on (32) averaged over it;
4. every C updates, set $w^- \leftarrow w$; repeat from step 2.

With the same network and settings for every game, and only the pixels and the score as input, DQN learned 49 Atari games. It reached more than three quarters of a professional human tester's score on 29 of them. Mnih et al. (2015) also removed each device in turn on five games, and removing either one lowered the scores.

A deep Q-network on CartPole

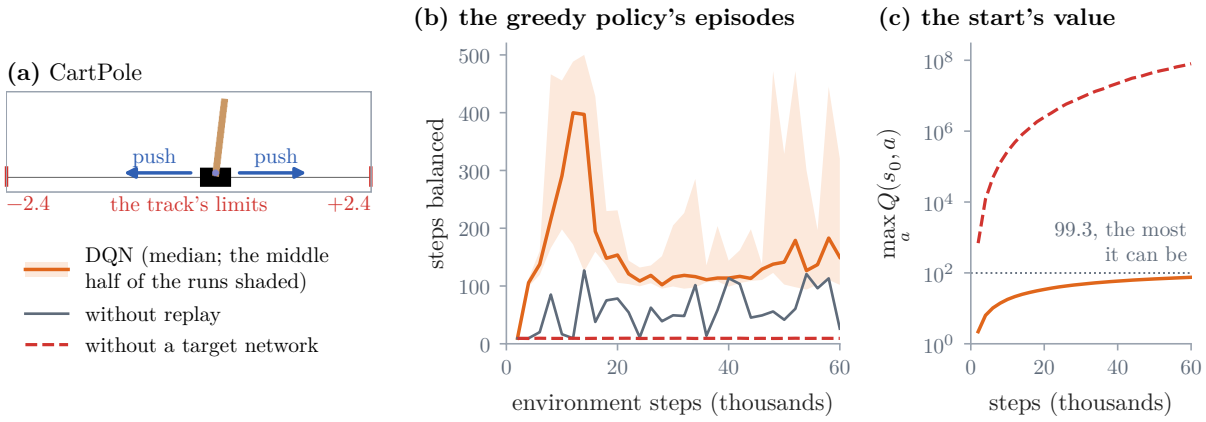


Figure 26. With a replay memory and a target network, a small DQN learns to balance the pole, though not stably; without the target network, its values diverge. (a) CartPole, rendered by Gymnasium (Towers et al., 2024): pushes left or right keep a pole upright on a cart; an episode ends when the pole leans past 12° or the cart leaves the track at ±2.4, and is cut at 500 steps; every step earns +1, and $\gamma = 0.99$. (b) The number of steps the greedy policy balances the pole, every 2,000 steps (mean of 5 episodes): the median of 10 runs, with the middle half of the DQN runs shaded. Without a target network the pole falls in 9 steps throughout. (c) The value the network gives the start, the median of the same runs (logarithmic axis); no 500-step episode can return more than 99.3 (dotted line).

We trained a small DQN on CartPole, the task of Barto et al. (1983) in the form of the Gym library (Brockman et al., 2016): with both devices, nine of 10 runs balanced the pole for the full 500 steps at some evaluation, but not stably, and after 12,000 steps the median fell back

to between 100 and 200 steps (Figure 26). Without the target network every run diverged, its values growing to about 8×10^7 against a true value of at most 99.3, and the pole fell in about 9 steps. Without the replay memory, training was erratic: seven runs balanced the pole for 500 steps at some evaluation, but none kept it (Appendix G gives the setup and the analysis).

Later improvements

DQN's maximum (32) overestimates values: the largest of several noisy estimates is, on average, larger than the largest true value. **Double DQN** (van Hasselt et al., 2016) applies double Q-learning (Appendix B.8), which chooses the next action with one estimate and evaluates it with another, using the two networks DQN already has: the online network w chooses the next action and the target network w^- evaluates it, so the target becomes $r+\gamma Q_{w^-}(s', \arg\max_{a'} Q_w(s', a'))$. *Prioritized replay* (Schaul et al., 2016) samples transitions with large TD errors more often, a *dueling* network (Wang et al., 2016) computes a state's value and the actions' advantages (7) in separate heads, and *distributional* methods (Bellemare et al., 2017) learn the whole distribution of the return rather than its mean. Hessel et al. (2018) combined six such improvements in one agent, Rainbow, which did better than any of them alone.

The limits of value-based methods

Acting greedily on action values (27) has two limits. The first is the maximum over actions, in the update and in acting. With a few discrete actions it is a comparison of a few numbers. With continuous actions, such as the torques of a robot's joints, finding the best action is itself an optimization problem at every step. The second is that the greedy policy is deterministic. That is enough in an MDP whose states the agent can see (Theorem 1), but not when it cannot tell states apart, as in the corridor, where the best policy is random. Both limits disappear if the agent learns the policy itself, which is the subject of Part 3.

11. The policy gradient and REINFORCE

Policy-based methods learn the policy itself, $\pi_\theta(a \mid s)$, by gradient ascent on its expected return $J(\theta)$ (5). This has two benefits over acting greedily on values. A policy can be random, which the corridor needs. And it handles many actions, or continuous ones, without a maximization at every step. The difficulty is the gradient. The return depends on θ only through the actions the policy samples, and the rewards and transitions have no gradient that the agent can compute.

Policies with parameters

For a finite set of actions, the usual policy computes a *preference* $h_\theta(s, a)$ for each action, for instance with a network, and turns the preferences into probabilities with a softmax (Figure 27a):

$$\pi_\theta(a \mid s) = \frac{\exp h_\theta(s, a)}{\sum_b \exp h_\theta(s, b)}. \tag{33}$$

A language model's next-token distribution is exactly this, with the logits as the preferences. In the bandit of §8 the policy has a single parameter: it pulls arm 1 with probability $p = \sigma(\theta)$, the logistic function of θ.

For a continuous action, the policy is usually a normal distribution whose mean a network computes (Figure 27b):

$$a \sim \mathcal{N}(\mu_\theta(s), \sigma^2). \tag{34}$$

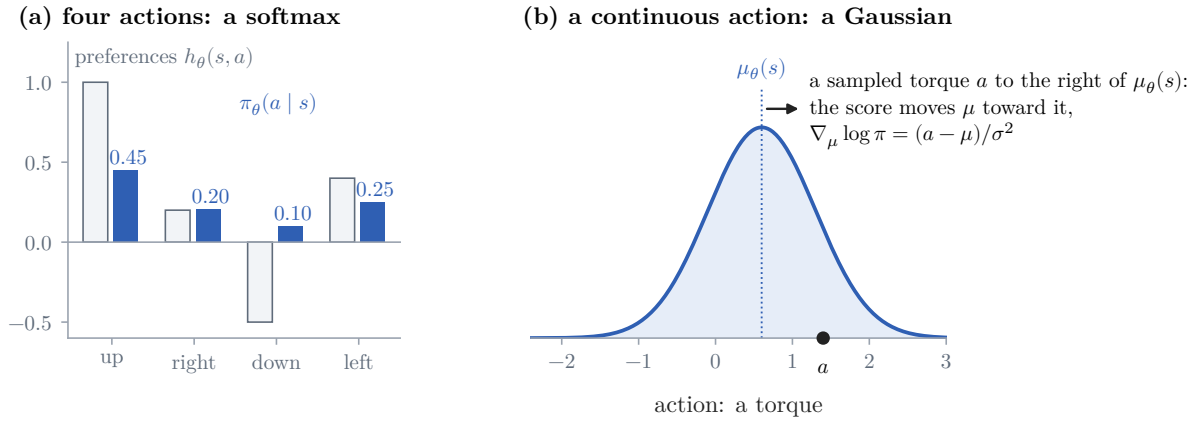


Figure 27. Two common policies: a softmax over a few actions, and a Gaussian over a continuous one. (a) A network gives each action a preference $h_\theta(s,a)$ (gray), and the softmax turns the preferences into probabilities (blue). (b) For a continuous action, such as a torque, a network gives the mean $\mu_\theta(s)$ of a normal distribution; a sampled action to the right of the mean has a score that moves the mean toward it.

The spread σ sets how widely the agent explores around its mean. It can be fixed by the designer, and lowered by hand over training. More often it is learned along with the mean: $\log\sigma$ is one more parameter in θ, either a single number or a second output of the network, and the policy gradient below adjusts it like any other parameter. A learned spread lets the agent explore widely where it is unsure and narrowly where it is not, but it can also shrink too early, which §13 counters with an entropy bonus (50).

The score-function identity

Start with one decision, as in a bandit. The expected reward is a sum over actions, $J(\theta) = \sum_a \pi_\theta(a)\, r(a)$, where $r(a)$ is the mean reward of action a. Only the probabilities depend on θ. Differentiate, then multiply and divide by $\pi_\theta(a)$:

$$\nabla_\theta J = \sum_a r(a)\, \nabla_\theta \pi_\theta(a) = \sum_a \pi_\theta(a)\, r(a)\, \frac{\nabla_\theta \pi_\theta(a)}{\pi_\theta(a)} = \mathbb{E}_{a\sim\pi_\theta}\big[r(a)\, \nabla_\theta \log \pi_\theta(a)\big]. \tag{35}$$

This is the **score-function identity**, also called the likelihood-ratio trick. The gradient of an expectation has become an expectation: of the reward times $\nabla_\theta \log \pi_\theta(a)$, which is called the **score** of the action. An expectation can be estimated by sampling. Draw an action from the policy and observe its reward; then $r\, \nabla_\theta \log \pi_\theta(a)$ is an unbiased estimate of the gradient, right on average. The reward is never differentiated, so it can come from anything: a slot machine, a program that checks a sum, or a person.

The score points in the direction in parameter space that makes the sampled action more likely, and the estimate moves that way in proportion to the reward. The score has mean zero,

$$\mathbb{E}_{a\sim\pi_\theta}\big[\nabla_\theta \log \pi_\theta(a)\big] = \sum_a \nabla_\theta \pi_\theta(a) = \nabla_\theta \sum_a \pi_\theta(a) = \nabla_\theta 1 = 0, \tag{36}$$

so without rewards the pushes cancel on average. With rewards, the actions that pay more are pushed up harder, and the probability moves toward them.

On the two-armed bandit with $p = \sigma(\theta)$, the score of arm 1 is $1-p$ and that of arm 2 is $-p$, since $d\log\sigma(\theta)/d\theta = 1-\sigma(\theta)$. The estimate averages to $p(1-p)(\mu_1-\mu_2)$, which is the gradient. But a single sample is far from it (Figure 28). At $p=\frac{1}{2}$, the gradient is 0.075, and the standard deviation of the estimate is 0.36, almost five times larger. For the Gaussian policy

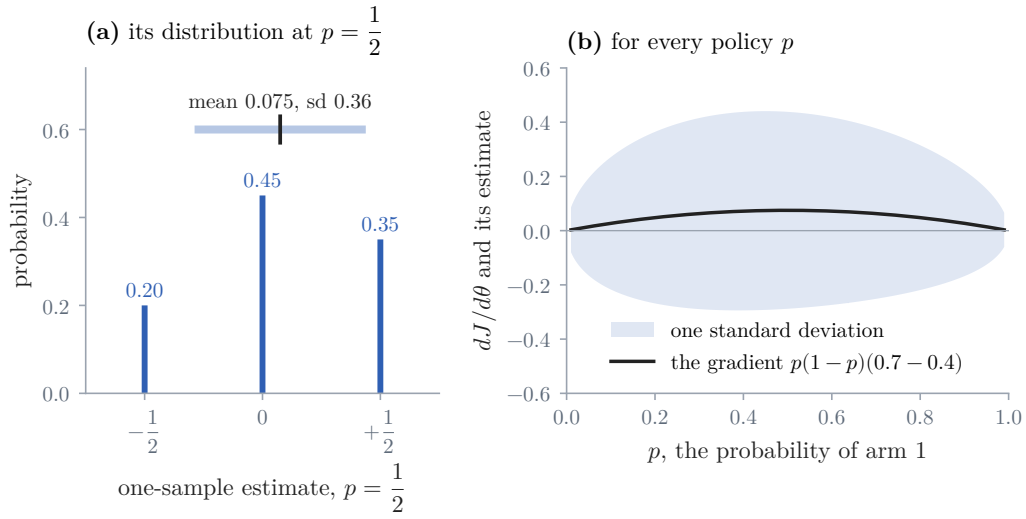


Figure 28. One sample gives an estimate of the gradient that is right on average and far from it every time. The two-armed bandit of §8 (arms paying 1 with probabilities 0.7 and 0.4), with the policy $p = \sigma(\theta)$. (a) At $p = \frac{1}{2}$, the one-sample estimate $r\,\nabla_\theta \log \pi_\theta(a)$ takes three values: $+\frac{1}{2}$ when arm 1 is pulled and pays (probability 0.35), $-\frac{1}{2}$ when arm 2 is pulled and pays (0.2), and 0 when the pull pays nothing (0.45). Its mean is the gradient, 0.075 (black line); its standard deviation, 0.36, is drawn as the blue bar. (b) The same for every p: the gradient $p(1-p)(0.7-0.4)$ (black) and the band of one standard deviation of the estimate around it (blue).

(34), the score moves the mean toward an action that earned a high reward and away from one that earned a low reward, and widens or narrows the spread (Figure 27b; Appendix A.4). The same identity (35), with the randomness in the parameters rather than the actions, is the evolution strategy of the optimizer tutorial (Section 7).

The policy gradient theorem

For whole episodes, the same steps apply with the trajectory in place of the action. The objective (5) is an expectation over trajectories, $J(\theta) = \sum_\tau p_\theta(\tau)\, G_0(\tau)$, and as in (35),

$$\nabla_\theta J = \sum_\tau G_0(\tau)\, \nabla_\theta p_\theta(\tau) = \mathbb{E}_{\tau\sim p_\theta}\big[G_0\, \nabla_\theta \log p_\theta(\tau)\big]. \tag{37}$$

The probability of a trajectory (2) is a product, so its logarithm is a sum, and only the policy's terms depend on θ:

$$\nabla_\theta \log p_\theta(\tau) = \nabla_\theta \Big[\log p(s_0) + \sum_{t=0}^{T-1} \big(\log \pi_\theta(a_t \mid s_t) + \log p(s_{t+1} \mid s_t, a_t)\big)\Big] = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t). \tag{38}$$

The environment's transitions have dropped out, so the gradient can be estimated without a model:

$$\nabla_\theta J = \mathbb{E}_\tau \Big[\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t) \sum_{k=0}^{T-1} \gamma^k r_k\Big]. \tag{39}$$

Each action's score is multiplied by every reward of the episode, including those that came before the action. Those earlier rewards average out. For $k < t$, the reward r_k is already fixed when a_t is drawn, so, conditioning on everything up to s_t and using (36),

$$\mathbb{E}\big[r_k\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\big] = \mathbb{E}\Big[r_k\, \mathbb{E}_{a_t\sim\pi_\theta(\cdot|s_t)}\big[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\big]\Big] = \mathbb{E}[r_k \cdot 0] = 0. \tag{40}$$

Dropping these terms leaves, for each step, the rewards from t on, which sum to $\sum_{k\geq t}\gamma^k r_k = \gamma^t G_t$:

$$\nabla_\theta J = \mathbb{E}_\tau\Big[\sum_{t=0}^{T-1}\gamma^t\, G_t\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big]. \tag{41}$$

Each action is credited only with the return that follows it. The expectation is the same as in (39), but the estimate from a sampled episode is less noisy, because it leaves out rewards that the action could not have caused. One more step replaces the return by its expected value. Given the state and the action, the expected return is $Q^\pi(s_t, a_t)$ (6), and the score is fixed, so G_t can be replaced by $Q^\pi(s_t, a_t)$ inside the expectation.

Theorem 6 (policy gradient theorem). *For a policy π_θ differentiable in θ, in an MDP whose episodes end,*

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\Big[\sum_{t=0}^{T-1}\gamma^t\, Q^{\pi_\theta}(s_t, a_t)\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big]. \tag{42}$$

The gradient of the expected return is the expected score of each action, weighted by how good the action is (Sutton et al., 2000). Changing θ also changes which states the agent visits, but no derivative of that appears. Appendix A.2 writes the theorem as a sum over states. Implementations with $\gamma < 1$ usually drop the factor γ^t, keeping the discount only inside G_t, and the result is then not quite the gradient of J (Appendix A.3). In this lecture's policy-gradient tasks $\gamma = 1$.

REINFORCE

The simplest algorithm samples episodes and steps along the estimate (41):

1. run the current policy for one episode, or a batch of them: $s_0, a_0, r_0, \ldots, s_T$;
2. for each step, compute the return from it, G_t;
3. take a step $\theta \leftarrow \theta + \eta \sum_t G_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$, averaged over the batch;
4. discard the episodes, and repeat from step 1.

This is **REINFORCE** (Williams, 1992). It is on-policy: the estimate is unbiased only for episodes sampled from the current policy, so each batch is used for one step and then discarded. In code it is written as a loss whose gradient is minus the estimate,

$$\mathcal{L}(\theta) = -\sum_t G_t \log \pi_\theta(a_t \mid s_t), \tag{43}$$

with the returns treated as constants. This is the negative log-likelihood of the actions the agent took, each weighted by the return that followed it. Supervised learning on demonstrations minimizes the same loss with every weight equal to 1 and the actions taken from the demonstrations. REINFORCE uses the agent's own actions, and weights them by how well they turned out. §19 makes this concrete for a language model.

In the corridor, REINFORCE finds the random policy that a value-based method cannot represent (Figure 29). Every run starts from $p = 0.05$, a policy that almost always goes left and takes 82 steps on average to reach the goal. Larger steps learn faster: the median run first comes within 0.5 of the best return at episode 249 with $\eta = 2^{-10}$, and at episode 35 with $\eta = 2^{-7}$. At $\eta = 2^{-6}$ learning breaks down: 68 of the 100 runs end stuck at a nearly deterministic policy, which takes hundreds of steps or more to reach the goal. The cause is the noise of the estimate (41). At $p = 0.4$, the estimate from one episode has a standard deviation of 32.1, against a gradient of 4.67. A single long episode can throw the policy to where it no longer tries the action that would correct it. §12 removes much of that noise.

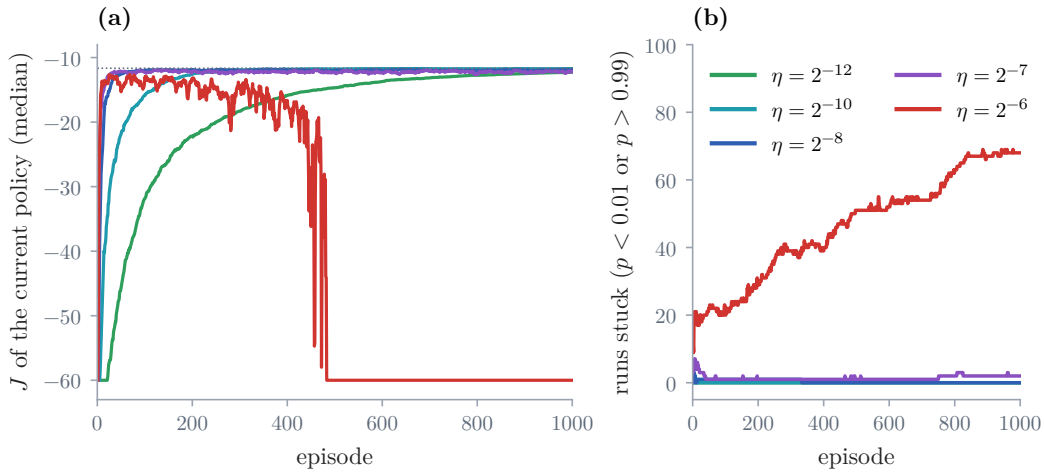


Figure 29. Larger steps learn faster, until runs collapse to a deterministic policy and stay there. REINFORCE in the short corridor, 100 runs of 1,000 episodes for each step size η, each from $p = 0.05$, with episodes cut at 1,000 steps. (a) The exact expected return J (13) of each run's current policy, the median over the runs (the dotted line is the best, -11.66; values below -60 are drawn at -60). (b) The number of runs whose policy is stuck, with p below 0.01 or above 0.99; for the four smaller step sizes it stays near 0, where their lines overlap.

12. Baselines and advantages

REINFORCE is unbiased, but its estimates (41) are so noisy that a large step can ruin the policy (§11). The policy gradient is a stochastic gradient, like the minibatch gradients of Lecture 5 (Section 6), and its noise limits how large a step can be. Two parts of that noise can be removed: the part that all returns share, and the part that comes from the luck of later actions.

The noise of a common offset

Add 10 to every reward of the bandit. The best arm is the same and so is the gradient, but every pull is now strongly reinforced, whichever arm it was. The estimate is about $+5$ when arm 1 is pulled and about -5 when arm 2 is, and the gradient, 0.075, survives only as the small difference between these large pushes. The standard deviation of the one-sample estimate goes from 0.36 to 5.28 (Figure 30).

Baselines

The remedy is to subtract from the return a **baseline** $b(s_t)$: any quantity that may depend on the state but not on the action.

Lemma 7 (baselines do not bias the gradient). *For any function b of the state,*

$$\mathbb{E}_\tau\Big[\sum_t \gamma^t\, b(s_t)\,\nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big] = 0, \qquad \text{so} \qquad \nabla_\theta J = \mathbb{E}_\tau\Big[\sum_t \gamma^t (G_t - b(s_t))\nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big]. \tag{44}$$

The proof is (36) once more. Given the state, $b(s_t)$ is a constant, so

$$\mathbb{E}_{a\sim\pi_\theta(\cdot|s)}\big[b(s)\,\nabla_\theta \log \pi_\theta(a \mid s)\big] = b(s)\,\nabla_\theta \sum_a \pi_\theta(a \mid s) = b(s)\,\nabla_\theta 1 = 0,$$

and the term of each step averages to zero. The argument fails if b depends on the action: then the factor cannot be taken out of the average.

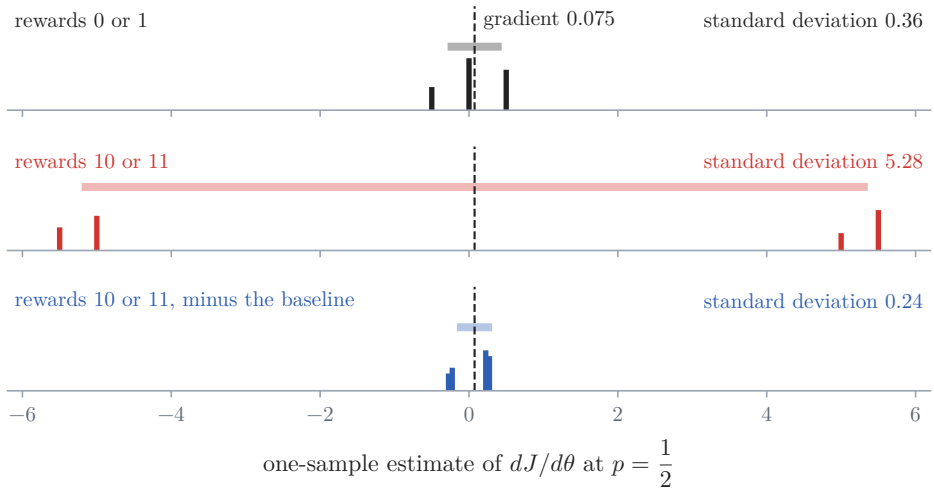


Figure 30. Raising every reward by 10 leaves the gradient unchanged and multiplies the noise by fifteen; a baseline removes it. The two-armed bandit at $p = \frac{1}{2}$. Each row shows the distribution of the one-sample estimate of $dJ/d\theta$: its possible values (stems, with height proportional to probability) and one standard deviation on either side of their mean (bar). Top: rewards of 0 or 1. Middle: every reward raised by 10. Bottom: rewards raised by 10, minus the baseline $J + 10$, the expected reward. The dashed line is the gradient, 0.075 in all three. Standard deviations from 100,000 samples.

With a baseline, an action is pushed up when its return beat the baseline, and down when it fell short. A natural baseline is the average return. In the shifted bandit, the expected reward as the baseline brings the standard deviation down to 0.24, below even that of the unshifted rewards. In practice the baseline is a running average of recent returns, the mean return of the other episodes in the batch, or a learned estimate of $V^\pi(s_t)$. The baseline that minimizes the variance is not quite the mean return (Appendix C).

The advantage

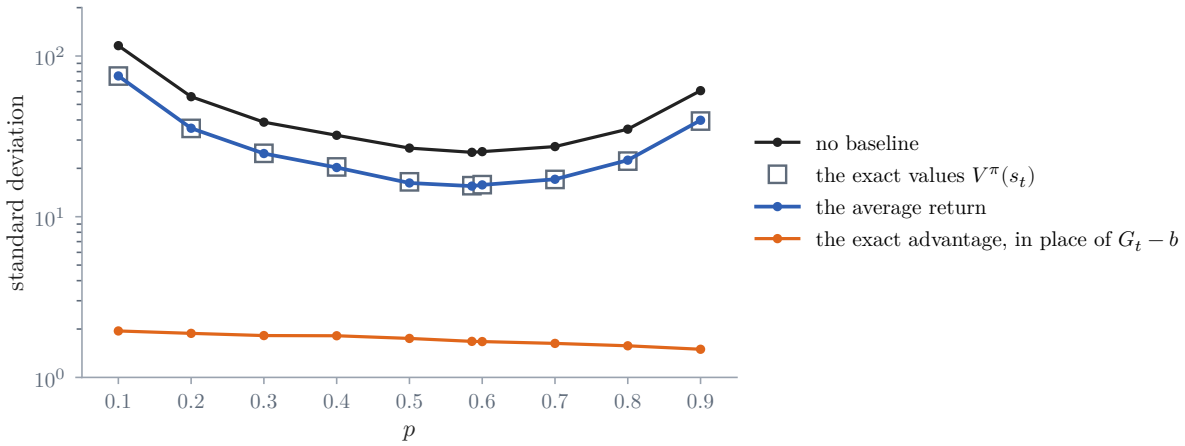


Figure 31. A baseline removes the common part of the returns; only the advantage removes their spread. The standard deviation of the one-episode estimate of $dJ/d\theta$ in the corridor, against the policy p, from 20,000 episodes at each p: with no baseline (black), with the average return as the baseline (blue), with the exact state values $V^\pi(s_t)$ as the baseline (gray squares, almost on the blue line), and with the exact advantage $A^\pi(s_t, a_t)$ in place of $G_t - b$ (orange). Logarithmic scale.

A baseline removes what the returns have in common. It cannot remove their spread, which comes from the random actions and transitions after step t. An action taken early in a long episode is weighted by a return that depends on everything that happened afterward. In the corridor at the best policy $p^\star$, the standard deviation of the one-episode estimate is 25.2 without a baseline and 15.5 with the average return as the baseline. Even the exact state values as the baseline give 15.6 (Figure 31).

Weighting each score by how good its own action was, rather than by the return that happened to follow it, removes the spread. That weight is the advantage (7), $A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$: the return to be expected after taking a_t, minus the return to be expected from s_t, with the luck of the later actions averaged out. Theorem 6 with Q^π, and Lemma 7 with $b = V^\pi$, give

$$\nabla_\theta J = \mathbb{E}_\tau\Big[\sum_t \gamma^t\, A^\pi(s_t, a_t)\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big]. \tag{45}$$

Weighting by the exact advantage brings the corridor's standard deviation to 1.7, against 25.2 without a baseline. Crediting each action with its advantage is how policy-gradient methods answer the problem of delayed rewards from §1: an action is judged by how much better it was than what the policy would have done anyway. But the agent does not know the advantage, and estimating it requires values, which §13 learns with a critic.

13. Actor–critic methods

The policy gradient (45) needs advantages, and advantages need values. An actor–critic method learns both: the policy, called the **actor**, by the policy gradient of §§11–12, and an estimate of its values, called the **critic**, by the TD learning of §7 (24), with a function as in §10. The critic's only job is to tell the actor how good each of its actions was.

The TD error as an advantage

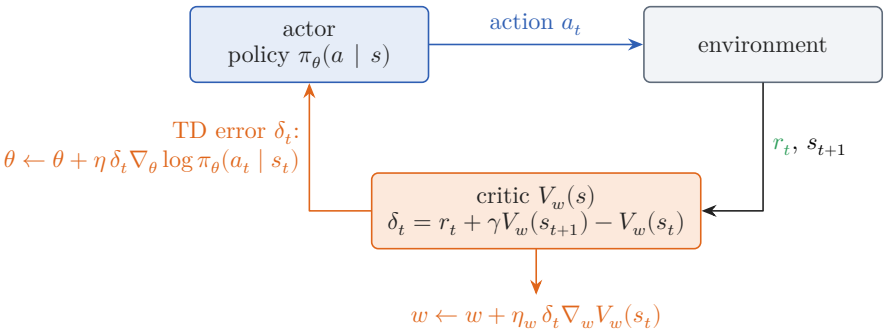


Figure 32. The critic's TD error tells the actor how much better than expected each action turned out. The actor (the policy, blue) acts; the environment returns a reward and the next state; the critic (orange) computes the TD error δ_t, steps its own parameters along it, and passes it to the actor, which steps along δ_t times the score of its action.

Let the critic be $V_w(s)$, with parameters w. After one step its TD error is

$$\delta_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t). \tag{46}$$

If the critic were exact, $V_w = V^\pi$, the TD error would be an unbiased estimate of the advantage. Averaging over the next state with (9),

$$\mathbb{E}[\delta_t \mid s_t, a_t] = \mathbb{E}[r_t + \gamma V^\pi(s_{t+1}) \mid s_t, a_t] - V^\pi(s_t) = Q^\pi(s_t, a_t) - V^\pi(s_t) = A^\pi(s_t, a_t). \tag{47}$$

The score $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$ is fixed once s_t and a_t are, so δ_t can replace $A^\pi(s_t, a_t)$ in (45) without changing its expectation. With a critic that is only approximately right, the error of $V_w(s_t)$ acts as a baseline and adds no bias, but the error of $V_w(s_{t+1})$ does: the bias is $\gamma\, \mathbb{E}[V_w(s_{t+1}) - V^\pi(s_{t+1}) \mid s_t, a_t]$ times the score.

The *one-step actor–critic* algorithm makes two updates after every step (Figure 32):

1. compute the TD error (46), with $V_w = 0$ at a terminal state;
2. update the critic by semi-gradient TD (31), $w \leftarrow w + \eta_w\, \delta_t\, \nabla_w V_w(s_t)$;
3. update the actor along the score, weighted by the TD error, $\theta \leftarrow \theta + \eta\, \delta_t\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)$.

The TD error credits an action only with what it changed: a reward and a next state better or worse than the critic expected. The luck of the later actions no longer enters. An early actor–critic learned to balance a pole on a cart (Barto et al., 1983), the task of §10.

All the policy-gradient methods so far differ only in the weight they give the score of each action:

weight on $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$	**method**	**bias**	**noise**
G_t	REINFORCE (§11)	none	largest
$G_t - b(s_t)$	REINFORCE with a baseline (§12)	none	smaller
$A^\pi(s_t, a_t)$	the exact advantage (§12)	none	smallest
$r_t + \gamma V_w(s_{t+1}) - V_w(s_t)$	one-step actor–critic	from the critic	small

Actor–critic in the corridor

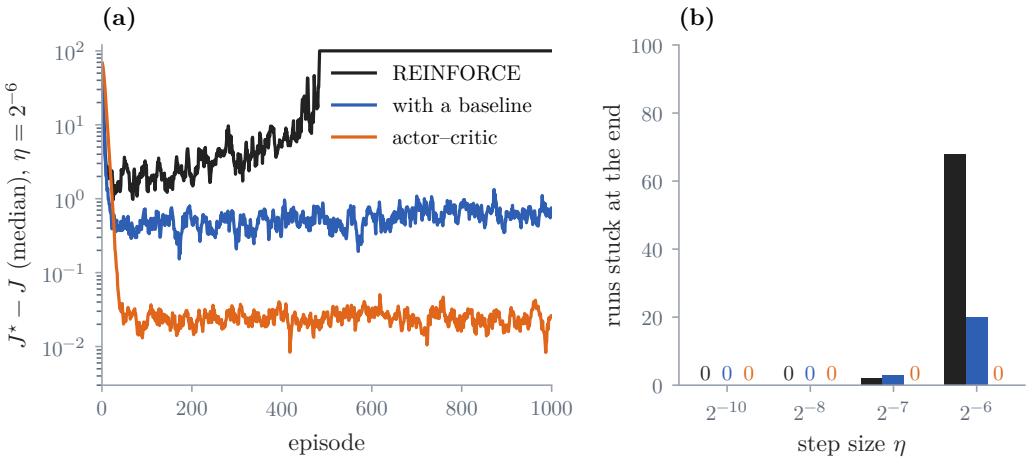


Figure 33. With a critic, no run gets stuck, even at the step size that breaks REINFORCE, and the runs end closer to the best policy. The short corridor from $p = 0.05$, 100 runs of 1,000 episodes per method, with episodes cut at 1,000 steps. (a) At $\eta = 2^{-6}$: how far the current policy is from the best, $J^\star - J$, the median over the runs (logarithmic axis; values above 100 drawn at 100), for REINFORCE (black), REINFORCE with a learned constant baseline (blue), and one-step actor–critic with a table of three state values, $\eta_w = 2^{-4}$ (orange). (b) The number of runs, of 100, that end stuck, with p below 0.01 or above 0.99, by step size.

In the corridor, the critic removes the failure of §11 (Figure 33). At $\eta = 2^{-6}$, 68 of 100 REINFORCE runs ended stuck. With a learned constant baseline 20 runs end stuck, and with

actor–critic none do. At no step size did an actor–critic run get stuck. The critic also brings the policy closer to the best one: after 1,000 episodes at $\eta = 2^{-6}$, the median run is 0.68 below the best return with the baseline, and 0.026 below it with the critic. Its critic here is a table of three numbers. The policy cannot tell the states apart, but the critic can be given the state, because it is used only in training, and the policy that acts still sees nothing. Appendix G lists every run.

Advantages from several steps

The TD error (46) is the advantage estimate with the most bias and the least noise; the return minus the critic's value, $G_t - V_w(s_t)$, is the one with no bias from the critic and the most noise. Between them, as in §7 (25), lie the estimates that use n rewards and then the critic. The n-step advantage is a sum of TD errors, because the critic's values in between cancel:

$$G_t^{(n)} - V_w(s_t) = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V_w(s_{t+n}) - V_w(s_t) = \sum_{l=0}^{n-1} \gamma^l \, \delta_{t+l}. \tag{48}$$

(Write each δ_{t+l} out by (46): the term $\gamma^{l+1} V_w(s_{t+l+1})$ of one cancels the term $-\gamma^{l+1} V_w(s_{t+l+1})$ of the next.) The **generalized advantage estimate** (GAE; Schulman et al., 2016) averages these estimates with the weights of TD(λ) (26), $(1-\lambda)\lambda^{n-1}$ for the n-step one. The TD error δ_{t+l} appears in every estimate with $n > l$, so its total weight is $(1-\lambda)\sum_{n>l} \lambda^{n-1} = \lambda^l$, and

$$\hat{A}_t = \sum_{l \geq 0} (\gamma\lambda)^l \, \delta_{t+l} = \delta_t + \gamma\lambda \, \hat{A}_{t+1}. \tag{49}$$

With $\lambda = 0$ it is the one-step TD error, and with $\lambda = 1$ it is $G_t - V_w(s_t)$. Values a little below 1, such as $\lambda = 0.95$, are common: they keep the bias small and remove part of the noise of the full return. The second form of (49) computes all the estimates of a finished batch in one backward pass, from $\hat{A} = 0$ after its last step. PPO (§14) usually estimates its advantages this way.

Many actors at once, and entropy

Actor–critic methods are on-policy, so each update needs fresh data from the current policy, and they cannot decorrelate it with a replay memory as DQN does; **A3C** (Mnih et al., 2016) instead runs many copies of the agent in parallel, each in its own copy of the environment, so that their experience together is far less correlated than one agent's and still comes from the current policy. Its synchronous form, **A2C**, waits for all the copies and makes one update from their combined batch.

A policy-gradient method can make its policy deterministic too early, and then it stops exploring. A common remedy adds to the objective (5) a small multiple β of the policy's entropy,

$$J(\theta) + \beta \, \mathbb{E}_s\big[H(\pi_\theta(\cdot \mid s))\big], \qquad H(\pi_\theta(\cdot \mid s)) = -\sum_a \pi_\theta(a \mid s) \log \pi_\theta(a \mid s), \tag{50}$$

which is largest for a uniform policy, so that the update is pushed to keep the policy random unless the rewards clearly favor one action. A3C used such a term.

For continuous actions, actor–critic methods can also learn off-policy, from a replay memory, with a critic that learns Q rather than V; DDPG (Lillicrap et al., 2016) and SAC (Haarnoja et al., 2018) are the best known.

14. Trust regions and PPO

Every method of §§11–13 uses a batch of experience for one gradient step and then throws it away, because the policy gradient (45) is an expectation under the current policy, and after one step the batch comes from a policy that no longer exists. Collecting experience is usually the expensive part of training. For a language model, a batch is thousands of sampled answers, each scored by a program or a reward model, and generating them costs far more than a gradient step on them. So it pays to take several steps on each batch. Two things are needed: a way to correct the gradient for the mismatch between the policy that collected the data and the policy being trained, and a limit on how far the policy may move before that correction can no longer be trusted. PPO provides both in a few lines, and it is the method that Week 7 adapts to language models.

Reusing a batch: importance sampling

Let π_{old} be the policy that collected the batch, and π_θ the policy being trained. An average under one distribution can be computed from samples of another, by weighting each sample with the ratio of its probabilities under the two. This is **importance sampling**:

$$\mathbb{E}_{a\sim\pi_\theta}[f(a)] = \sum_a \pi_\theta(a) f(a) = \sum_a \pi_{\text{old}}(a) \frac{\pi_\theta(a)}{\pi_{\text{old}}(a)} f(a) = \mathbb{E}_{a\sim\pi_{\text{old}}}\Big[\frac{\pi_\theta(a)}{\pi_{\text{old}}(a)} f(a)\Big]. \tag{51}$$

Weighting each sampled action by its **probability ratio**

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)} \tag{52}$$

gives the **surrogate objective**, the average over the batch's steps of the ratios times the advantages,

$$L(\theta) = \hat{\mathbb{E}}_t[\rho_t(\theta)\, \hat{A}_t], \tag{53}$$

where the advantages $\hat{A}_t$ are estimated once, under π_{old}, for example by GAE (49). At $\theta = \theta_{\text{old}}$ every ratio is 1, and since $\nabla_\theta \rho_t = \rho_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$,

$$\nabla_\theta L(\theta_{\text{old}}) = \hat{\mathbb{E}}_t[\hat{A}_t\, \nabla_\theta \log \pi_\theta(a_t \mid s_t)], \tag{54}$$

the policy-gradient estimate of (45). So the first step on the surrogate is an ordinary policy-gradient step, and later steps can reuse the batch.

The surrogate's error

The ratio corrects for the change in the actions only. The states in the batch are still the states that π_{old} visited, and the advantages are those of π_{old}. The exact change in the expected return is known, and it shows what the surrogate leaves out. With $d^\pi(s)$ the expected number of visits to s per episode under π, the *performance difference identity* (Kakade and Langford, 2002; Appendix D) says, for $\gamma = 1$,

$$J(\pi') - J(\pi) = \sum_s d^{\pi'}(s) \sum_a \pi'(a \mid s)\, A^\pi(s, a), \tag{55}$$

while the surrogate predicts

$$\sum_s d^\pi(s) \sum_a \pi'(a \mid s)\, A^\pi(s, a). \tag{56}$$

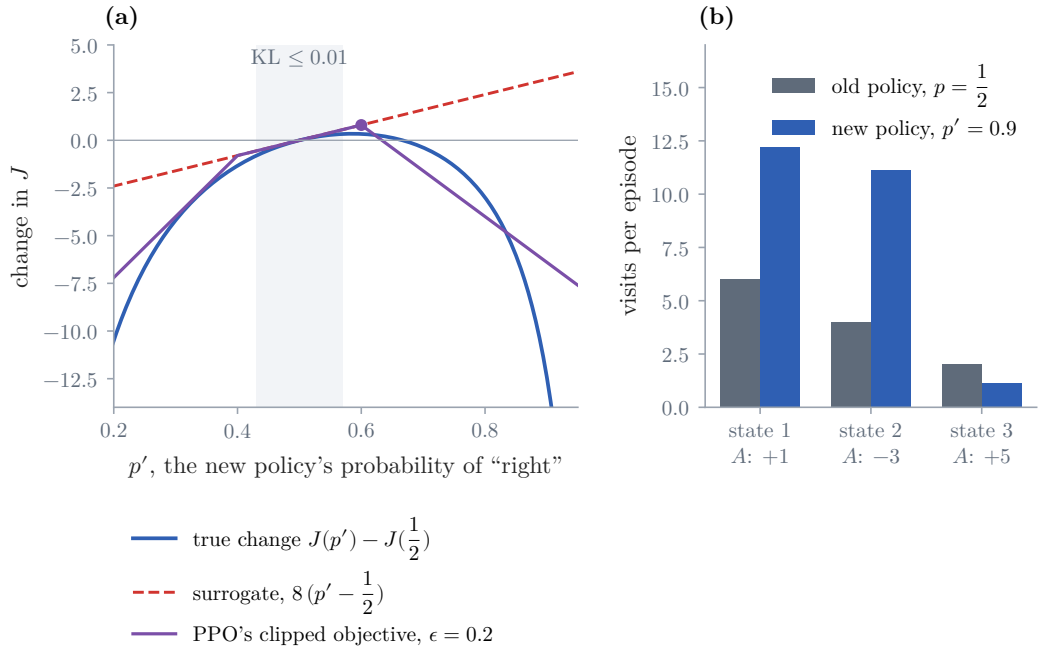


Figure 34. The surrogate is right near the old policy and wrong far from it; PPO's objective stops near where the true return peaks. The short corridor, from the policy $p = \frac{1}{2}$. (a) Against the new policy p': the true change of the expected return, $J(p') - J(\frac{1}{2})$ (blue); the surrogate's prediction (56), which keeps the old policy's visits to each state, $8(p' - \frac{1}{2})$ (red, dashed); and PPO's clipped objective with $\epsilon = 0.2$ (purple), largest at $p' = 0.6$. Shaded: the policies within a KL divergence of 0.01 of the old one. (b) The expected visits to each state per episode under the old policy and under $p' = 0.9$, with the advantage of "right" in each state at $p = \frac{1}{2}$.

They differ only in whose visits weight the states: the new policy's in the truth, the old policy's in the surrogate. Near π_{old} the visits barely change and the surrogate is accurate. Far from it, they can change completely.

In the corridor both (55) and (56) can be computed exactly (Figure 34). From $p = \frac{1}{2}$, the old policy visits the three states $d = (6, 4, 2)$ times per episode, and the advantages of "right" there are $A_{\text{R}} = (+1, -3, +5)$ (§4). At $p = \frac{1}{2}$ the advantages of the two actions in a state sum to zero, so "left" has advantage $-A_{\text{R}}$, and a new policy p' has $\sum_a \pi'(a \mid s) A^{\pi}(s, a) = (2p' - 1) A_{\text{R}}(s)$. The surrogate's prediction is therefore

$$\sum_s d(s)\,(2p' - 1)\,A_{\text{R}}(s) = (2p' - 1)(6 - 12 + 10) = 8(p' - \tfrac{1}{2}),$$

a straight line, which says that going right more often is always better. At $p' = 0.6$ it predicts $+0.8$, and the true change is $+0.33$. At $p' = 0.9$ it predicts $+3.2$, and the true change is -12.4. The new policy goes right so often that it shuttles between states 1 and 2, where "right" leads back, and visits the states 12.2, 11.1, and 1.1 times per episode. A step that trusts the surrogate too far ruins the policy, and with it the data for the next step.

Trust regions

TRPO (Schulman et al., 2015) keeps each update inside a **trust region**, a set of policies close enough to π_{old} for the surrogate (53) to be trusted. It measures closeness by the KL divergence between the old and the new action distributions, which Lecture 2 (Section 3) defined: 0 when two distributions are equal, and growing as they differ. TRPO solves

$$\max_{\theta} \; L(\theta) \quad \text{subject to} \quad \hat{\mathbb{E}}_t\Big[\mathrm{KL}(\pi_{\text{old}}(\cdot \mid s_t) \,\|\, \pi_\theta(\cdot \mid s_t))\Big] \le \kappa. \tag{57}$$

A small change in every state's action distribution usually changes the visits only a little, and with them the surrogate's error. (Not always: near $p = 1$ in the corridor, a small change of p changes the visits a great deal.) In Figure 34a, a bound of 0.01 allows the policies in the shaded band, where the surrogate is accurate. TRPO solves the constrained problem with second-order machinery: the natural gradient, conjugate gradients, and a line search (Appendix D). It works, but it is complicated, and it does not fit easily with networks whose policy and value share parameters.

PPO's clipped objective

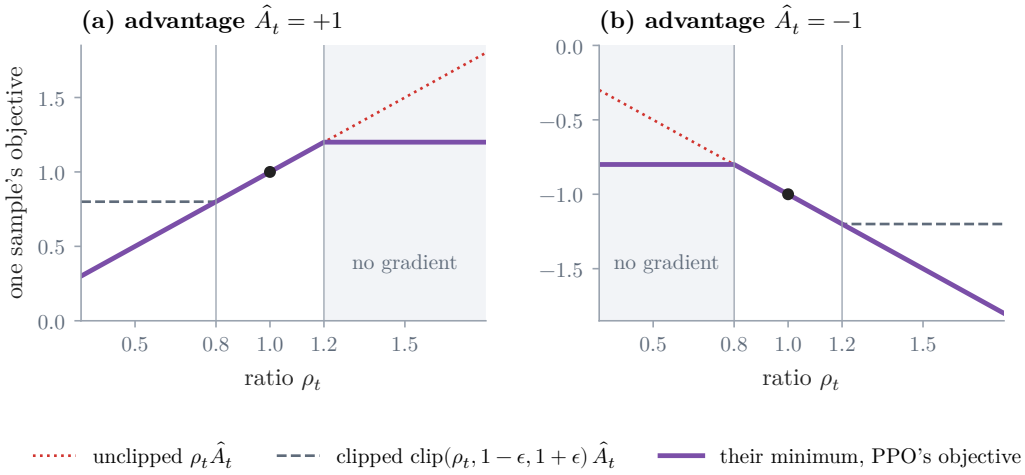


Figure 35. PPO stops rewarding a sample once its ratio has moved by ϵ in the direction its advantage favors. One sample's term in PPO's objective against its ratio ρ_t, with $\epsilon = 0.2$: the unclipped term $\rho_t \hat{A}_t$ (red, dotted), the clipped term (gray, dashed), and their minimum, the objective (purple). (a) A positive advantage: above $1 + \epsilon$, the objective is flat. (b) A negative advantage: below $1 - \epsilon$, it is flat. In the shaded regions the sample contributes no gradient. The dot is the old policy, $\rho_t = 1$. Redrawn after Schulman et al. (2017), Figure 1.

Proximal policy optimization (PPO; Schulman et al., 2017) keeps TRPO's aim with ordinary gradient steps and no constraint. It clips the ratio inside the objective:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\Big[\min\Big(\rho_t(\theta)\,\hat{A}_t,\; \mathrm{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_t\Big)\Big], \tag{58}$$

with $\epsilon = 0.2$ in the paper. Here $\mathrm{clip}(x, l, u)$ limits x to the interval from l to u; the ϵ is PPO's clip range, unrelated to the ε of ε-greedy exploration. The minimum is easier to read case by case. Taking $\hat{A}_t$ out of the minimum turns it into a minimum of the ratios when $\hat{A}_t > 0$, and a maximum when $\hat{A}_t < 0$:

$$\min(\rho_t \hat{A}_t, \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t) = \begin{cases} \hat{A}_t \,\min(\rho_t,\, 1+\epsilon) & \text{if } \hat{A}_t \ge 0, \\ \hat{A}_t \,\max(\rho_t,\, 1-\epsilon) & \text{if } \hat{A}_t < 0. \end{cases} \tag{59}$$

For a good action, $\hat{A}_t > 0$, the term rises with the ratio until $\rho_t = 1 + \epsilon$ and is flat beyond it (Figure 35a): once the action has been made 20% more likely than under the old policy, the sample asks for no more. For a bad action, $\hat{A}_t < 0$, the term is flat once the action has been made 20% less likely (Figure 35b). In the flat parts the sample's gradient is zero. Everywhere else it is the surrogate's (53), $\hat{A}_t \nabla_\theta \rho_t$. In particular, a sample whose ratio has moved the wrong way, making a good action less likely or a bad one more likely, is never clipped, so it pulls the policy back. In the corridor, the clipped objective (58) built at $p = \frac{1}{2}$ is largest at $p' = 0.6$, whose true return is within 0.01 of the best (Figure 34a).

In practice the actor and the critic are trained together, with one loss to minimize,

$$\mathcal{L}^{\text{PPO}}(\theta, w) = -L^{\text{CLIP}}(\theta) + c\,\hat{\mathbb{E}}_t\Big[\big(V_w(s_t) - \hat{G}_t\big)^2\Big] - \beta\,\hat{\mathbb{E}}_t\big[H(\pi_\theta(\cdot \mid s_t))\big], \tag{60}$$

where $\hat{G}_t = \hat{A}_t + V_{w_{\text{old}}}(s_t)$ is the critic's target, c weights the critic's squared error, and β the entropy bonus of (50). The algorithm:

1. run the current policy, $\pi_{\text{old}} = \pi_\theta$, to collect a batch of steps, for example a fixed number of steps from each of several copies of the environment;
2. estimate the advantages by GAE (49) with the critic, and normalize them over the batch to mean 0 and standard deviation 1;
3. take several epochs of gradient steps on (60), with Adam on minibatches of the batch;
4. repeat from step 1.

The PPO paper also gave a second version, which replaces the clipping (58) by a penalty on the KL divergence from π_{old} and adjusts the penalty's weight to keep the divergence near a target. Clipping did better in its experiments and became the default.

In the corridor, ten clipped epochs per batch were safe at step sizes at which ten unclipped epochs collapsed, but not at a large step, because clipping limits what each sample asks for, not the size of a gradient step. Nor does clipping keep every ratio inside $[1 - \epsilon, 1 + \epsilon]$, and PPO's published results also depend on details of its implementation (Appendix D.3).

PPO for language models

PPO is simple and works well across many tasks, which made it the default policy-gradient method. It trained OpenAI Five, the robot hand of §2, and InstructGPT. For a language model the policy is the model, an action is a token, an episode is an answer, and the reward usually comes once, at the end (§19). Week 7 changes (60) in two ways. The first adds to the loss a penalty that keeps the model near the model it started from, a *reference* policy π_{ref} that stays fixed through training:

$$+\beta_{\text{KL}}\,\hat{\mathbb{E}}_t\Big[\text{KL}(\pi_\theta(\cdot \mid s_t) \,\|\, \pi_{\text{ref}}(\cdot \mid s_t))\Big]. \tag{61}$$

It looks like TRPO's constraint (57), but it does a different job: the trust region keeps each update near the policy that collected the batch, which changes every iteration, while this penalty keeps the whole of training near π_{ref}, so that the model does not drift into text that games the reward. The second change, in GRPO (Shao et al., 2024), drops the critic. For each prompt it samples a group of K answers with rewards $r_1, \dots, r_K$, and gives every token of answer i the advantage

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \dots, r_K)}{\text{std}(r_1, \dots, r_K)}, \tag{62}$$

the reward relative to the other answers to the same prompt: a baseline of §12 (44), computed from the group instead of learned. The rest is PPO's clipped objective (58).

15. The methods so far, in equations

Parts 2 and 3 built about a dozen methods from a few equations. Collected, they come down to two forms, and they close the core of the lecture: every value-based method moves an estimate toward a target, and every policy-based method steps along one gradient, with different weights.

Value-based methods: one update, different targets

Every value-based method of Part 2 moves an estimate a step toward a target. They differ in the target, and in what computing it needs:

$$V(s_t) \leftarrow V(s_t) + \eta(\text{target} - V(s_t)) \quad \text{or} \quad Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \eta(\text{target} - Q(s_t, a_t)).$$

method	update	target	needs
policy evaluation (§6)	(16)	$\sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a)[r + \gamma V(s')]$	the model; $\eta = 1$
value iteration (§6)	(18)	$\max_a \sum_{s'} p(s' \mid s, a)[r + \gamma V(s')]$	the model; $\eta = 1$
Monte Carlo (§7)	(22)	G_t	the end of the episode
TD(0) (§7)	(24)	$r_t + \gamma V(s_{t+1})$	one step
SARSA (§9)	(29)	$r_t + \gamma Q(s_{t+1}, a_{t+1})$	the next action of the policy followed
Q-learning (§9)	(30)	$r_t + \gamma \max_{a'} Q(s_{t+1}, a')$	one step, from any policy
DQN (§10)	(32)	$r_t + \gamma \max_{a'} Q_{w^-}(s_{t+1}, a')$	a replay memory and a target network

The first two use the model to average over next states; the rest use samples. Monte Carlo waits for the return; the others bootstrap from their own estimates. The control methods act ε-greedily (28) on their action values, and with tables and suitable step sizes they converge to the right values (Theorem 9 for prediction, Theorems 10 and 11 for control; Appendix B.7). With a network in place of the table, the update is a semi-gradient step (31), $w \leftarrow w + \eta(\text{target} - Q_w(s_t, a_t))\nabla_w Q_w(s_t, a_t)$, and convergence is no longer guaranteed.

Policy-based methods: one gradient, different weights

Every policy-gradient method of Part 3 steps along an estimate of the same gradient (Theorem 6, Lemma 7). They differ in the weight Ψ_t on each action's score:

$$\nabla_\theta J \approx \mathbb{E}_\tau\Big[\sum_t \Psi_t \, \nabla_\theta \log \pi_\theta(a_t \mid s_t)\Big], \qquad \Psi_t \in \Big\{G_t, \;\; G_t - b(s_t), \;\; \textstyle\sum_{l \geq 0} (\gamma\lambda)^l \delta_{t+l}, \;\; \delta_t\Big\}, \quad (63)$$

for REINFORCE (41), REINFORCE with a baseline (44), GAE (49), and one-step actor–critic (46), with $\delta_t = r_t + \gamma V_w(s_{t+1}) - V_w(s_t)$ and $\gamma = 1$ as in our runs (with $\gamma < 1$, each term also carries the factor γ^t of Theorem 6). The first two are unbiased; the last two are exact only when $V_w = V^\pi$. In this order they go from no bias and the most noise to the most bias and the least noise, all estimating the advantage-weighted gradient (45). PPO reuses each batch for several epochs on the clipped surrogate (58), whose gradient at the old policy is the same estimate:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\Big[\min(\rho_t \hat{A}_t, \;\; \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon)\hat{A}_t)\Big], \qquad \rho_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

16. Planning with models

Part 4 asks one question: what did the systems that worked add to the methods of Parts 2 and 3, and where does reinforcement learning still fall short? Most of them added a model to plan with (§16) or demonstrations to start from (§17), and often both. §18 looks at the landmark systems, what they cost, and where reward and exploration still fail, and §19 applies the methods to language models.

Every method since §7 learned from experience alone, without a model of the environment. A **model** predicts the next state and the reward that follow an action. With one, an agent can **plan**: it can evaluate actions by imagining their consequences instead of trying them. Dynamic programming (§6) was planning with a known model. Models are also used in three other ways, and the strongest game-playing systems combine them with learned values and policies.

Learning a model, and Dyna

When the model is not given, it can be learned from the agent's own transitions, by supervised learning: from (s_t, a_t), predict s_{t+1} and r_t. **Dyna** (Sutton, 1991) learns such a model as it acts, and between real steps it trains its values on transitions generated by the model as well as on real ones. Each real step is then used many times, which saves experience. A learned model saves the most where experience is expensive, as with a physical robot.

Planning at decision time: tree search

A model can also be used at the moment of acting. Before each move, the agent searches ahead: it considers its actions, the states they lead to, the actions available there, and so on, and chooses the move whose consequences look best. A complete search to the end of the game is the exhaustive search of Figure 21, possible only for small games. Search programs for chess, such as Deep Blue, searched as deep as their time allowed and evaluated the positions at the frontier with a function written by people. Lecture 1 (Section 4) contrasted Deep Blue with AlphaGo, which learned its evaluation.

Go has too many moves in each position for a deep exhaustive search. **Monte Carlo tree search** (MCTS; Kocsis and Szepesvári, 2006; Coulom, 2006) grows a tree of continuations from the current position, selectively, by repeating one simulation many times (Figure 36a):

1. **select** a path down the tree, at each node taking the move whose average value so far, plus a bonus for moves that have been tried little, is largest;
2. **expand** the leaf at the end of the path by one move;
3. **evaluate** the new position, originally by playing the game to the end with fast random moves; AlphaGo averaged such a play-out with the estimate of a value network;
4. **back up** the result, adding it to the averages of every node on the path.

The selection rule is a bandit rule of §8 at every node: an average plus an optimistic bonus. After many simulations the search plays the move from the current position that it visited most. The tree concentrates on the moves that look promising, and every simulation sharpens the estimates along its path.

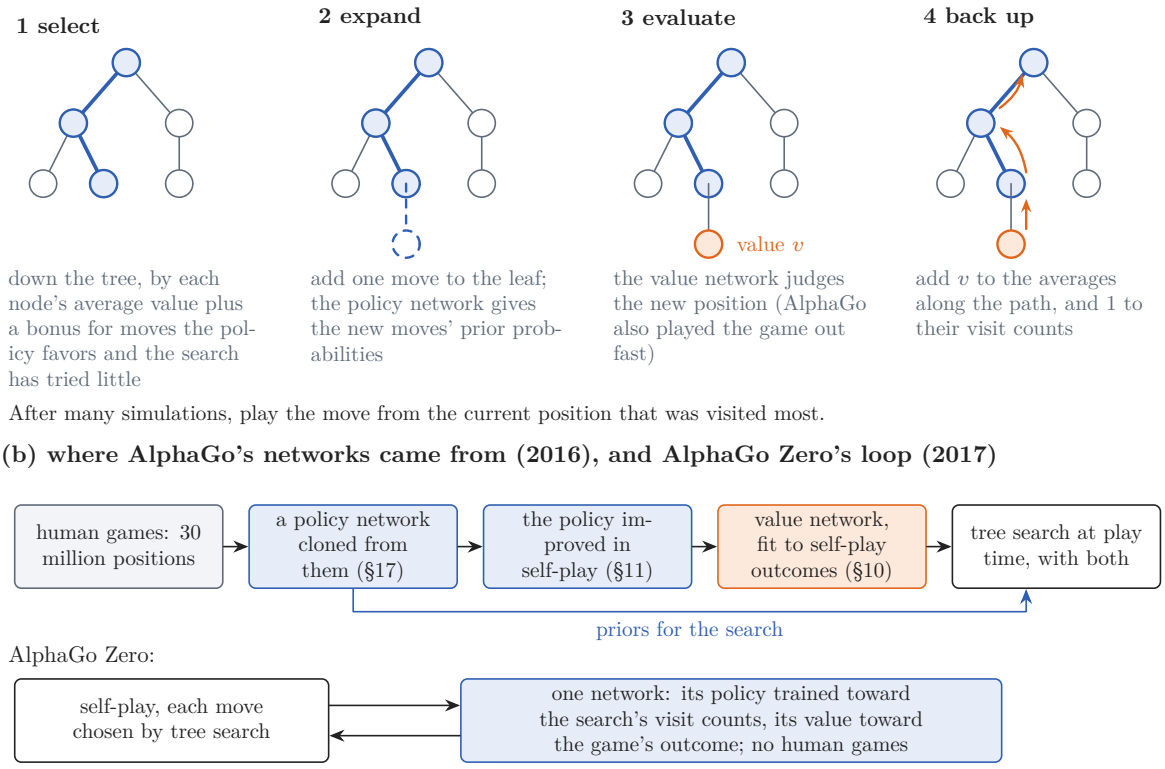


Figure 36. Search with learned networks, and where AlphaGo's networks came from. (a) One simulation of Monte Carlo tree search: select a path by average value plus an exploration bonus weighted by the policy network's prior (blue); expand one move; evaluate the new position with the value network (orange); back up the value along the path. (b) AlphaGo's networks and the methods that trained them (after Silver et al., 2016, Fig. 1), and AlphaGo Zero's single network (Silver et al., 2017).

AlphaGo to MuZero

AlphaGo (Silver et al., 2016) searched with the rules of Go as its model, and each of its networks came from a method of this lecture (Figure 36b). A policy network was trained to imitate the moves in 30 million positions from human games, which is behavior cloning (§17). It was improved by a policy gradient in games against earlier versions of itself (§11). A value network was trained by regression on the outcomes of those games (§10). At play time, the cloned policy network gave the search its prior probabilities for each move, and the value network, together with fast play-outs, evaluated the positions the search reached.

AlphaGo Zero (Silver et al., 2017) removed the human games. It used the fact that the search plays better than the network that guides it. Its single network's policy is trained to imitate the search's choices, the visit counts of the moves, and its value to predict the outcome of the game. A better network makes a better search, which provides better targets, in a loop that started from random play. AlphaGo Zero played 4.9 million games against itself in three days and beat the version of AlphaGo that had defeated Lee Sedol. AlphaZero (Silver et al., 2018) applied the same algorithm to chess and shogi. **MuZero** (Schrittwieser et al., 2020) learned a model good enough to search with, without being told the rules, and matched AlphaZero at Go, chess, and shogi while also learning Atari games.

World models

A model can also serve as a simulator in which to train a policy. **World models** learn to predict, in a compact learned representation of the observations, what the agent will observe next and what reward it will get, and the policy is then trained on imagined trajectories (Ha and Schmidhuber, 2018). The Dreamer agents train an actor–critic this way. One of them collected diamonds in Minecraft from scratch, a task that needs long sequences of actions with rare rewards (Hafner et al., 2025).

A model can save experience by orders of magnitude, but its errors compound over an imagined trajectory, as a cloned policy's errors do over a real one (§17). A policy trained in a flawed model learns to exploit the flaws.

17. Learning from demonstrations

Rewards are not the only way to teach an agent. An expert can show it what to do. Learning from demonstrations is supervised learning, so it is simpler and cheaper than learning from rewards, and several of the landmarks of §18 began with it. It also has a failure of its own, which explains why it is usually followed by reinforcement learning.

Behavior cloning

Behavior cloning collects demonstrations, the states an expert visited and the actions it took there, and fits the policy to them by maximum likelihood:

$$\max_{\theta} \sum_{(s,a)\in\mathcal{D}} \log \pi_\theta(a \mid s), \tag{64}$$

where $\mathcal{D}$ is the set of demonstrated state–action pairs. The fine-tuning of Lecture 5 (Section 13) was behavior cloning, with demonstrations written as conversations. It needs no reward and no exploration. Its weakness is the one Lecture 5 (Section 14) noted. The demonstrations show the states the expert visits, and the learner is trained only there. When the learner makes a mistake, it reaches a state that no demonstration contains, where its actions are poorly trained, so it is likely to err again, and errors compound along the episode. A cloned policy's expected total error can grow with the square of the episode's length (Ross et al., 2011).

DAgger

Ross et al. (2011) gave a remedy, **DAgger**, short for dataset aggregation. It collects labels where the learner goes, not only where the expert goes:

1. train a policy on the expert's demonstrations;
2. run the learner's policy, and ask the expert what it would have done in each state the learner visits;
3. add these labeled states to the dataset, and retrain the policy on all of it;
4. repeat from step 2.

With DAgger the total error grows only about linearly with the episode's length. Its cost is an expert who can be asked about any state, which a person cannot always provide.

A small experiment shows the problem and two ways around it (Figure 37). An expert walks a grid past four pits, and anyone who acts in it slips now and then. The learner copies the expert's action in each state its labels contain, and acts at random elsewhere.

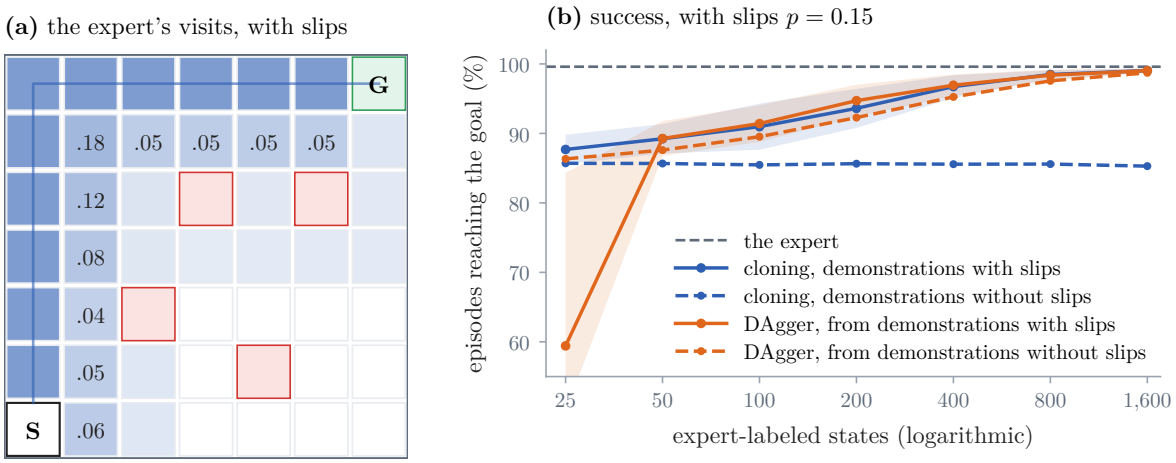


Figure 37. Cloning needs labels where the learner goes: DAgger asks for them, and demonstrations recorded in the slippery world supply them. A 7×7 grid from the bottom-left corner to the goal at the top right, past four pits (red); whoever acts, a slip replaces the chosen action by a random one with probability 0.15. The expert is the optimal policy. (a) Where the expert goes with slips: its expected visits to each cell per episode, over 2,000 episodes (darkest, about once: its path). (b) The share of 2,000 episodes that reach the goal, against the number of expert-labeled states (logarithmic axis): the expert (gray, dashed); cloning from demonstrations recorded with slips (blue) and without (blue, dashed); and DAgger (orange), which takes half its labels from demonstrations, with slips (solid) or without (dashed), and half from its own runs. Mean of 20 seeds; bands, one standard deviation.

1. **Demonstrations without slips.** If the demonstrations come from an expert that never slips, they show only its path, 12 states. Cloning then reaches the goal 86% of the time however many demonstrations it gets: the learner's own slips take it where no demonstration went. This is the failure DAgger is for. Starting from the same demonstrations, DAgger labels the states the learner's runs reach, and climbs from 86% with 25 labeled states to 98.7% with 1,600, toward the expert's 99.6%.
2. **Demonstrations with slips.** Demonstrations recorded in the slippery world help in another way. They also contain the states the expert's own slips lead to, which are the states the learner's slips lead to as well, so cloning from them improves with data, from 88% to 99.0%. Adding noise to the demonstrator on purpose, so that its demonstrations show how to recover from errors, is a known remedy, DART (Laskey et al., 2017). With such demonstrations, DAgger has little to add: it is within about 1 point of cloning from 50 labels on, and worse with 25 (59%), when its first runs spend labels on states far from the path.

What matters is that the labels cover the states the learner reaches. Noisy demonstrations supply them when the learner's mistakes resemble the demonstrator's slips. When its mistakes take it where no demonstration goes, only asking the expert about those states does. The explorer runs cloning against DAgger on this grid.

Other ways to use demonstrations

Inverse reinforcement learning (Ng and Russell, 2000) infers the reward that the expert seems to be maximizing, and then optimizes it. A reward can transfer to situations the demonstrations never covered, where a cloned policy cannot. **Offline reinforcement learning** (Levine et al., 2020) learns from a fixed dataset of past interactions, with their rewards, without acting at all. It must avoid trusting the values of actions the data never tried, which an off-policy method such as Q-learning would otherwise overestimate (§9).

Imitation first, then rewards

A cloned policy is rarely better than its demonstrations, and it errs where they are thin. A policy learned from rewards can surpass any demonstrator, but it needs rewards to arrive, and a random policy rarely earns them. So the landmark systems used the two in sequence. AlphaGo, AlphaStar, and ChatGPT began by cloning human data, which gave them a policy that earned rewards often enough to learn from, and reinforcement learning then improved it beyond the data (§18).

18. The landmarks, and the limits of reinforcement learning

With the methods of §§6–17 in place, the systems of §2 can be explained. Their successes, and the cost of those successes, also show where reinforcement learning is the right tool and where it is not.

How the landmark systems learned

System	Task	How it learned
Samuel's checkers (1959)	checkers	a linear evaluation of positions, moved toward the result of a search from a later position, in games against itself; moves chosen by search
MENACE (1961)	noughts and crosses	a table of move probabilities (beads in matchboxes), reinforced by each game's outcome: a policy learned from rewards
TD-Gammon (1992–1995)	backgammon	a neural network's evaluation of positions, trained by TD(λ) in games against itself (§10)
DQN (2013–2015)	49 Atari games	Q-learning with a convolutional network, a replay memory, and a target network; ε-greedy exploration (§10)
AlphaGo (2016)	Go	behavior cloning on human games; a policy gradient in self-play; a value network trained on the outcomes; tree search with both at play time (§16)
AlphaGo Zero, AlphaZero (2017–2018)	Go; chess and shogi	self-play with tree search from random play, the network trained toward the search's choices; no human games (§16)
OpenAI Five (2019)	Dota 2	PPO in self-play, at very large scale (§14)
AlphaStar (2019)	StarCraft II	behavior cloning on human games, then actor–critic learning in a league of agents that play each other
Rubik's cube hand (2019)	robot manipulation	PPO in simulation, with the simulations varied at random so that the policy transfers to a real hand

System	Task	How it learned
MuZero (2020)	Go, chess, shogi, Atari	AlphaZero's search with a learned model in place of the rules (§16)
InstructGPT, ChatGPT (2022)	following instructions	behavior cloning on demonstrations; a reward model learned from people's comparisons; PPO with a penalty for moving away from the cloned model
DeepSeek-R1-Zero, DeepSeek-R1 (2025)	mathematics and code	a policy gradient with baselines from groups of answers (GRPO), on rewards that a program checks; R1 after brief fine-tuning on worked examples

Sources: Samuel (1959); MENACE (Michie, 1963); TD-Gammon (Tesauro, 1995); DQN (Mnih et al., 2015); AlphaGo (Silver et al., 2016); AlphaGo Zero (Silver et al., 2017) and AlphaZero (Silver et al., 2018); OpenAI Five (Berner et al., 2019); AlphaStar (Vinyals et al., 2019); the robot hand (OpenAI et al., 2019); MuZero (Schrittwieser et al., 2020); InstructGPT (Ouyang et al., 2022); DeepSeek-R1 (DeepSeek-AI, 2025).

The table shows four patterns.

1. **Search and learning together.** The strongest game players learn a policy and a value, and search with both at every move (§16). Samuel's program already combined the two.
2. **Self-play.** In a game, the agent's opponent can be a copy of itself, so the opponent improves as the agent does, and experience costs only computation. TD-Gammon, AlphaZero, OpenAI Five, and AlphaStar's league all learned this way.
3. **Imitation first, then rewards**, as §17 described.
4. **Simple methods, at scale.** DQN and PPO are short algorithms. The landmarks came from running them with deep networks on vast amounts of simulated experience (Figure 38).

Where it works

Reinforcement learning is worth its difficulty when three conditions hold.

1. **Decisions in sequence, judged by a result.** The task is a sequence of choices whose quality shows in an outcome that can be scored, such as winning a game, finishing a lap, or giving a correct answer, while the right choice at each step is hard to write down. When the right outputs can be labeled, supervised learning is simpler and cheaper.
2. **Cheap experience.** The agent can try things many times. Every game and robot landmark of the table since TD-Gammon had a game engine or a simulator.
3. **A competent start.** Rewards teach only what the agent sometimes does (§8), so a policy cloned from demonstrations, or a pretrained model, is often the starting point (§17).

Where it falls short

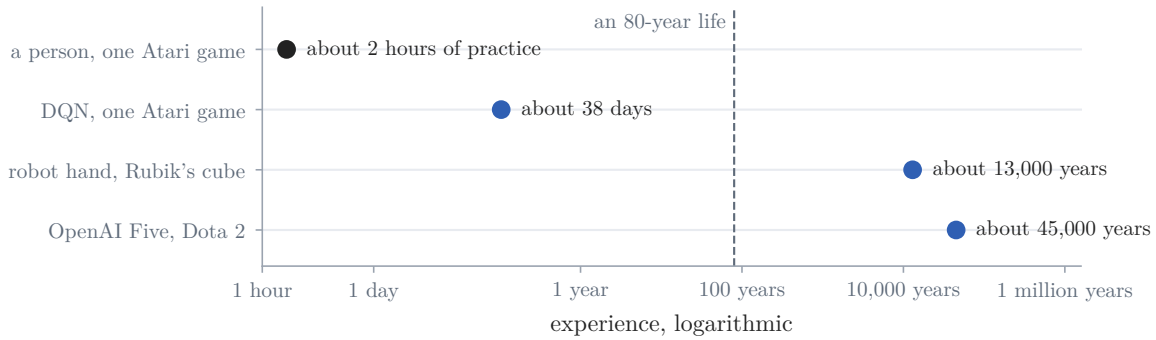
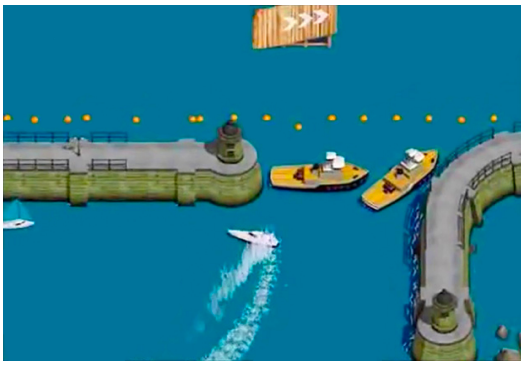


Figure 38. The landmarks needed far more experience than a person. Published figures on one logarithmic axis: a person's practice before the human scores DQN was compared with, and DQN's training, for one Atari game (Mnih et al., 2015); the simulated experience of the robot hand that solved a Rubik's cube (OpenAI et al., 2019, Section 6.3); and OpenAI Five's self-play (OpenAI, 2019). AlphaGo Zero, whose experience is counted in games, is not shown.

(a) CoastRunners



(b) Montezuma's Revenge

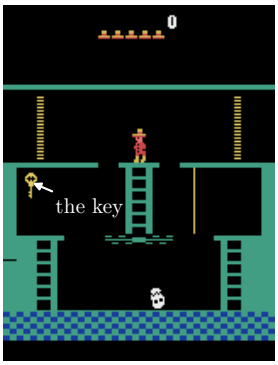


Figure 39. A reward can be gamed, and a sparse one may never be found. (a) CoastRunners, the boat-racing game in which an agent rewarded for hitting targets learned to circle a lagoon instead of finishing the race. Reproduced from Clark and Amodei (2016). (b) The first room of Montezuma's Revenge, rendered by the Arcade Learning Environment (Bellemare et al., 2013): the first reward is the key, at the end of a precise series of jumps and climbs, and DQN never reached it.

1. **It needs a great deal of experience** (Figure 38). DQN trained on 50 million frames of each Atari game, about 38 days of play, where the human testers it was compared with had practiced each game for about two hours (Mnih et al., 2015). The robot hand of §2 used about 13,000 years of simulated experience (OpenAI et al., 2019), and OpenAI Five about 45,000 years of Dota 2 (OpenAI, 2019). Where experience is slow, costly, or dangerous, as for a robot or a patient, this rules much of reinforcement learning out, or forces training in simulation.
2. **The reward is the specification, and agents exploit it.** An agent maximizes the reward it is given, not the intention behind it. In the boat-racing game CoastRunners (Figure 39a), an agent rewarded for hitting targets along the course learned to circle in a lagoon, hitting the same targets as they reappeared, catching fire, and never finishing the race, while scoring more than human players (Clark and Amodei, 2016). Krakovna et al. (2020) collected dozens of such cases of **specification gaming**. Week 7 meets its form in language models, reward hacking.

3. **Sparse rewards make exploration hard.** When the reward comes only after a long, specific sequence of actions, random exploration may never find it. DQN scored 0 on Montezuma's Revenge, an Atari game whose first reward needs a precise series of jumps and climbs (Figure 39b). With a bonus for reaching states it could not yet predict, random network distillation became the first method to beat the average human score on it without demonstrations (Burda et al., 2018), and Go-Explore, which returns to promising states before exploring from them, scored far higher (Ecoffet et al., 2021). Our chat model never writes a correct sum of two four-digit numbers, so rewards for such sums would all be 0, and could teach it nothing (§19).
4. **Training is unstable and hard to reproduce.** The data change as the policy changes, and values learned with function approximation can diverge (§10). Our DQN on CartPole balanced the pole for 500 steps and then lost the skill again (§10). Henderson et al. (2018) found that the same algorithm, run with different random seeds or in different implementations, gave significantly different results; that choices such as the network's size or the rewards' scale changed performance a great deal; and that which method did best depended on the environment.
5. **What is learned in simulation may not transfer.** A policy trained in a simulator learns the simulator, flaws included. Training on many randomized versions of the simulation, as for the robot hand, is one remedy.
6. **Credit is hard to assign over long horizons.** With one reward at the end of a long episode, every earlier action shares the credit, and the estimates are noisy (§§12–13).

Irpan (2018) wrote a widely read account of these difficulties in deep reinforcement learning. In practice they mean that rewards are used last: when the right outputs can be labeled or demonstrated, supervised learning comes first, and reinforcement learning improves on its result.

19. From games to language models

A language model is a policy, and the methods of Part 3 apply to it directly. With rewards from a program that checks its sums, our chat model of Lecture 5 can be trained by REINFORCE, and the run shows what Week 7 must add.

An answer as an episode

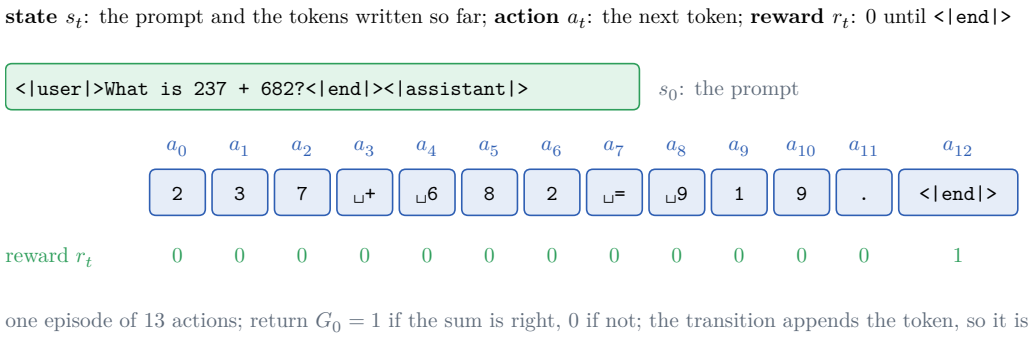


Figure 40. An answer is an episode: one token per action, and one reward at the end. Our chat model writing the answer to *What is 237 + 682?* as a Markov decision process. The first state is the prompt in the chat template; each action a_t is a token (blue; the mark before a digit is a space); each state is the prompt and the tokens so far; every reward is 0 until <|end|>, after which the program's check gives 1 or 0. The answer shown is the right one, tokenized by our tokenizer.

A language model writing an answer is an MDP with a simple structure (Figure 40).

1. The **state** is the prompt and the tokens written so far.
2. An **action** is the next token, and the **policy** is the model's next-token distribution, the softmax (33) over logits of Lectures 2–5.
3. The **transition** appends the chosen token to the text. It is deterministic and known exactly.
4. The **reward** is 0 after every token but the last. After `<|end|>`, it is the score of the whole answer: here, 1 if a program finds the right sum after "=", and 0 otherwise.

An episode is one answer, a few to a few thousand tokens long, with $\gamma = 1$; seen whole, the answer is also one action of a contextual bandit (§8), and §14 showed how GRPO (62) uses that view for its advantage while PPO with a critic scores every token.

A language model explores by sampling (§8). At temperature 1 it writes each answer with the probability its policy gives it. A lower temperature exploits more, and temperature 0, always the most probable token, does not explore at all.

REINFORCE on answers

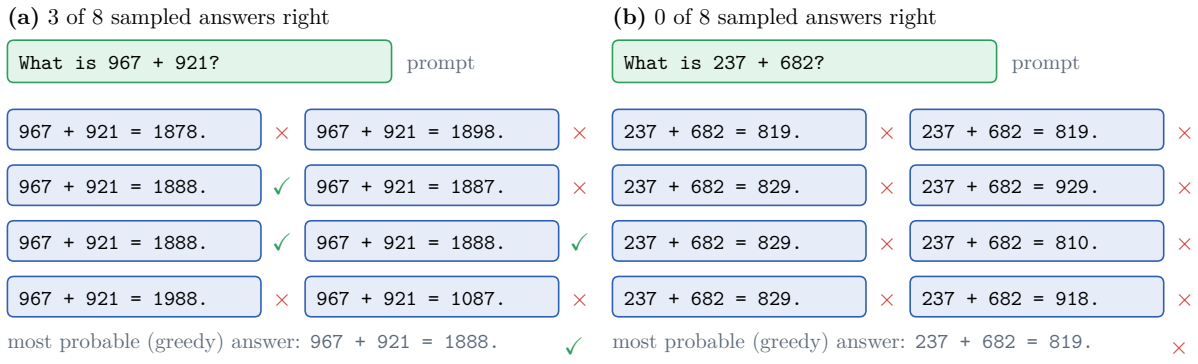


Figure 41. Our chat model is sometimes right, and a program can tell when. Lecture 5's chat model asked two sums from its held-out set (none of them occurs in its training data), eight answers sampled at temperature 1 for each, and a check that the number after "=" is the sum (green tick, a reward of 1; red cross, 0). (a) The second held-out sum: three of eight answers are right, and the most probable answer is right. (b) The first: none of eight is right, and neither is the most probable answer. The answers are the model's own, saved by the experiment script.

Our chat model gets 73% of the 200 held-out sums of two three-digit numbers right with its most probable answer, as Lecture 5 (Section 14) found, and 47% of the answers it samples at temperature 1 are right. Figure 41 shows eight sampled answers to each of two sums. A program can mark each answer right or wrong. Fine-tuning has no use for the mark, because it needs the right answer written out. REINFORCE can use it.

For one answer $y = (y_1, \dots, y_T)$ to a prompt x, with reward r, REINFORCE's loss (§11) is

$$\mathcal{L}(\theta) = -r \sum_{t=1}^{T} \log \pi_\theta(y_t \mid x, y_{<t}), \tag{65}$$

where π_θ is the model's next-token distribution. This is the loss of the fine-tuning in Lecture 5, with two changes (Figure 42). The answer is the model's own, sampled from it, instead of a demonstration. And the loss is multiplied by the answer's reward. With rewards of 1 and 0, REINFORCE fine-tunes the model on its own right answers and ignores its wrong ones. With

(a) fine-tuning (Lecture 5): $-\sum_t \log \pi_\theta(a_t \mid s_t)$ on a demonstration, weight 1

`967 + 921 = 1888.` weight 1

(b) REINFORCE: $-\sum_t w \log \pi_\theta(a_t \mid s_t)$ on the model's own answers, with weight $w = r$ or $w = r - b$

`967 + 921 = 1878.`	×	$r = 0,\ r - b = -0.43$	`967 + 921 = 1898.`	×	$r = 0,\ r - b = -0.43$
`967 + 921 = 1888.`	✓	$r = 1,\ r - b = +0.71$	`967 + 921 = 1887.`	×	$r = 0,\ r - b = -0.43$
`967 + 921 = 1888.`	✓	$r = 1,\ r - b = +0.71$	`967 + 921 = 1888.`	✓	$r = 1,\ r - b = +0.71$
`967 + 921 = 1988.`	×	$r = 0,\ r - b = -0.43$	`967 + 921 = 1087.`	×	$r = 0,\ r - b = -0.43$

b: the mean reward of the other seven answers to the same prompt

Figure 42. REINFORCE is the fine-tuning loss on the model's own answers, weighted by their rewards. (a) The loss of supervised fine-tuning (Lecture 5 (Section 13)) on one demonstration: every token's log-probability, weight 1. (b) REINFORCE's loss on eight answers our chat model sampled to the same prompt (Figure 41a), each weighted by its reward r (1 for the three right answers, 0 for the five wrong ones), or by $r - b$ with a baseline b, here the mean reward of the other seven answers: a right answer is then pushed up by 0.71, and a wrong one down by 0.43.

a baseline b (44), the weight becomes $r - b$: right answers are pushed up and wrong ones down. A natural baseline for a language model is the mean reward of the other answers sampled for the same prompt, which needs no critic (Kool et al., 2019; Ahmadian et al., 2024).

Our chat model, trained with REINFORCE

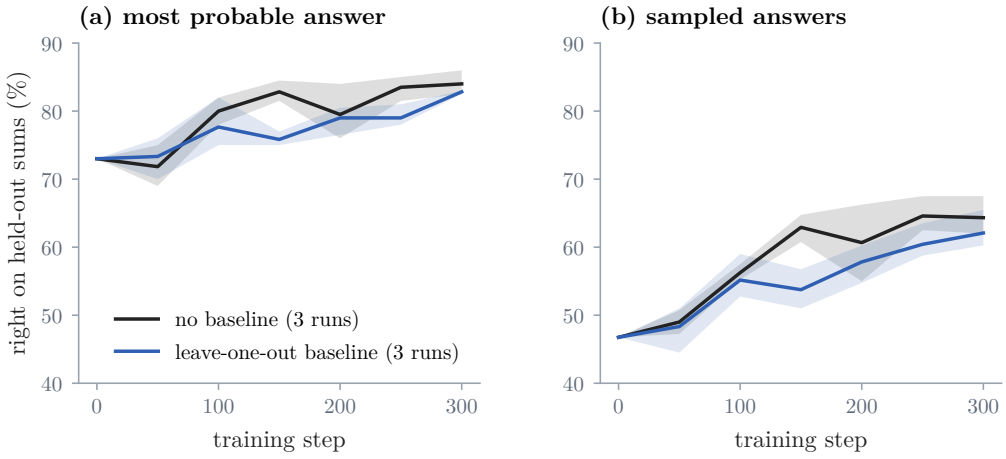


Figure 43. REINFORCE raises our chat model's accuracy on held-out sums. Our chat model trained by REINFORCE on sums of two three-digit numbers drawn at random (the held-out pairs excluded): each step, 8 prompts and 8 answers to each, sampled at temperature 1; reward 1 for a right sum; Adam at 3×10^{-5}; 300 steps. With the leave-one-out baseline (blue) and without a baseline (black), 3 runs each (lines: mean; band: lowest to highest). (a) The most probable (greedy) answer's accuracy on the 200 held-out sums. (b) The accuracy of sampled answers, 2 per held-out sum.

We trained the chat model with REINFORCE on sums of two random three-digit numbers, excluding the held-out pairs. Each step sampled 8 answers to each of 8 prompts, and each answer's baseline was the mean reward of the other seven answers to the same prompt, a leave-one-out baseline (Figure 43). Over 300 steps, 19,200 sampled answers and 16 to 21 minutes

on one processor core, the accuracy of the most probable answer on the held-out sums went from 73% to 83% (mean of 3 runs), and that of sampled answers from 47% to 62%. None of the held-out sums was in the training. On the sum of Figure 41b, which none of eight sampled answers got right before training, five of eight were right after it. Here the baseline made no difference: without it the runs reached 84%. Appendix C explains why.

Two limits of §18 appear on a small scale. On sums of two four-digit numbers the model was never right before training, 0 of 800 sampled answers, and it is still never right after it. Every reward for such a sum was 0, so there was nothing to learn from. And the reward says nothing about anything but sums.

The reference policy

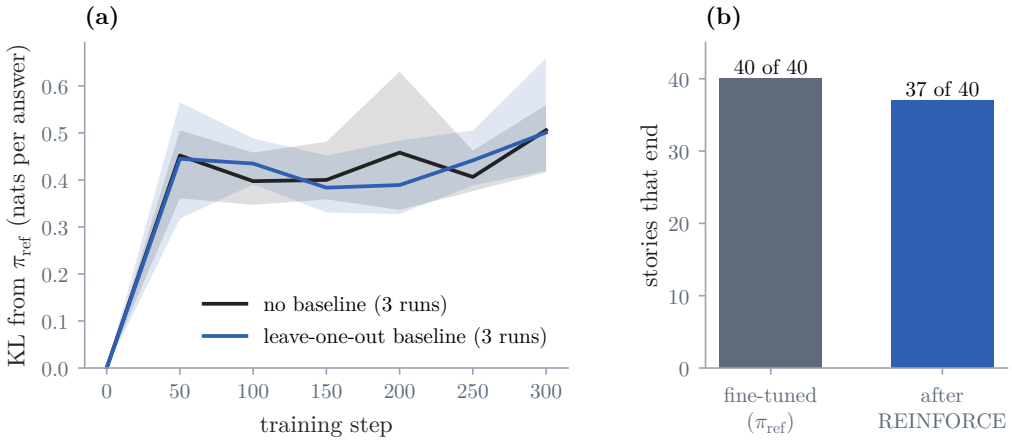


Figure 44. Rewards for sums move our chat model away from the fine-tuned model it started from. The runs of Figure 43. (a) The KL divergence of the trained policy from the fine-tuned model (the reference policy), per answer, against training steps, measured every 50 steps on 400 sampled answers, two per held-out sum; lines, the mean of 3 runs; bands, lowest to highest. (b) Forty held-out story requests answered by the fine-tuned model and by the model after 300 steps with the baseline (one run, one answer each at temperature 0.8): the number of stories that end with <|end|>; the 3 that do not end ran to the 400-token limit.

Nothing in a reward for sums protects what fine-tuning taught the model about anything else. Training moved the model 0.50 nats per answer away from the fine-tuned model it started from, measured on the held-out sums, much of it in the first 50 steps (Figure 44a). We measured the cost elsewhere once. In one run, 3 of 40 answers to story requests ran past the length limit without ending, against none before training (Figure 44b), too few to show that the stories changed.

This is what the penalty toward the reference policy of §14, (61), is for: it keeps the whole run close to the fine-tuned model it improves, while PPO's clipping (58) keeps each update close to the policy that sampled its batch.

20. Summary, and next week

An agent learns to act from a reward for its own actions rather than from examples of the right action. The reward may come late, and the agent learns only about what it tries. Three assumptions make the problem learnable, and they define the Markov decision process: a Markov state, stationary dynamics, and a goal that is a sum of rewards. Values say how much reward to expect, and the Bellman equations (10) and (11) tie each value to the values one step later; one deterministic policy, greedy with respect to $Q^\star$, is optimal in every state at once (§§1–5).

Value-based methods estimate values and act on them. With a model, dynamic programming evaluates and improves a policy until it is optimal. Without one, Monte Carlo averages returns, TD bootstraps from its own estimates, and SARSA and Q-learning learn action values while exploring; with a network, DQN needs a replay memory and a target network to keep its values from diverging (§§6–10). Policy-based methods follow the policy gradient (42), which needs no gradient of the rewards or the environment. REINFORCE estimates it from returns, a baseline and the advantage remove much of its noise, a critic estimates the advantage, and PPO's clipped objective (58) lets one batch serve several updates (§§11–14). §15 puts all of them in two tables.

The landmark systems combined these methods with search, self-play, and imitation, at a scale of experience far beyond a person's, and reinforcement learning still struggles where experience is costly, rewards are sparse or can be gamed, or training is unstable (§§16–18). A language model is a policy, and REINFORCE on its own answers raised our chat model from 73% to 83% on held-out sums, while moving it 0.50 nats per answer from the model it started from (§19).

Next week applies this to our chat model. Training with rewards is the last stage of the recipe that made ChatGPT, described in Lecture 5 (Section 1). Week 7 starts from §19's run and adds what that run left out: a penalty toward the reference policy (61), with a measure of what it preserves, and GRPO's normalized group baselines (62) and clipping (58). Rewards for what no program can check are learned from people's comparisons of two answers (the Bradley–Terry model and reward models). The reward is maximized with a penalty on the KL divergence from the fine-tuned model, whose best policy has a closed form, and that form gives direct preference optimization (DPO), which learns from preferences without a reward model or sampling. Rewards that a program can check train reasoning models with GRPO. We will also see how these methods fail: rewards that can be gamed, answers that grow longer, and models that agree too readily.

Resources

Videos. Gonkee, *The FASTEST introduction to Reinforcement Learning on the internet* (2024), an hour and a half from Markov decision processes and Monte Carlo through TD, deep Q-networks, and policy gradients to neuroscience and the field's limits. David Silver's *UCL Course on Reinforcement Learning* (2015), ten lectures; Lecture 1 introduces the problem and its vocabulary. Sergey Levine's *CS 285: Deep Reinforcement Learning* (Berkeley) for deep reinforcement learning in depth.

Interactive. The reinforcement learning explorer of this lecture: gridworlds with value iteration, SARSA, and Q-learning, cliff walking, Monte Carlo against TD, bandits, the policy gradient, and PPO's clipped objective. Andrej Karpathy's REINFORCEjs: dynamic programming and TD learning on a gridworld, and deep Q-networks, in the browser. Arthur Juliani's Beginner's RL Playground (2025): a gridworld you can edit, with Q-learning, SARSA, expected SARSA, Monte Carlo control, actor–critic, and the successor representation.

Books and guides. Sutton and Barto (2018), free online, the standard textbook. OpenAI's *Spinning Up in Deep RL* (2018), the key algorithms of deep reinforcement learning with code. Hugging Face's *Deep Reinforcement Learning Course*, with notebooks.

Background reading

The basics of this lecture are developed in full in the RL module of the Background Notes on Machine Learning and AI:

Topic	Background Notes	Textbook or article
Markov decision processes; dynamic programming	RL 1, RL 2	Sutton and Barto, Chapters 3–4
Bandits and exploration	RL 3	Sutton and Barto, Chapter 2; Lattimore and Szepesvári, *Bandit Algorithms*
Monte Carlo and TD; SARSA and Q-learning	RL 5, RL 6, RL 7	Sutton and Barto, Chapters 5–6
Deep Q-networks	RL 16	Mnih et al. (2015)
Policy gradients and actor–critic	RL 13; Lab 6	Sutton and Barto, Chapter 13; *Spinning Up*, Part 3
Trust regions and PPO	RL 20; Lab 10	Huang et al., *The 37 Implementation Details of PPO*
Imitation and offline learning	RL 25, RL 26	Ross et al. (2011)
Models, planning, and search	RL 10, RL 23, RL 24	Sutton and Barto, Chapter 8
Reinforcement learning for language models	RL 28	Week 7

Chapters 3 and 13 of Sutton and Barto are the reading for this week, with Schulman et al. (2017) on PPO, which is short.

References and further reading

Reinforcement learning and its history (§§1–2, 18, Appendix F)

- **Richard S. Sutton and Andrew G. Barto (2018), Reinforcement Learning: An Introduction, second edition.** Book.
- **Gonkee (2024), The FASTEST Introduction to Reinforcement Learning on the Internet.** Video.
- **Edward L. Thorndike (1898), Animal Intelligence: An Experimental Study of the Associative Processes in Animals.** Paper.
- **Richard Bellman (1957), Dynamic Programming.** Book.
- **Arthur L. Samuel (1959), Some Studies in Machine Learning Using the Game of Checkers.** Paper.
- **Donald Michie (1963), Experiments on the Mechanization of Game-Learning, Part I.** Paper.
- **Paul F. Christiano et al. (2017), Deep Reinforcement Learning from Human Preferences.** Paper.

- **Nisan Stiennon et al. (2020), Learning to Summarize from Human Feedback.** Paper.
- **Andrew G. Barto, Richard S. Sutton, and Charles W. Anderson (1983), Neuronlike Adaptive Elements That Can Solve Difficult Learning Control Problems.** Paper.
- **Gerald Tesauro (1995), Temporal Difference Learning and TD-Gammon.** Paper.
- **Volodymyr Mnih et al. (2015), Human-level Control through Deep Reinforcement Learning.** Paper.
- **David Silver et al. (2016), Mastering the Game of Go with Deep Neural Networks and Tree Search.** Paper.
- **David Silver et al. (2017), Mastering the Game of Go without Human Knowledge.** Paper.
- **David Silver et al. (2018), A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play.** Paper.
- **David Silver, Thomas Hubert, et al. (2017), Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm** (the arXiv version of AlphaZero). Paper.
- **Christopher Berner et al. (2019), Dota 2 with Large Scale Deep Reinforcement Learning.** Paper.
- **OpenAI (2019), OpenAI Five Defeats Dota 2 World Champions.** Article.
- **Oriol Vinyals et al. (2019), Grandmaster Level in StarCraft II Using Multi-agent Reinforcement Learning.** Paper.
- **OpenAI et al. (2019), Solving Rubik's Cube with a Robot Hand.** Paper.
- **Julian Schrittwieser et al. (2020), Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model.** Paper.
- **Jonas Degrave et al. (2022), Magnetic Control of Tokamak Plasmas through Deep Reinforcement Learning.** Paper.
- **Elia Kaufmann et al. (2023), Champion-level Drone Racing Using Deep Reinforcement Learning.** Paper.

Values, exploration, and control (§§3–10, Appendices B and E)

- **Stuart Russell and Peter Norvig (2020), Artificial Intelligence: A Modern Approach, fourth edition**, Chapter 17. Book.
- **Andrew Y. Ng, Daishi Harada, and Stuart Russell (1999), Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping.** Paper.
- **Dimitri P. Bertsekas and John N. Tsitsiklis (1996), Neuro-Dynamic Programming.** Book.
- **Herbert Robbins and Sutton Monro (1951), A Stochastic Approximation Method.** Paper.
- **Richard S. Sutton (1988), Learning to Predict by the Methods of Temporal Differences.** Paper.
- **Christopher J. C. H. Watkins (1989), Learning from Delayed Rewards.** Thesis.
- **Christopher J. C. H. Watkins and Peter Dayan (1992), Q-learning.** Paper.

- **Tommi Jaakkola, Michael I. Jordan, and Satinder P. Singh (1994), On the Convergence of Stochastic Iterative Dynamic Programming Algorithms.** Paper.
- **Satinder Singh, Tommi Jaakkola, Michael L. Littman, and Csaba Szepesvári (2000), Convergence Results for Single-Step On-Policy Reinforcement-Learning Algorithms.** Paper.
- **John N. Tsitsiklis and Benjamin Van Roy (1997), An Analysis of Temporal-Difference Learning with Function Approximation.** Paper.
- **Gavin A. Rummery and Mahesan Niranjan (1994), On-Line Q-Learning Using Connectionist Systems.** Report.
- **Wolfram Schultz, Peter Dayan, and P. Read Montague (1997), A Neural Substrate of Prediction and Reward.** Paper.
- **Wolfram Schultz (2016), Dopamine Reward Prediction Error Coding.** *Dialogues in Clinical Neuroscience* 18(1), 23–32. Paper.
- **Ryunosuke Amo et al. (2022), A Gradual Temporal Shift of Dopamine Responses Mirrors the Progression of Temporal Difference Error in Machine Learning.** Paper.
- **Matteo Hessel et al. (2018), Rainbow: Combining Improvements in Deep Reinforcement Learning.** Paper.
- **Hado van Hasselt (2010), Double Q-learning.** Paper.
- **Hado van Hasselt, Arthur Guez, and David Silver (2016), Deep Reinforcement Learning with Double Q-Learning.** Paper.
- **Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver (2016), Prioritized Experience Replay.** Paper.
- **Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas (2016), Dueling Network Architectures for Deep Reinforcement Learning.** Paper.
- **Marc G. Bellemare, Will Dabney, and Rémi Munos (2017), A Distributional Perspective on Reinforcement Learning.** Paper.
- **Greg Brockman et al. (2016), OpenAI Gym.** Paper.
- **Mark Towers et al. (2024), Gymnasium: A Standard Interface for Reinforcement Learning Environments.** Paper; code (MIT license).
- **Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling (2013), The Arcade Learning Environment: An Evaluation Platform for General Agents.** Journal of Artificial Intelligence Research 47, 253–279. Paper; code (GPL-2.0).
- **William R. Thompson (1933), On the Likelihood that One Unknown Probability Exceeds Another in View of the Evidence of Two Samples.** Paper.
- **Tze Leung Lai and Herbert Robbins (1985), Asymptotically Efficient Adaptive Allocation Rules.** Paper.
- **Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer (2002), Finite-time Analysis of the Multiarmed Bandit Problem.** Paper.
- **Shipra Agrawal and Navin Goyal (2012), Analysis of Thompson Sampling for the Multi-armed Bandit Problem.** Paper.
- **Tor Lattimore and Csaba Szepesvári (2020), Bandit Algorithms.** Book.

Policy gradients, actor–critic, and PPO (§§11–15, Appendices A, C, and D)

- **Ronald J. Williams (1992), Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning.** Paper.
- **Richard S. Sutton, David McAllester, Satinder Singh, and Yishay Mansour (2000), Policy Gradient Methods for Reinforcement Learning with Function Approximation.** Paper.
- **Lex Weaver and Nigel Tao (2001), The Optimal Reward Baseline for Gradient-Based Reinforcement Learning.** Paper.
- **Evan Greensmith, Peter L. Bartlett, and Jonathan Baxter (2004), Variance Reduction Techniques for Gradient Estimates in Reinforcement Learning.** Paper.
- **Chris Nota and Philip S. Thomas (2020), Is the Policy Gradient a Gradient?** Paper.
- **Wouter Kool, Herke van Hoof, and Max Welling (2019), Buy 4 REINFORCE Samples, Get a Baseline for Free!** Paper.
- **Volodymyr Mnih et al. (2016), Asynchronous Methods for Deep Reinforcement Learning.** Paper.
- **Timothy P. Lillicrap et al. (2016), Continuous Control with Deep Reinforcement Learning.** Paper.
- **Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine (2018), Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor.** Paper.
- **John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel (2016), High-Dimensional Continuous Control Using Generalized Advantage Estimation.** Paper.
- **Sham Kakade and John Langford (2002), Approximately Optimal Approximate Reinforcement Learning.** Paper.
- **Sham Kakade (2001), A Natural Policy Gradient.** Paper.
- **John Schulman, Sergey Levine, Philipp Moritz, Michael Jordan, and Pieter Abbeel (2015), Trust Region Policy Optimization.** Paper.
- **John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov (2017), Proximal Policy Optimization Algorithms.** Paper.
- **Yuhui Wang, Hao He, and Xiaoyang Tan (2020), Truly Proximal Policy Optimization.** Paper.
- **Logan Engstrom et al. (2020), Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO.** Paper.
- **Shengyi Huang et al. (2022), The 37 Implementation Details of Proximal Policy Optimization.** Article.

Planning and demonstrations (§§16–17)

- **Stéphane Ross, Geoffrey Gordon, and Drew Bagnell (2011), A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning.** Paper.
- **Michael Laskey, Jonathan Lee, Roy Fox, Anca Dragan, and Ken Goldberg (2017), DART: Noise Injection for Robust Imitation Learning.** Paper.
- **Andrew Y. Ng and Stuart Russell (2000), Algorithms for Inverse Reinforcement Learning.** Paper.
- **Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu (2020), Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems.** Paper.
- **Richard S. Sutton (1991), Dyna, an Integrated Architecture for Learning, Planning, and Reacting.** Paper.
- **Levente Kocsis and Csaba Szepesvári (2006), Bandit Based Monte-Carlo Planning.** Paper.
- **Rémi Coulom (2006), Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search.** Paper.
- **David Ha and Jürgen Schmidhuber (2018), World Models.** Paper.
- **Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap (2025), Mastering Diverse Control Tasks through World Models.** Paper.

In practice, and for language models (§§18–19)

- **Jack Clark and Dario Amodei (2016), Faulty Reward Functions in the Wild.** Article.
- **Victoria Krakovna et al. (2020), Specification Gaming: The Flip Side of AI Ingenuity.** Article.
- **Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov (2018), Exploration by Random Network Distillation.** Paper.
- **Adrien Ecoffet, Joost Huizinga, Joel Lehman, Kenneth O. Stanley, and Jeff Clune (2021), First Return, Then Explore.** Nature 590, 580–586. Paper; arXiv.
- **Peter Henderson et al. (2018), Deep Reinforcement Learning That Matters.** Paper.
- **Alex Irpan (2018), Deep Reinforcement Learning Doesn't Work Yet.** Article.
- **Long Ouyang et al. (2022), Training Language Models to Follow Instructions with Human Feedback.** Paper.
- **OpenAI (2022), Introducing ChatGPT.** Article.
- **OpenAI (2024), Learning to Reason with LLMs.** Article.
- **Zhihong Shao et al. (2024), DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models** (GRPO). Paper.
- **DeepSeek-AI (2025), DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning** (first version, January 2025). Paper.
- **Arash Ahmadian et al. (2024), Back to Basics: Revisiting REINFORCE-Style Optimization for Learning from Human Feedback in LLMs.** Paper.

Code and explanations

- **OpenAI (2018), Spinning Up in Deep RL.** Article.
- **Andrej Karpathy (2015), REINFORCEjs.** Demos.
- **Andrej Karpathy (2016), Deep Reinforcement Learning: Pong from Pixels.** Article.

Image credits

Pictures that are not our own drawings or runs. Each is used for teaching, with credit; where a license is given, it is the license stated by the source.

- **Figure 2a:** "People playing chess", photo by Nokedlyy (file page); CC0, outlines added.
- **Figures 2b and 5 (the robot hand):** OpenAI et al. (2019), arXiv:1910.07113, Fig. 1, top-left panel, at its native 640 × 360 pixels.
- **Figure 2c:** "Waymo Jaguar I-Pace Interior In Motion", photo by 9yz (file page); CC BY-SA 4.0, cropped, with outlines added (the figure's panel is shared under the same license).
- **Figure 20a:** Schultz (2016), Fig. 2, from the open-access article in PubMed Central, reproduced unmodified; CC BY-NC-ND 3.0. It reproduces Fig. 1 of Schultz, Dayan, and Montague (1997).
- **Figures 4, 39b, and 5 (Breakout):** frames of Breakout and Montezuma's Revenge rendered for these notes by the Arcade Learning Environment (Bellemare et al., 2013; GPL-2.0) through Gymnasium; the games are Atari's.
- **Figure 26a:** CartPole-v1 rendered for these notes by Gymnasium (Towers et al., 2024; MIT license), with the pushes and the track's limits added.
- **Figure 5, Samuel:** Arthur Samuel at the console of an IBM 7090, courtesy of IBM (IBM History, "The games that helped AI evolve"); © IBM.
- **Figure 5, MENACE:** a matchbox replica of MENACE, photo by Matthew Scroggs (file page); CC BY-SA 4.0.
- **Figure 5, backgammon:** photo by Ptkfgs (file page); public domain.
- **Figure 5, AlphaZero:** the board of the English Opening in Silver, Hubert, et al. (2017), arXiv:1712.01815, Table 2.
- **Figure 5, Dota 2:** Berner et al. (2019), arXiv:1912.06680, Fig. 11a; the game's imagery is Valve's.
- **Figure 5, the tokamak:** "TCV vue gen", photo by CRPP-EPFL, Association Suisse-Euratom (file page); CC BY-SA 2.5.
- **Figure 5, the drone:** Kaufmann et al. (2023), Nature 620, 982–987, Fig. 1b, from the copy on Wikimedia Commons; CC BY 4.0.
- **Figure 5, InstructGPT:** step 2 of Ouyang et al. (2022), arXiv:2203.02155, Fig. 2.
- **Figure 5, a reasoning model:** part of DeepSeek-AI (2025), arXiv:2501.12948v1, Table 3.
- **Figure 39a:** CoastRunners, the image of Clark and Amodei (2016), "Faulty reward functions in the wild"; © OpenAI, the game's art its developer's.

Appendix A: Derivations (optional)

A.1 The corridor in closed form

With the goal worth 0, the values of the three states under the policy p satisfy the Bellman equations (10) of §4:

$$v_1 = -1 + p\,v_2 + (1-p)\,v_1, \qquad v_2 = -1 + p\,v_1 + (1-p)\,v_3, \qquad v_3 = -1 + (1-p)\,v_2.$$

The first gives $v_1 = v_2 - 1/p$. Substituting it and the third into the second gives $p(1-p)\,v_2 = p - 3$, and then

$$J(p) = v_1 = -\frac{2(2-p)}{p(1-p)}.$$

Setting $J'(p) = 0$ gives $p^2 - 4p + 2 = 0$, whose root in $(0,1)$ is $p^\star = 2 - \sqrt{2}$, with $J(p^\star) = -(6 + 4\sqrt{2}) \approx -11.66$. At $p = \frac{1}{2}$, $v = (-12, -10, -6)$. The action values follow from one step and the values, by (9): $Q(s,a) = -1 + v(s')$, where s' is where a leads from s. The expected visits to each state per episode, $d = (6, 4, 2)$ at $p = \frac{1}{2}$, solve $d = e_1 + dP$, where P holds the probabilities of moving between the three states and $e_1 = (1, 0, 0)$ is the start; §14 and Appendix D use them.

A.2 The policy gradient theorem as a sum over states

§11 derived the theorem along a trajectory. Grouping the terms of (42) by the state gives the form of Sutton et al. (2000). With $d^\pi(s) = \sum_t \gamma^t \Pr(s_t = s)$, the discounted expected number of visits to s per episode,

$$\nabla_\theta J = \sum_s d^\pi(s) \sum_a Q^\pi(s,a)\,\nabla_\theta \pi_\theta(a \mid s),$$

since $\sum_a \pi_\theta(a \mid s)\,Q^\pi(s,a) \nabla_\theta \log \pi_\theta(a \mid s) = \sum_a Q^\pi(s,a) \nabla_\theta \pi_\theta(a \mid s)$. The visits d^π depend on θ, but no derivative of them appears. The same result follows from the Bellman equations alone. Differentiating $V^\pi(s) = \sum_a \pi_\theta(a \mid s) Q^\pi(s,a)$ (8), with $Q^\pi(s,a) = \sum_{s'} p(s' \mid s,a)[r(s,a,s') + \gamma V^\pi(s')]$ (9), gives

$$\nabla_\theta V^\pi(s) = \sum_a \nabla_\theta \pi_\theta(a \mid s)\,Q^\pi(s,a) + \gamma \sum_a \pi_\theta(a \mid s) \sum_{s'} p(s' \mid s,a)\,\nabla_\theta V^\pi(s'),$$

a Bellman equation for the gradient, whose "reward" in state s is $\sum_a \nabla_\theta \pi_\theta(a \mid s) Q^\pi(s,a)$. Unrolling it from the start state sums that reward over the states visited, weighted by γ^t, which is the formula above.

A.3 The discount that implementations drop

With $\gamma < 1$, the gradient (42) of the discounted return from the start carries a factor γ^t on step t's term, which down-weights late steps. Nearly every implementation drops it and keeps γ only inside the returns. Nota and Thomas (2020) showed that the direction so obtained is not, in general, the gradient of any function, and built an example in which it drives the policy toward the worst deterministic policy; in practice γ then serves to reduce the variance of long returns rather than to define the objective (5). In this lecture's policy-gradient tasks $\gamma = 1$ and the question does not arise.

A.4 The score of a Gaussian policy

For the Gaussian policy (34) of §11, $\log \pi_\theta(a \mid s) = -\big(a - \mu_\theta(s)\big)^2/2\sigma^2 - \log \sigma$ plus a constant, so the score of the mean is

$$\nabla_\theta \log \pi_\theta(a \mid s) = \frac{a - \mu_\theta(s)}{\sigma^2}\, \nabla_\theta \mu_\theta(s), \tag{66}$$

which moves the mean toward an action that earned a high reward and away from one that earned a low reward (Figure 27b). The score of $\log \sigma$ is $(a - \mu_\theta(s))^2/\sigma^2 - 1$: positive for an action far from the mean, so a far action that pays well widens the spread, and a near one narrows it.

Appendix B: Theory for §§1–10 (optional)

B.0 proves the shaping theorem of §1. B.1–B.6 prove the theorems of §§4 and 6 for a finite MDP with $\gamma < 1$, and B.6 says what changes for $\gamma = 1$. B.7 states when learning from samples converges (§§7 and 9), and B.8 gives two variants of SARSA and Q-learning (§§9–10).

B.0 Reward shaping

§1 noted that a bonus added to a sparse reward can change which policy is best, so that the agent pursues the bonus instead of the goal. One kind of bonus never does.

Theorem 8 (potential-based shaping). *Let Φ be any function of the state, with $\Phi = 0$ at terminal states, and add to every reward the term $\gamma\Phi(s_{t+1}) - \Phi(s_t)$. Then every policy that is optimal for the new rewards is optimal for the original ones, and conversely* (Ng et al., 1999).

The reason is that the added terms telescope. Along any trajectory from s_0 to the end of an episode, their discounted sum is $\sum_t \gamma^t(\gamma\Phi(s_{t+1}) - \Phi(s_t)) = -\Phi(s_0)$, so every return (3) from s_0 changes by the same amount, whichever actions are taken, and the ranking of policies is unchanged. A natural potential is minus the distance to the goal, which rewards each step that brings the agent closer.

B.1 Two operators

Write values as vectors over the states, $V \le U$ when $V(s) \le U(s)$ in every state, and $\|V\| = \max_s |V(s)|$. For a policy π, the *Bellman operator* $\mathcal{T}^\pi$ maps values to their one-step backup under π, and the *optimality operator* $\mathcal{T}^\star$ to the best one-step backup:

$$(\mathcal{T}^\pi V)(s) = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a)\big[r(s, a, s') + \gamma V(s')\big],$$
$$(\mathcal{T}^\star V)(s) = \max_a \sum_{s'} p(s' \mid s, a)\big[r(s, a, s') + \gamma V(s')\big].$$

The Bellman equation (10) says $V^\pi = \mathcal{T}^\pi V^\pi$, and the optimality equation (15) says $V = \mathcal{T}^\star V$. One sweep of policy evaluation (16) is $V_{k+1} = \mathcal{T}^\pi V_k$, and one sweep of value iteration (18) is $V_{k+1} = \mathcal{T}^\star V_k$. Both operators have two properties.

1. *Monotone:* if $V \le U$, then $\mathcal{T}^\pi V \le \mathcal{T}^\pi U$ and $\mathcal{T}^\star V \le \mathcal{T}^\star U$, since the backups weight V and U by the same nonnegative probabilities, and a maximum of larger numbers is larger.
2. *Contractions:* $\|\mathcal{T}^\pi V - \mathcal{T}^\pi U\| \le \gamma\|V - U\|$, and the same for $\mathcal{T}^\star$. For $\mathcal{T}^\pi$, the rewards cancel, and $(\mathcal{T}^\pi V - \mathcal{T}^\pi U)(s) = \gamma \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a)(V(s') - U(s'))$ is γ times an average of differences, none larger than $\|V - U\|$. For $\mathcal{T}^\star$, apply the same bound to each action's backup and use $|\max_a f(a) - \max_a g(a)| \le \max_a |f(a) - g(a)|$.

B.2 Policy evaluation and value iteration converge (Theorems 2 and 5)

Let $\mathcal{T}$ be either operator. A contraction has at most one fixed point: if $V = \mathcal{T}V$ and $U = \mathcal{T}U$, then $\|V-U\| = \|\mathcal{T}V-\mathcal{T}U\| \le \gamma\|V-U\|$, so $V = U$. If V is a fixed point, the iterates $V_{k+1} = \mathcal{T}V_k$ approach it geometrically,

$$\|V_k - V\| = \|\mathcal{T}V_{k-1} - \mathcal{T}V\| \le \gamma\|V_{k-1} - V\| \le ... \le \gamma^k\|V_0 - V\|,$$

which is the bound in Theorems 2 and 5. A fixed point exists. For $\mathcal{T}^\pi$ it is $V^\pi = (I-\gamma P_\pi)^{-1}r_\pi$ (12). For $\mathcal{T}^\star$, successive iterates satisfy $\|V_{k+1} - V_k\| \le \gamma^k\|V_1 - V_0\|$, so they form a Cauchy sequence, whose limit $\bar{V}$ satisfies $\bar{V} = \mathcal{T}^\star\bar{V}$ because $\mathcal{T}^\star$ is continuous (this is Banach's fixed-point theorem). B.3 shows that $\bar{V} = V^\star$.

B.3 Optimal policies exist (Theorem 1)

Let $\bar{V}$ be the fixed point of $\mathcal{T}^\star$.

1. *No policy does better than $\bar{V}$.* For any policy, $V^\pi = \mathcal{T}^\pi V^\pi \le \mathcal{T}^\star V^\pi$, since a maximum over actions is at least the policy's average. Applying $\mathcal{T}^\star$ again and again and using monotonicity, $V^\pi \le \mathcal{T}^\star V^\pi \le (\mathcal{T}^\star)^2 V^\pi \le ...$, and the right side converges to $\bar{V}$ by B.2. So $V^\pi \le \bar{V}$ in every state.
2. *One policy reaches $\bar{V}$ everywhere.* Let $\bar{\pi}$ take, in each state, an action that attains the maximum in $\mathcal{T}^\star\bar{V}$. Then $\mathcal{T}^{\bar{\pi}}\bar{V} = \mathcal{T}^\star\bar{V} = \bar{V}$: $\bar{V}$ is a fixed point of $\mathcal{T}^{\bar{\pi}}$, whose only fixed point is $V^{\bar{\pi}}$. So $V^{\bar{\pi}} = \bar{V}$.

Together, $\max_\pi V^\pi(s) = \bar{V}(s)$ in every state, and the deterministic policy $\bar{\pi}$ attains all these maxima at once. So $V^\star = \bar{V}$ satisfies (15), and is its only solution. Defining $Q^\star(s,a) = \sum_{s'} p(s' \mid s,a)[r(s,a,s') + \gamma V^\star(s')]$, the greedy actions with respect to $Q^\star$ are exactly those that attain the maximum in $\mathcal{T}^\star V^\star$, so every policy greedy with respect to $Q^\star$ is optimal; and $Q^\pi \le Q^\star$ for every π follows from $V^\pi \le V^\star$ by (9).

B.4 Policy improvement (Theorem 3)

Let π' be greedy with respect to V^π, as in (17). Then $\mathcal{T}^{\pi'}V^\pi = \mathcal{T}^\star V^\pi \ge \mathcal{T}^\pi V^\pi = V^\pi$. By monotonicity,

$$V^\pi \le \mathcal{T}^{\pi'}V^\pi \le (\mathcal{T}^{\pi'})^2 V^\pi \le \cdots \longrightarrow V^{\pi'},$$

so $V^{\pi'} \ge \mathcal{T}^{\pi'}V^\pi \ge V^\pi$. Each application of $\mathcal{T}^{\pi'}$ follows π' for one more step: taking the greedy action once and then following π is at least as good as following π throughout. If $V^{\pi'} = V^\pi$, then $V^\pi = \mathcal{T}^{\pi'}V^{\pi'} = \mathcal{T}^{\pi'}V^\pi = \mathcal{T}^\star V^\pi$, so V^π is the fixed point of $\mathcal{T}^\star$, which is $V^\star$ by B.3.

B.5 Policy iteration (Theorem 4)

Policy iteration changes the action in a state only when another action is strictly better, $Q^{\pi_k}(s,a) > Q^{\pi_k}(s,\pi_k(s)) = V^{\pi_k}(s)$. If the policy changes, then $\mathcal{T}^{\pi_{k+1}}V^{\pi_k} \ge V^{\pi_k}$ with strict inequality in the states that changed, and by B.4, $V^{\pi_{k+1}} \ge \mathcal{T}^{\pi_{k+1}}V^{\pi_k}$. So the values never decrease, and increase strictly in some state. No policy can therefore occur twice, and since there are finitely many deterministic policies, $|\mathcal{A}|^{|\mathcal{S}|}$, policy iteration stops. It stops when no action is strictly better than the current one in any state, that is, when $\mathcal{T}^\star V^{\pi_k} = \mathcal{T}^{\pi_k}V^{\pi_k} = V^{\pi_k}$. Then V^{π_k} solves the optimality equation (15), and π_k is optimal by B.3. This is why the stopping point of policy iteration is optimal: a policy that is greedy with respect to its own values satisfies the Bellman optimality equation.

B.6 Without discounting

With $\gamma = 1$, as in the corridor and the 4×3 world, the operators are not contractions in $\|\cdot\|$. If every policy reaches a terminal state with probability 1, they are contractions in a weighted maximum norm, $\max_s |V(s)|/\xi(s)$ for suitable weights $\xi > 0$, and all the arguments above go through with it. Neither of our examples meets that condition: "always right" in the corridor and "left" in every cell of the 4×3 world's first column never end an episode. Both meet the weaker condition of a *stochastic shortest path* problem: some policy ends every episode with probability 1, and every policy that may not end has value $-\infty$ from some state, because every step costs something. Under that condition the optimality equation (15) still has $V^\star$ as its only solution, value iteration and policy iteration (started from a policy that ends) still converge, and a greedy policy is still optimal (Bertsekas and Tsitsiklis, 1996, Chapter 2). The contraction factor of a policy that ends can be close to 1, which is why "always up" in the 4×3 world needs 703 sweeps.

B.7 Convergence from samples

The methods of §§7 and 9 replace the expectation in a backup by one sample, and with a constant step size their estimates keep moving with the noise of the latest samples (Figure 19). To converge, the step sizes must shrink, but not too fast. The conditions are those of Robbins and Monro (1951) for stochastic approximation:

$$\sum_{n=1}^{\infty} \eta_n = \infty, \qquad \sum_{n=1}^{\infty} \eta_n^2 < \infty. \tag{67}$$

The first lets the estimates travel any distance from where they started; the second makes the noise average out. The steps $\eta_n = 1/n$ satisfy both, and so do $\eta_n = n^{-0.7}$; a constant step satisfies only the first.

Theorem 9 (convergence of tabular TD). *In a finite MDP with bounded rewards, and $\gamma < 1$ or episodes that end with probability 1, suppose every state is visited infinitely often, and the step size used for the nth update of each state satisfies (67). Then the TD(0) estimates (24) converge to V^π with probability 1.*

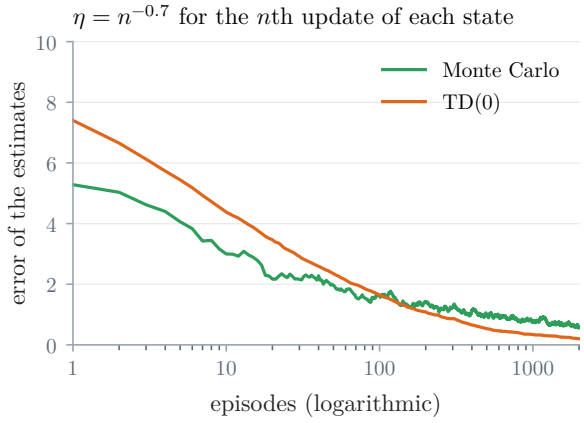


Figure 45. With decreasing steps, both Monte Carlo and TD converge. As Figure 19, with the step size $\eta = n^{-0.7}$ for the nth update of each state, for 2,000 episodes: every-visit Monte Carlo (green) and TD(0) (orange), the root mean square error over the three states, averaged over 100 runs.

In expectation, a TD update moves $V(s)$ a fraction η of the way toward its backup in iterative policy evaluation (16), which converges to V^π by Theorem 2, and the theorem says that the noise

of sampling one transition in place of the expectation does not prevent convergence (Jaakkola et al., 1994). Monte Carlo with the same step sizes converges as well. In Figure 45, with $\eta_n = n^{-0.7}$, the errors after 2,000 episodes are 0.56 for Monte Carlo and 0.20 for TD, and still falling.

The two methods converge to the same values, but on a finite batch of experience they disagree. If the same episodes are replayed until the estimates stop changing, Monte Carlo gives the average return from each state, while TD gives the values of the Markov model that best fits the data, with transition probabilities estimated by counting (Sutton and Barto, 2018, Section 6.3). When the problem really is Markov, the second uses the data better, which is one reason TD usually learns faster in practice.

The control methods of §9 converge under the same conditions on the steps, and SARSA needs one more, on its exploration.

Theorem 10 (convergence of SARSA). *In a finite MDP with bounded rewards, and $\gamma < 1$ or episodes that end with probability 1, suppose the step sizes for each state and action satisfy (67), every action is taken infinitely often in every state, and the policy becomes greedy with respect to Q in the limit (for example, ε-greedy with $\varepsilon = 1/n(s)$ in a state visited $n(s)$ times). Then the estimates of SARSA (29) converge to $Q^\star$, and its policy to an optimal policy, with probability 1.*

This is a result of Singh et al. (2000). A policy with the two properties of the theorem, exploring forever but less and less often, is called *greedy in the limit with infinite exploration*.

Theorem 11 (convergence of Q-learning). *In a finite MDP with bounded rewards, and $\gamma < 1$ or episodes that end with probability 1, suppose every state and action is updated infinitely often, and the step sizes for each of them satisfy (67). Then the estimates of Q-learning (30) converge to $Q^\star$ with probability 1, whatever policy chose the actions.*

Watkins and Dayan (1992) gave the proof, and Jaakkola et al. (1994) a shorter one by stochastic approximation, as for Theorem 9. Unlike SARSA, Q-learning does not need its exploration to fade: an agent can explore with $\varepsilon = 0.1$ forever and still learn the values of the policy that never explores. And since the theorem allows any behavior, Q-learning can learn from stored transitions, which deep Q-networks use (§10).

B.8 Variants of SARSA and Q-learning

Expected SARSA replaces the sampled next action by an average over the policy's actions,

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \eta \Big[r_t + \gamma \sum_{a'} \pi(a' \mid s_{t+1})\, Q(s_{t+1}, a') - Q(s_t, a_t) \Big]. \tag{68}$$

It removes the noise of that choice, and with a greedy π it is Q-learning (30).

The maximum in Q-learning also has a bias. The estimates are noisy, and the largest of several noisy estimates is, on average, larger than the largest of the true values. So Q-learning tends to *overestimate* action values, most where several actions have similar values. **Double Q-learning** (van Hasselt, 2010) keeps two tables, Q^A and Q^B, and at each step updates one of them at random, using it to choose the next action and the other to evaluate that action:

$$Q^A(s_t, a_t) \leftarrow Q^A(s_t, a_t) + \eta \Big[r_t + \gamma\, Q^B\big(s_{t+1}, \arg\max_{a'} Q^A(s_{t+1}, a')\big) - Q^A(s_t, a_t) \Big]. \tag{69}$$

The two tables are trained on different samples, so their errors are nearly independent, and the action chosen for its high Q^A is not selected for a high error in Q^B. Double DQN (§10) uses the same idea in deep Q-networks.

Appendix C: The best baseline, and our chat model (optional)

For one parameter, with $g = \frac{d}{d\theta} \log \pi_\theta(a)$ the score and G the return, the estimate $(G - b)g$ has a mean that does not depend on b, so its variance is smallest where its second moment is:

$$\mathbb{E}[(G-b)^2 g^2] = \mathbb{E}[G^2 g^2] - 2b\,\mathbb{E}[G g^2] + b^2\,\mathbb{E}[g^2], \qquad \text{minimized at} \qquad b^\star = \frac{\mathbb{E}[G\,g^2]}{\mathbb{E}[g^2]}$$

(Weaver and Tao, 2001; Greensmith et al., 2004): an average of the returns weighted by the squared score. It equals the mean return when the returns and the squared scores are uncorrelated, and it moves toward the returns of the actions with large scores, the unlikely ones. For a vector of parameters each component has its own $b^\star$; with one b for all, the same calculation with $\|g\|^2$ in place of g^2 gives the best common one. In the corridor it helps little more than the average return.

Why the baseline made no difference on our chat model (§19; without it the runs reached 84% greedy and 64% sampled). Two measurements say why.

1. **The rewards are already nearly centered.** About half the sampled answers are right, so a reward of 0 or 1 is not far from $r - \frac{1}{2}$, and there is little common offset for a baseline to remove.
2. **The wrong answers carry the larger scores.** A wrong answer is a less likely one, so its score, the gradient of its log-probability, is larger: on 384 sampled answers its squared size is 5.7 times that of a right answer, on average. The best constant baseline, $b^\star$ with $\|g\|^2$, is then 0.14, far below the mean reward, 0.48. Subtracting the mean puts weight on the wrong answers' large scores and adds noise of its own.

What a bad offset costs is visible all the same. Raising every reward by 1 leaves the expected gradient unchanged and nearly doubles the size of the estimate from one batch, from 9.6 to 18.1. And the estimates are noisy in any form: the gradients of two independent batches of 64 answers point in nearly unrelated directions, with a cosine of 0.13 without a baseline and 0.07 with one, lower, as the second point predicts (mean of 32 pairs). Two of the base model's pretraining batches had a cosine of 0.30 (optimizer tutorial, Section 2). Most of each step is noise, and progress comes from many steps.

Appendix D: Trust regions, and PPO in practice (optional)

D.1 The performance difference identity

For two policies π and π' with the same start, Kakade and Langford (2002) showed that

$$J(\pi') - J(\pi) = \mathbb{E}_{\tau\sim\pi'}\Big[\sum_t \gamma^t A^\pi(s_t, a_t)\Big] :$$

the improvement is the old policy's advantage of the new policy's actions, summed over the states the *new* policy visits. Nothing is approximated. By (47), $A^\pi(s_t, a_t)$ is the expected value of $r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)$ given s_t and a_t, so along trajectories of π',

$$\mathbb{E}_{\pi'}\Big[\sum_t \gamma^t A^\pi(s_t, a_t)\Big] = \mathbb{E}_{\pi'}\Big[\sum_t \gamma^t r_t\Big] + \mathbb{E}_{\pi'}\Big[\sum_t (\gamma^{t+1} V^\pi(s_{t+1}) - \gamma^t V^\pi(s_t))\Big] = J(\pi') - V^\pi(s_0),$$

because the second sum telescopes to $-V^\pi(s_0)$, with $V^\pi = 0$ at the terminal state; and $V^\pi(s_0) = J(\pi)$. Grouping by states, with $\gamma = 1$, gives (55). The surrogate of §14 (56) is this identity with the new policy's visits replaced by the old one's, which is why it is accurate near π and not far from it. In the corridor at $p' = 0.9$, the identity with the new visits, $(12.2, 11.1, 1.1)$, reproduces the true change, -12.44, exactly.

D.2 TRPO and the natural gradient

TRPO (Schulman et al., 2015) maximizes the surrogate subject to a bound κ on the average KL divergence between the old and new action distributions (57). To second order the KL divergence is $\frac{1}{2}\Delta\theta^\top F \Delta\theta$, with F the Fisher information matrix of the policy, so the step is along the **natural gradient** $F^{-1}\nabla_\theta J$ (Kakade, 2001), scaled to the bound, computed by conjugate gradients and checked by a line search. The bound on the change of the policy controls the change of the visited states, and with it the surrogate's error.

D.3 PPO in practice

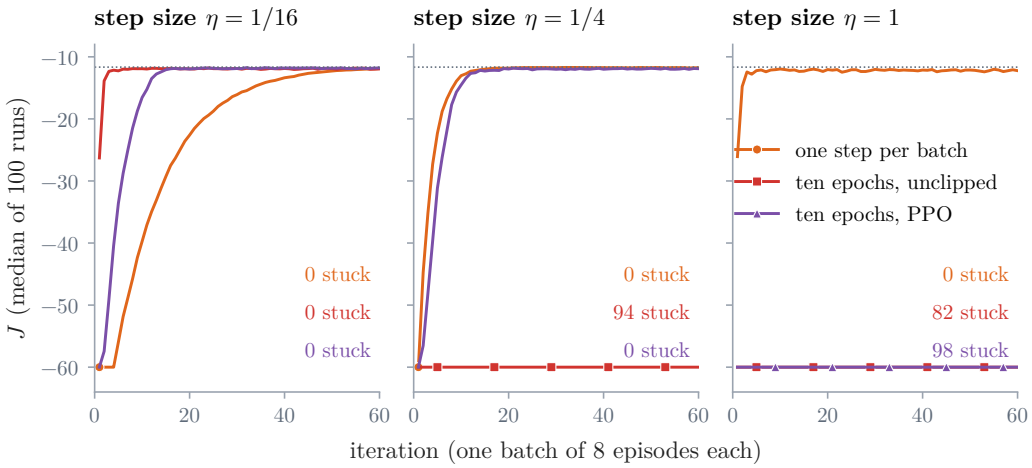


Figure 46. Ten epochs per batch without clipping run away at a moderate step; with clipping they are safe at moderate steps, but not at a large one. The short corridor from $p = 0.05$, 100 runs of 60 iterations; each iteration samples 8 episodes, estimates advantages with a table of state values learned from the returns, normalizes them, and takes gradient steps of size η on the policy's parameter: one step per batch (orange), ten steps on the unclipped surrogate (red; equation (53)), or ten steps on PPO's objective (purple; equation (58), with $\epsilon = 0.2$). The median over the runs of the exact J after each iteration (values below -60 drawn at -60), and the number of runs that end stuck, at $p < 0.01$ or $p > 0.99$.

In the corridor, Figure 46 compares three ways to use each batch of 8 episodes, at three step sizes. The table gives the iteration at which the median run first comes within 0.5 of the best return, and how many of the 100 runs end within 0.5 of it, or how many end stuck at a nearly deterministic policy.

step size η	**one step per batch**	**ten unclipped epochs**	**ten PPO epochs**
$\frac{1}{16}$	iteration 52; 81 runs	iteration 3; 62 runs	iteration 14; 79 runs
$\frac{1}{4}$	iteration 13; 83 runs	94 runs stuck	iteration 18; 76 runs
1	iteration 5; 46 runs	82 runs stuck	98 runs stuck

Reusing a batch without clipping learns fast at a small step and collapses at a moderate one, because the policy moves far from the one that collected the data: at $\eta = \frac{1}{16}$, the largest ratio in the median run is 4.25 without clipping and 1.32 with it. With clipping, ten epochs per batch are safe at both small and moderate steps. At $\eta = 1$ PPO fails too: clipping limits what each sample asks for, not the size of a gradient step, and ten large steps on one batch carry the policy too far. With one parameter, one larger step per batch does about as well as ten clipped

ones. PPO is for large problems, where collecting experience costs far more than an update, and reusing each batch safely is what saves the cost.

Clipping does not keep every ratio inside $[1-\epsilon, 1+\epsilon]$. All samples share the parameters, so a step taken for some samples moves the others, and one large step can carry a ratio far past the boundary before the clipping takes effect. Studies of PPO agents found that the ratios are not bounded in practice (Wang et al., 2020; Engstrom et al., 2020), while the average KL divergence from the old policy stays small (Engstrom et al., 2020). Implementations therefore also stop a batch's epochs early when the measured KL divergence exceeds a target. PPO's published results also depend on details beyond its objective, such as normalized advantages and observations, a clipped gradient norm, and the initialization: Engstrom et al. (2020) found that such details account for much of PPO's advantage over TRPO, and Huang et al. (2022) list 37 of them in the reference implementation.

Appendix E: Exploration and regret (optional)

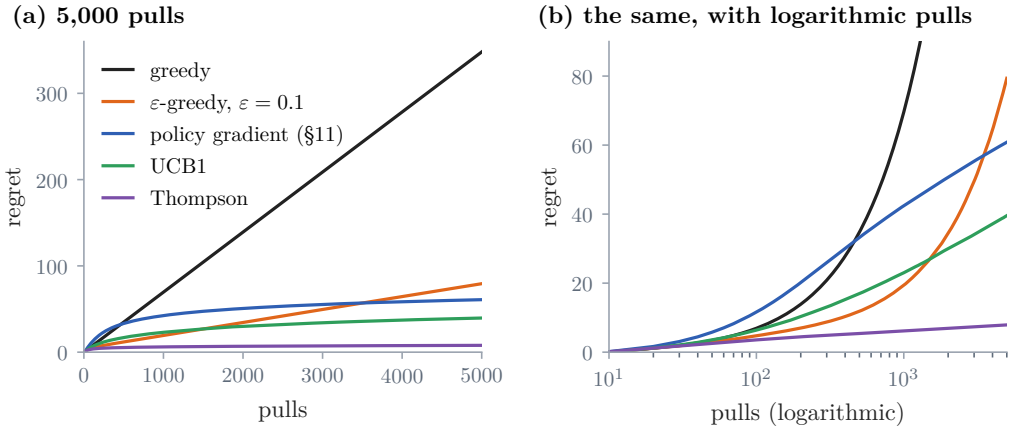


Figure 47. Optimism and Thompson sampling lose ever less per pull; greedy and ε-greedy keep losing. The two-armed bandit of Figure 22: the regret, the expected reward lost against always pulling the better arm, summed over the pulls so far, averaged over 1,000 runs. (a) Over 5,000 pulls. (b) The same runs with logarithmic pulls, the regret axis cut at 90; a straight line here is regret that grows with the logarithm of the pulls.

How well a rule explores is measured by its **regret** after n pulls: the reward it lost, in expectation, against always pulling the best arm,

$$\text{regret}(n) = n\,\mu^\star - \mathbb{E}\Big[\sum_{t=1}^{n} r_t\Big], \tag{70}$$

where $\mu^\star$ is the largest mean. Figure 47 compares five rules on the bandit of §8. Greedy locks onto the worse arm in about a fifth of its runs and then loses 0.3 on every pull, so its regret grows in proportion to the pulls: 349 after 5,000. ε-greedy with a fixed ε also loses a fixed amount per pull, through its random pulls of the worse arm, and reaches 79. The policy gradient's random policy sharpens as it learns, and its regret, 61, grows ever more slowly. UCB1 and Thompson sampling spend ever fewer pulls on the worse arm, and their regret grows only with the logarithm of the pulls: 40 for UCB1 and 8 for Thompson sampling after 5,000.

The two rules that do best are provably good: UCB1's regret is at most of order $\log n$ (Auer et al., 2002), and Thompson sampling's too (Agrawal and Goyal, 2012). No algorithm can do better than logarithmic regret on every bandit: Lai and Robbins (1985) proved a lower bound of

that order, with a constant set by how hard the arms are to tell apart. Lattimore and Szepesvári (2020) develop the theory in full.

Appendix F: A longer history (optional)

Trial and error. Edward Thorndike put hungry cats in puzzle boxes, cages that opened when the cat pressed a lever or pulled a loop, and timed how long each escape took. The times fell over the trials, gradually rather than all at once, and he concluded that an action followed by satisfaction becomes more likely in the same situation, the *law of effect* (Thorndike, 1898); he named it in 1911. Behaviorist psychology studied this kind of learning, in rats and pigeons, for the next half century. The first machines that learned by trial and error appeared soon after computers did. Arthur Samuel's checkers program (Samuel, 1959) played against itself and adjusted its evaluation of a position toward the evaluation found by searching ahead from it, an early form of the TD update of §7 (24).

MENACE. In 1961 Donald Michie built a learning machine out of matchboxes (Michie, 1963). The Matchbox Educable Noughts And Crosses Engine had 304 matchboxes, one for each position it could face (counting rotations and reflections as one), each holding colored beads, one color per possible move. To choose a move, its operator drew a bead at random from the box for the current position. After a game, the beads drawn were rewarded: more beads of the same color were added after a win (three for each bead drawn), one more after a draw, and the drawn beads were removed after a loss. The proportions of beads are a policy, a table of probabilities for each state, and the bead counts change in proportion to the result, much as REINFORCE changes probabilities in proportion to the return. In Michie's first tournament against it, 220 games over two days, MENACE was easily beaten at first and drew consistently after its twentieth game, as Martin Gardner reported in *Scientific American* in 1962.

Optimal control. Richard Bellman's dynamic programming (Bellman, 1957) treated sequential decisions under uncertainty as Markov decision processes and gave the recursion for their values. Ronald Howard's policy iteration followed in 1960. This thread assumed a known model, and for decades it developed apart from learning.

Temporal differences and the modern field. Harry Klopf argued in the 1970s that learning systems should seek reward rather than only fit examples, and Andrew Barto and Richard Sutton took up the idea at the University of Massachusetts. Their actor–critic of 1983 learned to balance a pole on a cart (Barto et al., 1983); Sutton's temporal-difference learning followed in 1988, and Christopher Watkins' Q-learning in 1989 joined it to Bellman's equations. Ronald Williams' REINFORCE (1992) began the policy-gradient line, and Gerald Tesauro's TD-Gammon (1992–1995) showed what TD learning with a neural network could do. Sutton and Barto's textbook appeared in 1998, and its second edition in 2018.

Deep reinforcement learning. DQN learned Atari games from pixels in 2013, and its Nature paper followed in 2015. AlphaGo beat the European champion Fan Hui 5–0 in October 2015 and Lee Sedol 4–1 in March 2016; AlphaGo Zero and AlphaZero (2017) learned from the rules alone. OpenAI Five beat the world champions of Dota 2, and AlphaStar reached the top level of StarCraft II, both in 2019. Reinforcement learning also left games: a robot hand turned a puzzle cube (2019), a controller shaped the plasma of a tokamak (2022), and a drone beat champion human pilots in races (2023).

Language models. Learning rewards from people's comparisons (Christiano et al., 2017) was applied to language models for style and summaries in 2019–2020 (Ziegler et al., 2019; Stiennon et al., 2020) and for following instructions in InstructGPT (Ouyang et al., 2022), which led to ChatGPT. Since 2024, reasoning models such as OpenAI's o1 and DeepSeek-R1 have been

trained with rewards from answers a program can check (§19). Barto and Sutton received the 2024 Turing Award.

Appendix G: Our runs in detail (optional)

The gridworlds (§§4, 6, 9, 17). *The 4 × 3 world:* value iteration with $\gamma = 1$, from $V = 0$ in every cell but the exits, which hold their rewards (the same as giving the step into an exit its reward and the exit the value 0), stopped when no value changes by more than 10^{-10} (40 sweeps); $Q^\star$ from $V^\star$ by one backup. Policy iteration from "up" in every cell evaluates each policy exactly, by solving the 9 linear equations of its non-terminal cells, and changes a cell's action only when another action is strictly better, so that ties cannot make it cycle; iterative evaluation of "up" from $V = 0$ needs 703 sweeps to come within 10^{-6} of the exact values. *Cliff walking:* 200 runs of 500 episodes each for SARSA and Q-learning, from $Q = 0$, ε-greedy with $\varepsilon = 0.1$ and step 0.5, $\gamma = 1$; the paths of Figure 24a are the greedy paths of the first run's final values. *Behavior cloning:* a 7 × 7 grid, a reward of +1 at the goal, −1 in a pit (both end the episode), and −0.02 per step; the expert is the optimal policy under slips, by value iteration with $\gamma = 0.99$; episodes are cut at 60 steps. Each labeled state is a (state, expert action) pair, and the learner takes the most frequent label of each state it has seen and a random action elsewhere. Demonstrations are whole expert episodes, recorded with slips (or, for the dashed line of Figure 37b, without), cut at the budget. DAgger spends half its labels on demonstrations, with slips (or, for its dashed line, without), and the rest in rounds that run the current learner and label the states it visits; the learner's own runs always slip. 20 seeds per budget, each policy scored on 2,000 episodes; the runs of DAgger from demonstrations without slips use a random stream of their own.

The bandit (§8): Bernoulli arms with means 0.7 and 0.4. Figure 22: 1,000 runs of 500 steps. Figure 47: 1,000 runs of 5,000 pulls; greedy and ε-greedy pull each arm once first and break ties at random; UCB1 pulls each arm once and then the arm with the largest $\hat{Q}(a) + \sqrt{2 \ln t / n_a}$; Thompson sampling uses Beta(1, 1) priors; the gradient bandit takes steps of 0.1 with the average reward so far as its baseline, from $p = \frac{1}{2}$. The regret is computed from the means, 0.3 for each pull of the worse arm. At 1,000 pulls it is 70 for greedy, 20 for ε-greedy, 43 for the gradient bandit, 23 for UCB1, and 6 for Thompson sampling.

Small computations for §§6 and 7. *Value iteration's greedy policy* (Figure 15): after each sweep of value iteration in the 4 × 3 world, the policy greedy with respect to V_k, with ties broken toward the first action (up, right, down, left), and its exact value at the start, by solving its linear equations. *Monte Carlo by hand* (Figures 16 and 18): three episodes of the corridor at $p = \frac{1}{2}$ from one random seed; in Figure 18, TD(0) and every-visit Monte Carlo with $\eta = \frac{1}{2}$ on the first of them, from $V = 0$. *Decreasing steps* (Figure 45): 100 runs of 2,000 episodes at $p = \frac{1}{2}$, every-visit Monte Carlo and TD(0) from $V = 0$, each with the step $n^{-0.7}$ for the nth update of a state. *Dopamine* (Figure 20b): TD(0) on trials of 25 steps, one state per step since the cue (a "complete serial compound"), a cue at step 5 whose value before it is fixed at 0, a reward of 1 at step 15, $\gamma = 1$, step 0.2, 200 trials, then one trial without the reward.

CartPole (§10). The state is four numbers: the cart's position and velocity, and the pole's angle and angular velocity. The two actions push the cart left or right. Every step the pole stays up earns +1, and an episode ends when the pole falls past 12° or the cart leaves the track, or after 500 steps. The network has two hidden layers of 64 units and one output per action. It acts ε-greedily, with ε falling from 1 to 0.05 over the first 10,000 steps, and learns from a replay memory of 50,000 transitions in minibatches of 64, with targets from a copy refreshed every 500 updates. Each run takes 60,000 steps, about 40 seconds on one processor core. We

ran it 10 times, and 10 times each without the target network and without the replay memory (Figure 26).

With both devices, the network learns to balance the pole quickly. Nine of the 10 runs balanced it for the full 500 steps at some evaluation, the first after 4,000 steps, and the median over the runs reached about 400 steps at 12,000. Then performance fell back. Single runs swung between about 100 and 500 steps for the rest of training, and the median stayed between 100 and 200.

The failures were of one kind. After the first 10,000 steps, 90% of the greedy policy's failed episodes ended with the cart running off the track, not with the pole falling. The pole falls within a few steps of a bad push, but the cart drifts off the track over a hundred steps or more, and with $\gamma = 0.99$ the return (3) gives a failure 100 steps away a weight of $0.99^{100} \approx 0.37$, about a third of the weight of one now. Panel (c) shows a second reason: the values are far from converged. The value of the start rises slowly, to 75 by the end, against the 99.3 that balancing for 500 steps is worth. Each refresh of the target network lets the values look about one step further ahead, as each sweep of value iteration did (§6), and a run of 60,000 steps has 118 refreshes.

Without the target network, every run diverged. The values grew without limit, to about 8×10^7 by the end of each run, against a true value of at most 99.3, and the greedy policy pushed the same way every time, which drops the pole in about 9 steps. All three ingredients of the deadly triad are present, and in our runs the target network is what kept them in check. Without the replay memory, each update used only the latest transition, and training was erratic: seven runs balanced the pole for 500 steps at some evaluation, but none kept it, and after 10,000 steps the median evaluation was 63 steps, against 127 with the memory.

The implementation: the dynamics of Gym's CartPole-v1 (Euler steps of 0.02 s, a force of 10 N, a pole of half-length 0.5 m and mass 0.1 kg on a cart of 1 kg), written in NumPy; a failure ends the episode with a final reward of +1 and no bootstrapping, and an episode cut at 500 steps bootstraps from its last state. The network, $4 \to 64 \to 64 \to 2$ with ReLU and the state scaled by $(2.4, 3, 0.21, 3.5)$, is trained by Adam at 5×10^{-4} on the Huber loss of the TD error (its gradient clipped at 1 per sample), with the gradient's norm clipped at 10; learning starts after 1,000 steps, with one update per step. The variants change one thing each: without the target network, the targets use the network being trained; without the replay memory, each update uses only the latest transition; without either, both. Every 2,000 steps the greedy policy runs 5 episodes from random starts. 10 runs per variant, 60,000 steps each, 31 to 57 seconds on one core. For DQN: 9 runs reach 500 steps at some evaluation (the first at 4,000 steps; the tenth peaks at 365); the median over runs is 399 at 12,000 steps and 150 at the end (middle half 102 to 320); of the evaluation episodes after 10,000 steps that failed, 941 of 1,050 ended with the cart off the track. The median value of the start is 75.1 at the end. Without the target network, every run's greedy policy balances for 8 to 10 steps throughout, and the values of the start end between 6.9×10^7 and 8.7×10^7. Without the replay memory, 7 runs reach 500 steps at some evaluation and none stays there; the median of the evaluations after 10,000 steps is 63 steps, against 127 for DQN. Without either, every run behaves like the runs without a target network.

REINFORCE and actor–critic in the corridor (§§11, 13): 100 runs of 1,000 episodes per step size from $p = 0.05$, episodes cut at 1,000 steps. For each method and step size: the runs with an episode cut, the runs stuck at the end ($p < 0.01$ or $p > 0.99$), the runs within 0.5 of the best return at the end, and the episode at which the median run first comes within 0.5 of the best return (— if it never does).

Step size η	**REINFORCE**	**with a baseline**	**actor–critic**
2^{-12}	0 / 0 / 39 / —	0 / 0 / 23 / —	0 / 0 / 1 / —
2^{-10}	0 / 0 / 95 / 249	0 / 0 / 99 / 263	0 / 0 / 100 / 245
2^{-8}	3 / 0 / 70 / 72	1 / 0 / 79 / 59	0 / 0 / 100 / 66
2^{-7}	15 / 2 / 48 / 35	9 / 3 / 70 / 37	0 / 0 / 100 / 39
2^{-6}	93 / 68 / 11 / —	34 / 20 / 42 / 21	0 / 0 / 100 / 25

The baseline is a learned constant, moved toward each return with step 2^{-6}; the critic is a table of three values with step 2^{-4}.

PPO in the corridor (§14, Appendix D.3): 100 runs of 60 iterations of 8 episodes from $p = 0.05$. For each update and step size: the runs stuck at the end, the runs within 0.5 of the best return at the end, the iteration at which the median run first comes within 0.5 of the best return, and the median over runs of the largest ratio of a batch's policies.

Step size η	**one step per batch**	**ten epochs, unclipped**	**ten epochs, PPO**
$\frac{1}{16}$	0 / 81 / 52 / 1.14	0 / 62 / 3 / 4.25	0 / 79 / 14 / 1.32
$\frac{1}{4}$	0 / 83 / 13 / 1.55	94 / 0 / — / —	0 / 76 / 18 / 1.71
1	0 / 46 / 5 / 3.87	82 / 0 / — / —	98 / 0 / — / 2.96

A ratio of — means the runs collapsed, and the ratio of a policy at $p \approx 0$ or 1 is not finite.

The chat model (§19, Appendix C): each run, 300 steps of 8 prompts with 8 answers each, Adam at 3×10^{-5} with $\beta = (0.9, 0.95)$, one processor core, 15 to 21 minutes. At the end, the accuracy of the most probable answer on the 200 held-out sums, of sampled answers (2 per sum), the share of sampled answers that end with `<|end|>`, and the KL divergence per answer from the fine-tuned model. The sum of Figure 41b, which none of 8 sampled answers got right before training, gets 5 of 8 right after it, and four-digit sums 0 of 800 (one run).

Run	**Most probable answer right**	**Sampled answers right**	**Ended**	**KL from the reference (nats per answer)**	**Minutes**
fine-tuned model, before training	73%	47%	99.8%	0	—
leave-one-out baseline, seed 0	83%	66%	100.0%	0.42	21
leave-one-out baseline, seed 1	82%	60%	99.8%	0.66	19
leave-one-out baseline, seed 2	83%	60%	99.8%	0.42	16
no baseline, seed 0	82%	62%	100.0%	0.56	16
no baseline, seed 1	86%	68%	100.0%	0.54	16
no baseline, seed 2	84%	64%	100.0%	0.42	15